Advanced Big Data Analytics

Assignment I: Smart Energy Grid Analytics

AUCA MSc | Issued 1 October 2026 | Due 15 October 2026, 23:59 CAT

Group member	Student ID
ERIC BYIRINGIRO	101389
SANDRINE MFURANZIMA	101411
NDANYUZWE DUNCAN	101413

All times are CAT (UTC+02:00). Measurements were taken on one machine (RSA06-B). Observations (what was measured) are kept separate from explanations (why we think it happened).

Contents

Contents....2
1 Build the ingestion pipeline....4
1.1 Confirm the environment....4
1.2 Create the database structures....8
1.3 Capture and publish a pilot stream....11
2 Capture and prepare the historical dataset....13
2.1 Capture and load the complete stream....13
2.2 Investigate data quality....15
2.3 Clean and preserve a consistent benchmark....16
3 Compare chunk intervals....18
3.1 Create and verify three configurations....18
3.2 Implement and time the five workloads....19
3.3 Explain the chunk results....24
4 Compare compression strategies....24
4.1 Create the two compressed configurations....24
4.2 Measure storage and compression cost....28
4.3 Repeat the query comparison....30
4.4 Select a storage configuration....35
5 Investigate the utility data....35
5.1 Identify demand patterns....35
5.2 Compare regions and customer types....38
5.3 Assess reporting quality....39
5.4 Investigate abnormal consumption....40
5.5 Management recommendation....42
6 Create and investigate continuous aggregates....42
6.1 Create the three aggregate views....42
6.2 Test freshness and delayed arrivals....48
7 Build and compare dashboards....51
7.1 Build the Raw dashboard....51
7.2 Build the Aggregate dashboard....52
7.3 Verify matching results....53
7.4 Measure performance and judge the tradeoff....54
8 Forecast the next day....56
8.1 Prepare chronological data and features....56
8.2 Evaluate the previous-day baseline....57
8.3 Train and evaluate one regression model....58
8.4 Judge forecast usefulness....59
9 Submit the report and code....61
9.1 Organize the report....61
9.2 Make the work reproducible....61
9.3 Document contributions....62

Appendix A: dashboard panel queries.....63
Raw dashboard.....63
Aggregate dashboard.....63

1 Build the ingestion pipeline

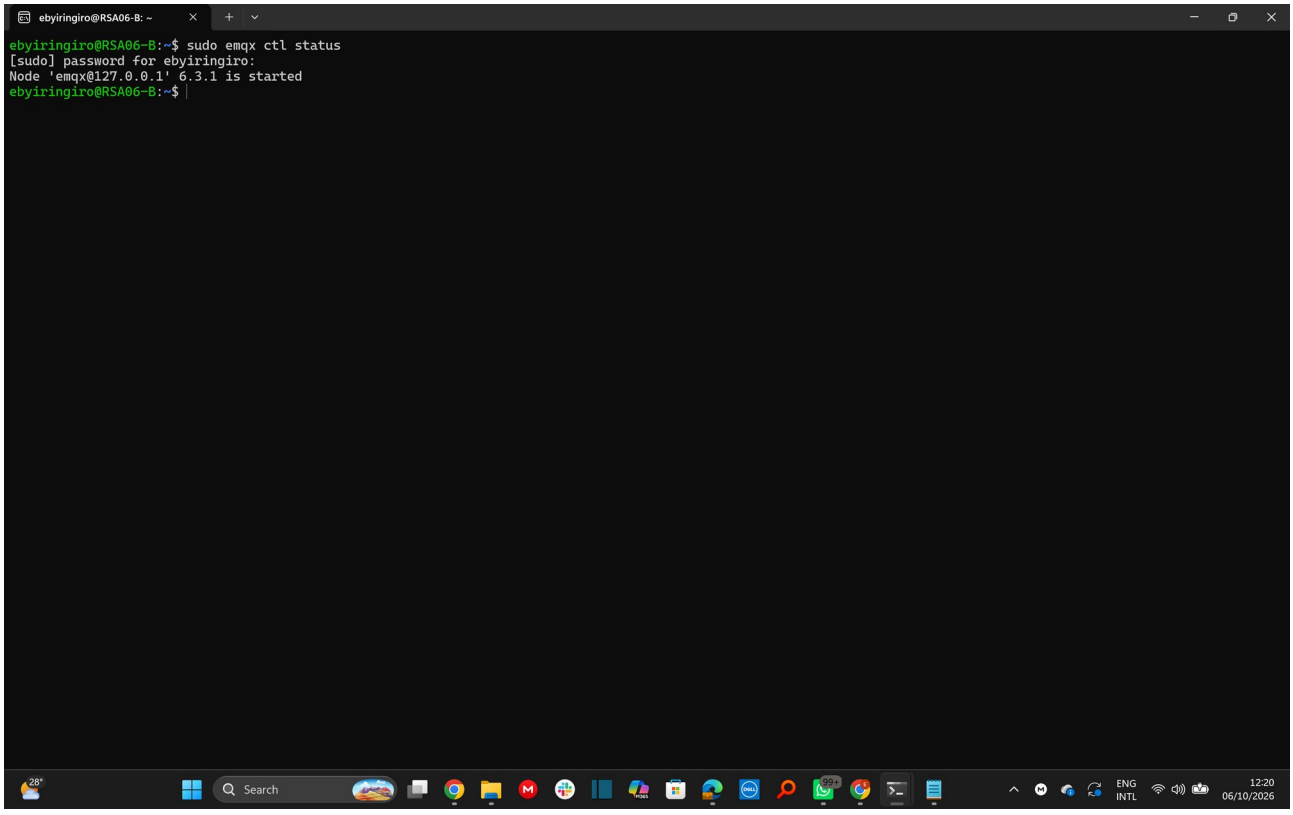
1.1 Confirm the environment

Item	Value
Machine	RSA06-B (Windows 11 Pro, build 26200)
CPU	Intel Core Ultra 9 185H, 16 cores / 22 logical processors
RAM	31.5 GB
Storage	KIOXIA BG6 1 TB NVMe SSD
Python	3.13.6 (psycopg 3.3.6, paho-mqtt 2.x)
EMQX	6.3.1, running in WSL2 (broker reached on 127.0.0.1:1883)
PostgreSQL	18.6, native Windows service postgresql-x64-18
TimescaleDB	2.29.2 (Community), enabled in database smart_meters
shared_buffers	8,053 MB (1,030,784 x 8 kB)
work_mem	11,713 kB
max_parallel_workers_per_gather	11
jit	off
Database / session time zone	smart_meters / Africa/Kigali (set with ALTER DATABASE); daily buckets use 'Africa/Kigali' so they align to CAT midnight

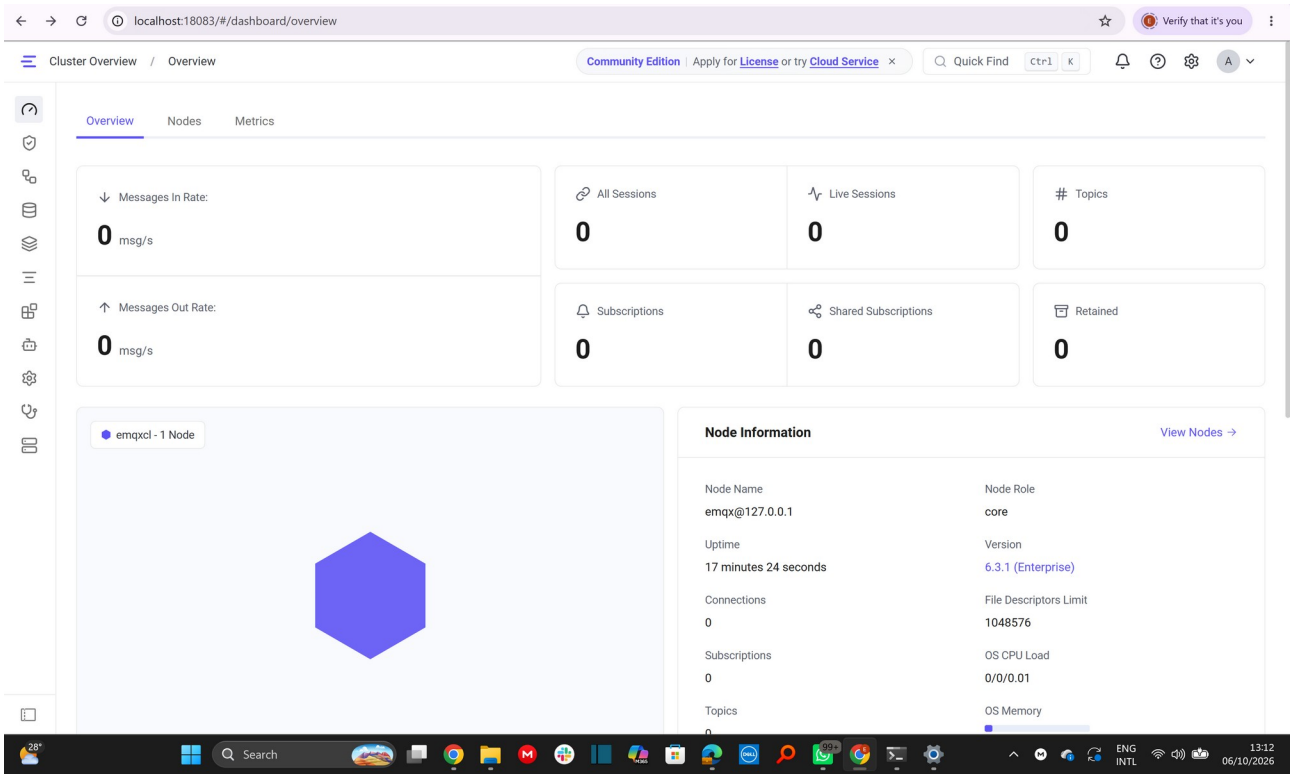
PostgreSQL runs natively on Windows, not in a container or virtual machine, so no CPU or RAM limit applies beyond the machine itself. EMQX runs inside WSL2.

The Windows clock is synchronized with time.windows.com (w32tm /query /status: stratum 5, last successful sync 06/10/2026 11:43:11, poll interval 1024 s). Get-Date matched the taskbar clock (12:27 CAT on 6 October 2026, when the environment was set up) and the WSL clock was compared with Windows in screenshot 1.1-9. The later experiments (12-14 October) were run on the same machine; each program prints CAT-stamped times and every later screenshot shows its own taskbar clock.

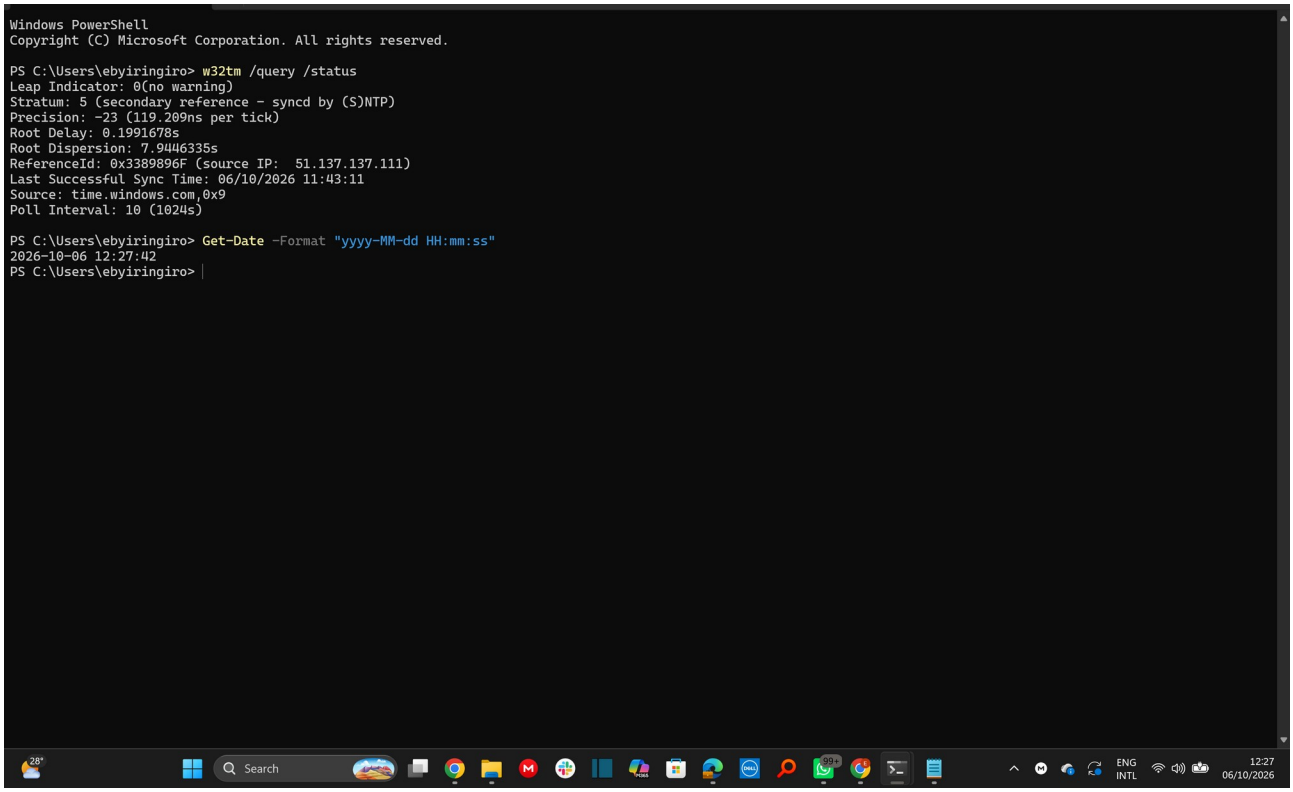
Screenshot 1.1-1 (1-1-1-postgres-db-extension-timezone.png)



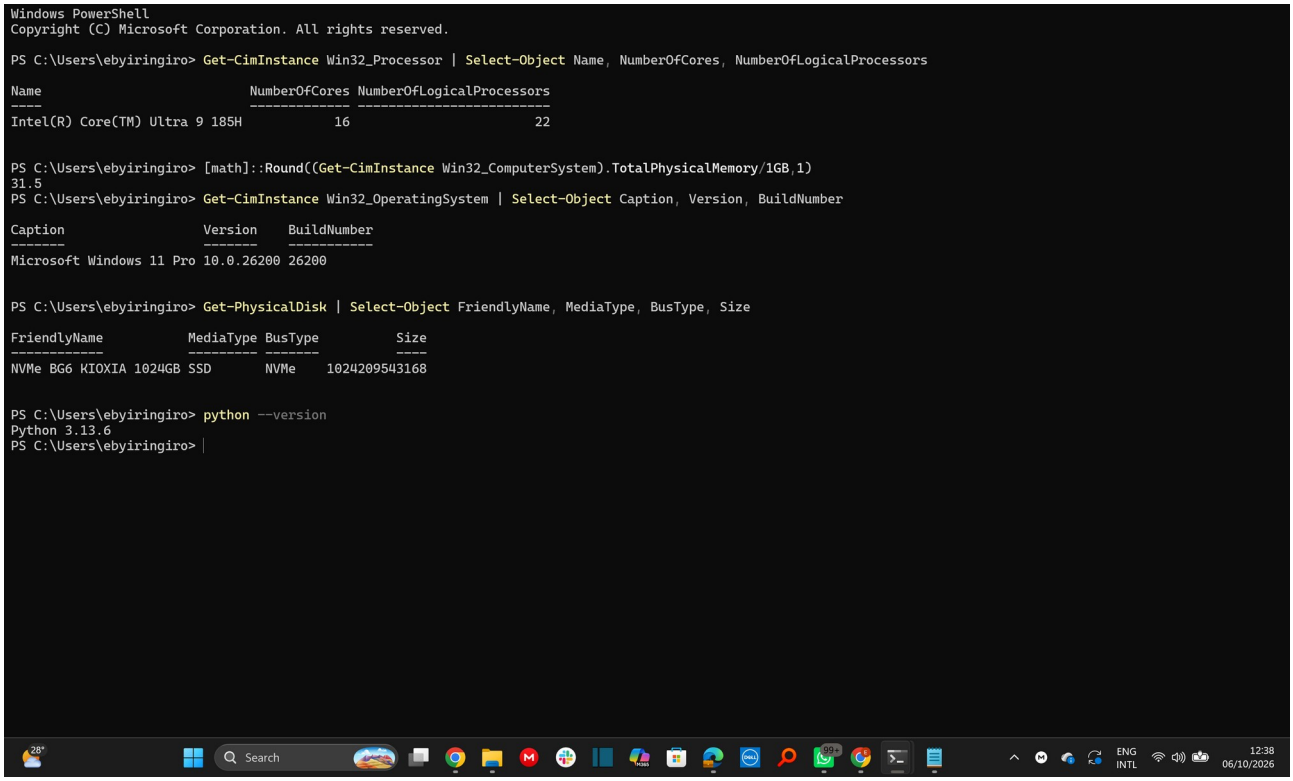
Screenshot 1.1-2 (1-1-2-emqx-broker-status.png)



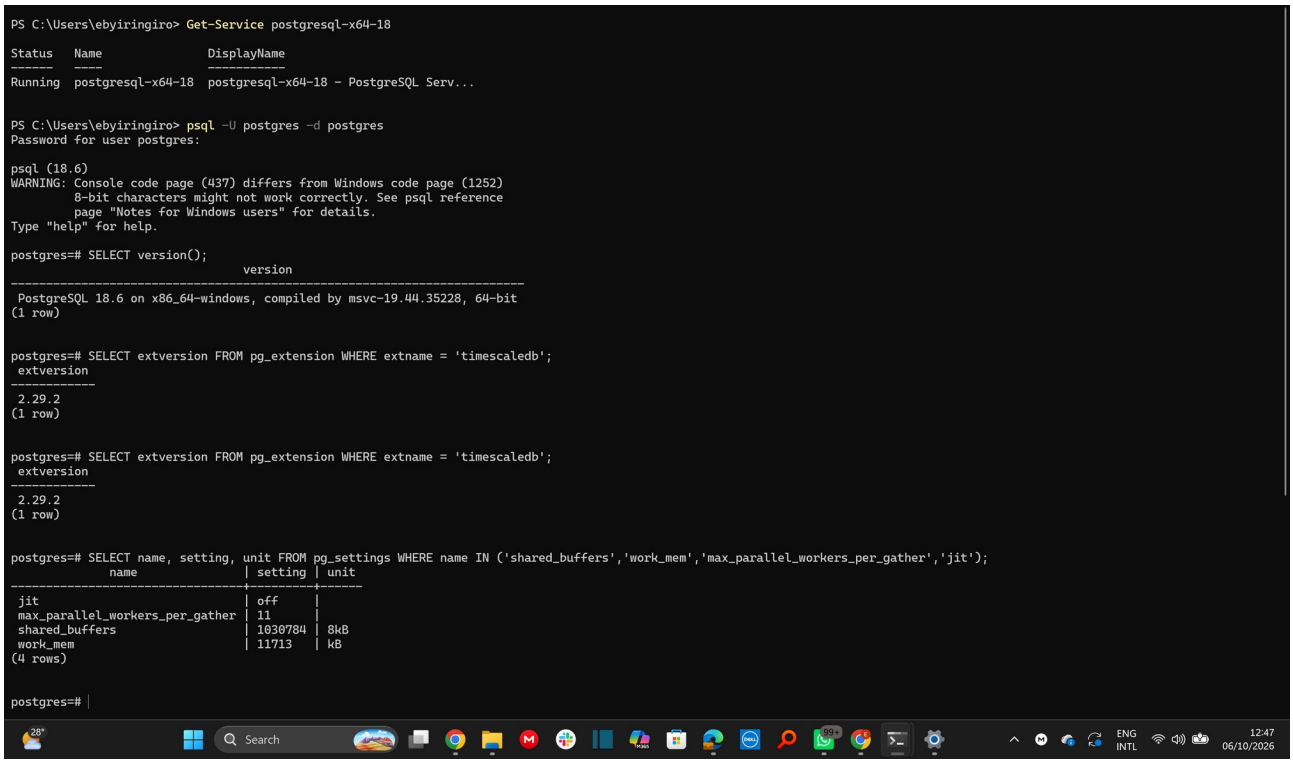
Screenshot 1.1-3 (1-1-3-emqx-dashboard.png)



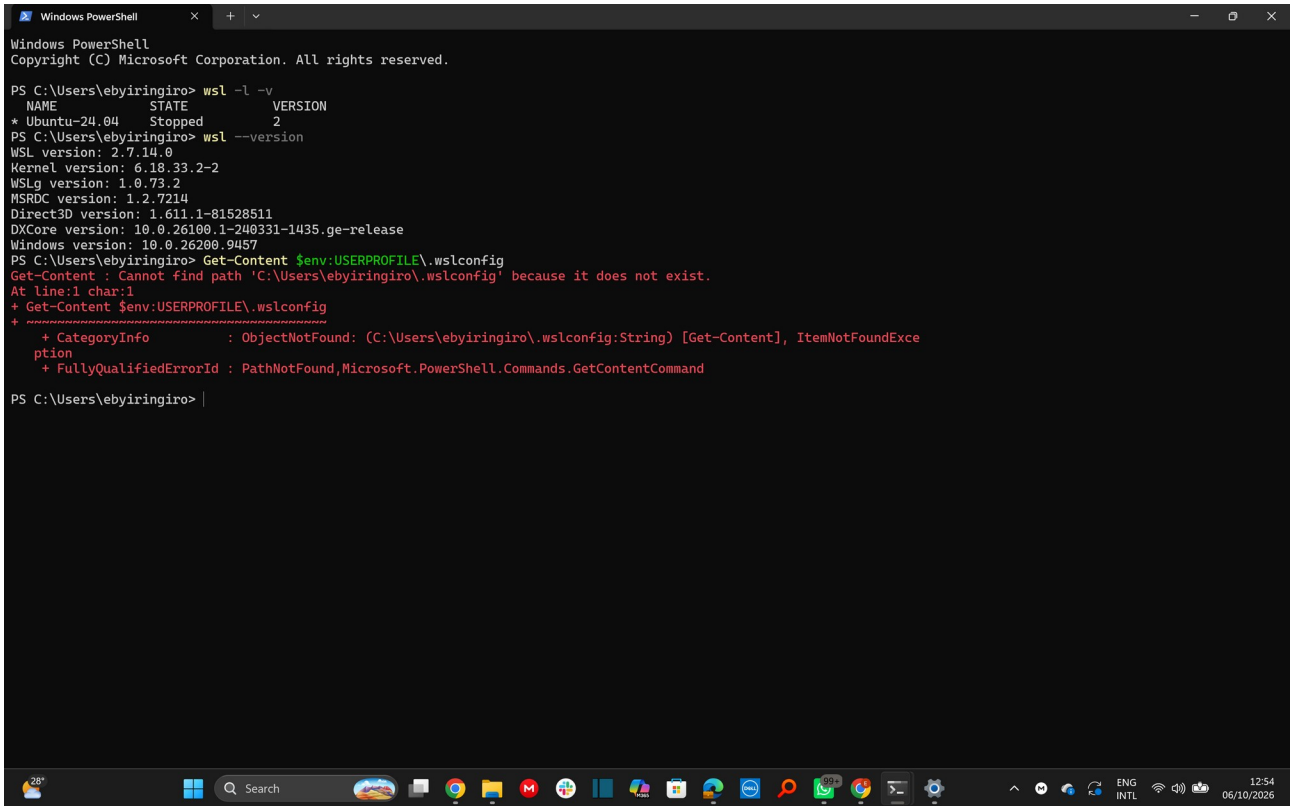
Screenshot 1.1-4 (1-1-4-windows-clock-sync.png)



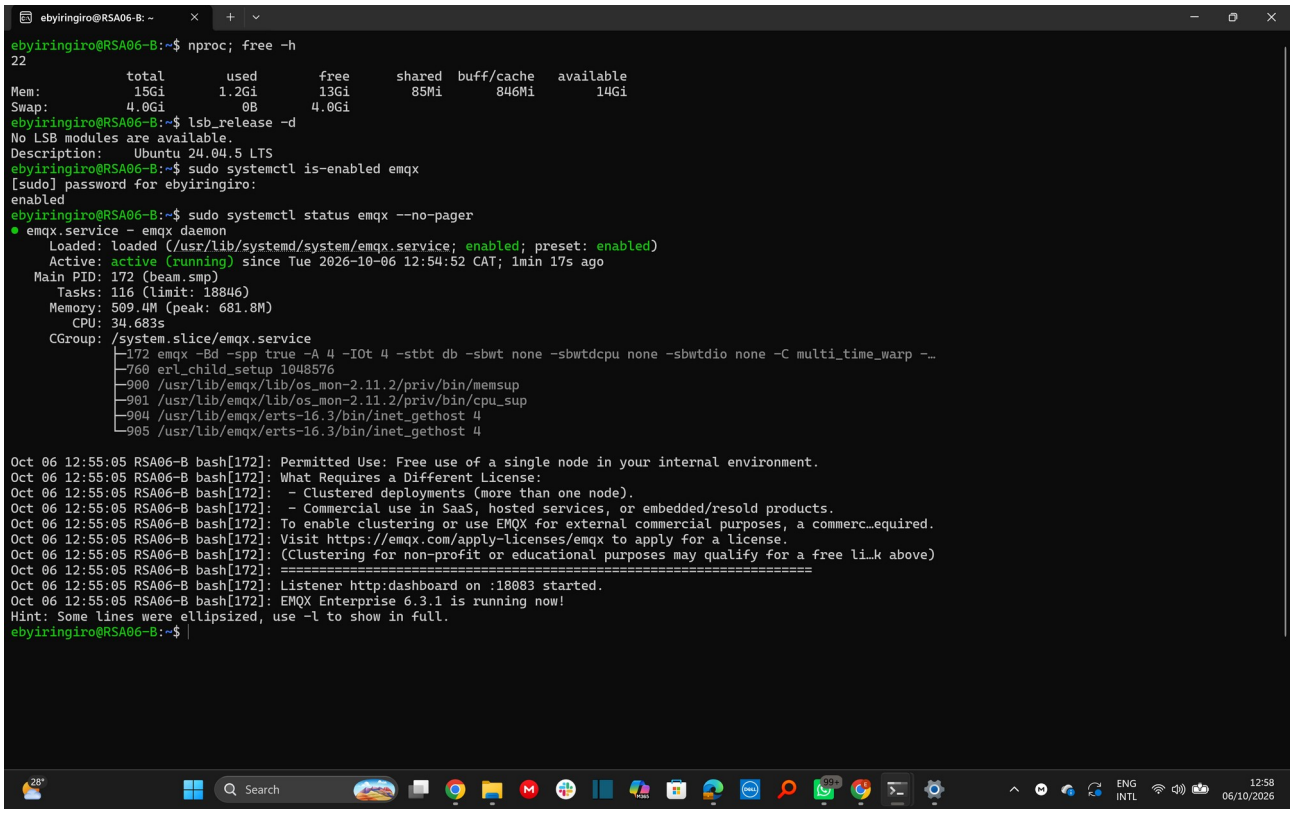
Screenshot 1.1-5 (1-1-4-hardware_os_python.png)



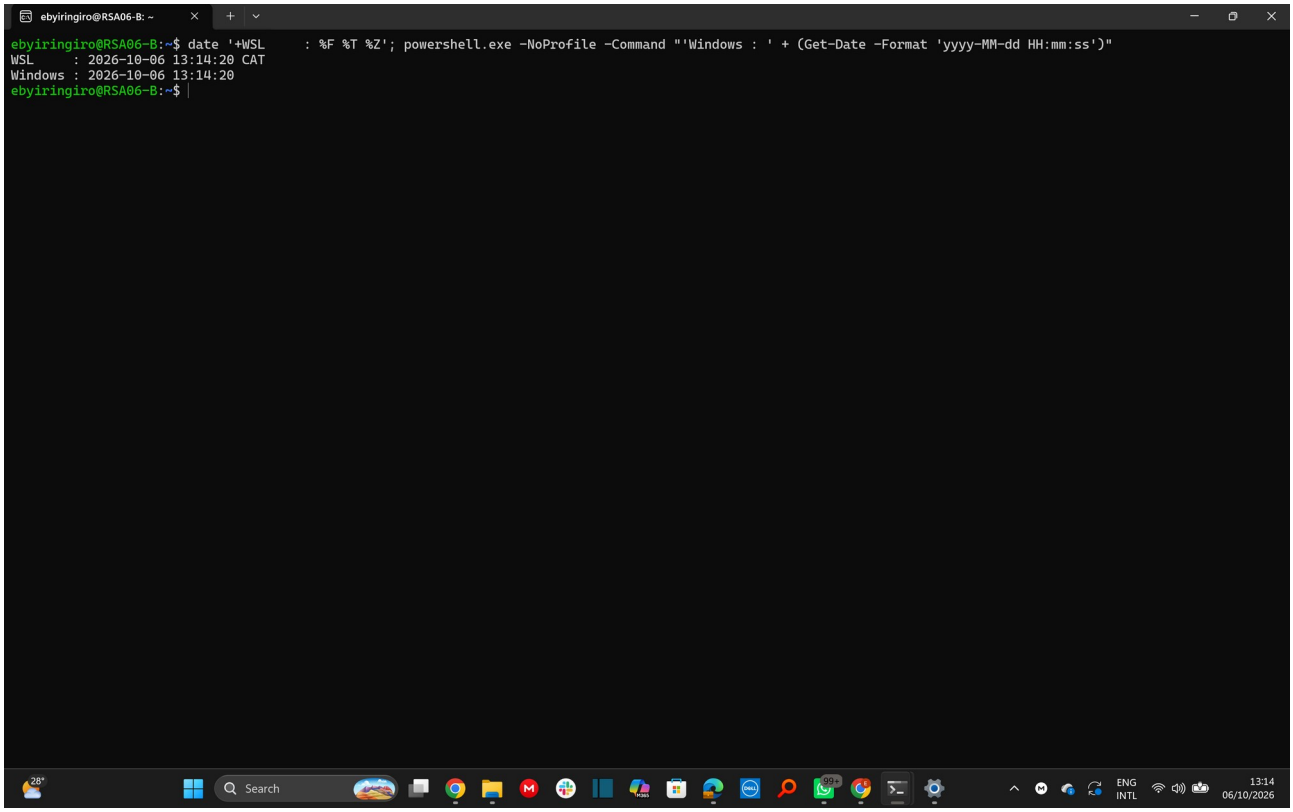
Screenshot 1.1-6 (1-1-5postgres-version-settings.png)



Screenshot 1.1-7 (1-1-6-wsl-version-limits.png)



Screenshot 1.1-8 (1-1-7-wsl-resources-emqx-service.png)



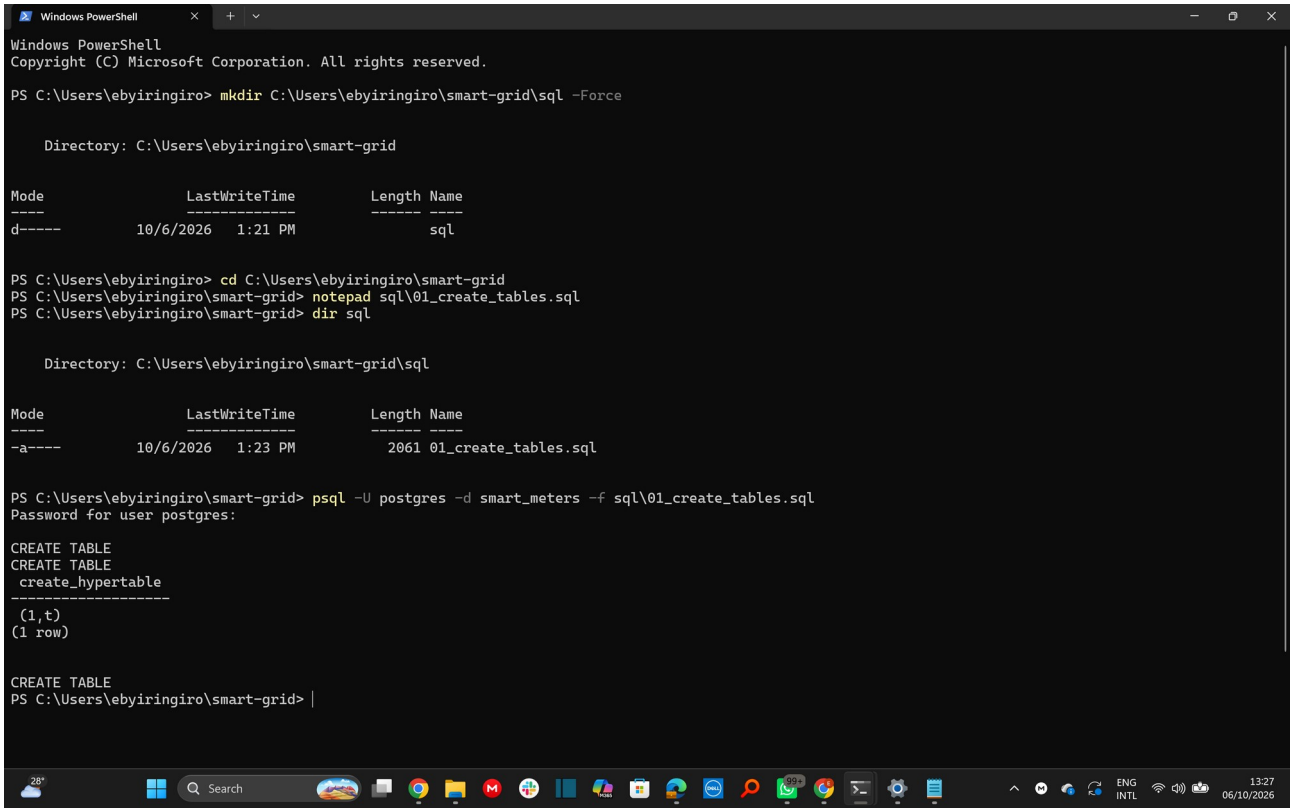
Screenshot 1.1-9 (1-1-8-wsl-windows-clock-match.png)

1.2 Create the database structures

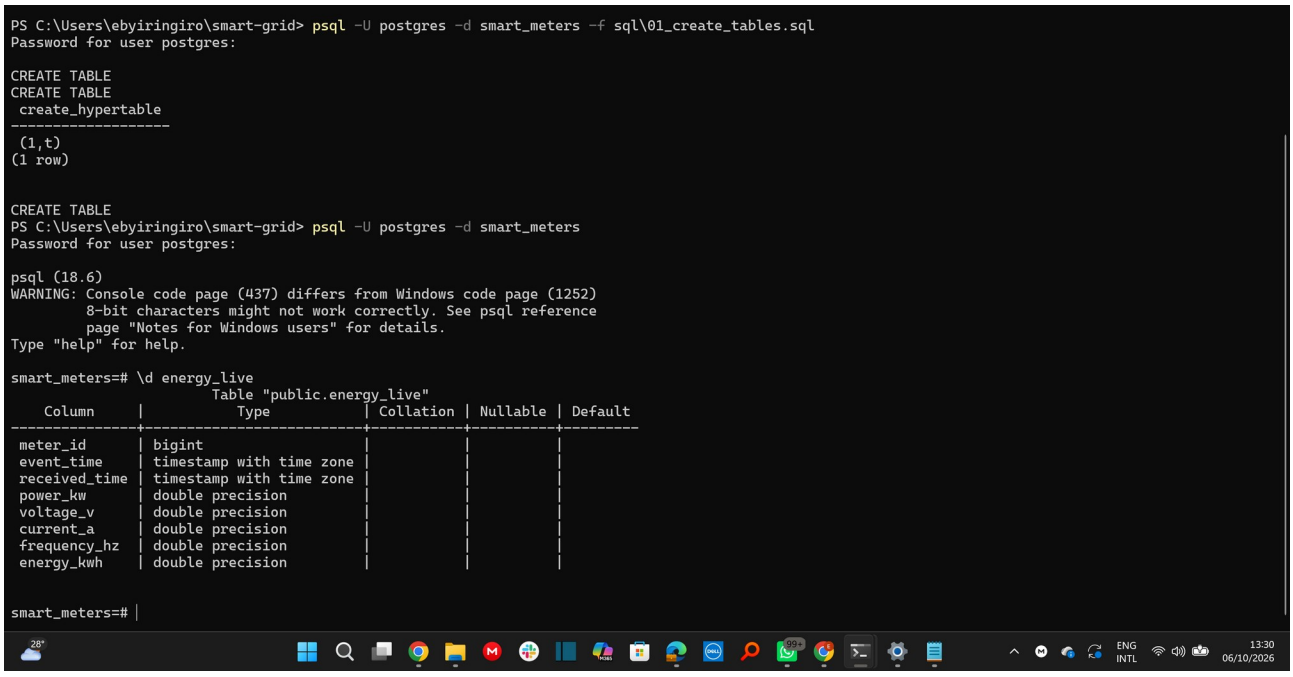
energy_live is a regular table. energy_readings is a TimescaleDB hypertable partitioned by event_time with one-day chunks (29 chunks, 2026-08-31 02:00 to 2026-09-29 02:00 CAT). Both reading tables have the same eight columns and allow duplicates at this stage. energy_readings has two indexes: (event_time DESC) and (meter_id, event_time). The meters table (meter_id primary key) was filled by load_meters.py, which imports smart_meter_simulator, calls meter_metadata() and inserts each dictionary; it holds 4,000 rows.

Data dictionary

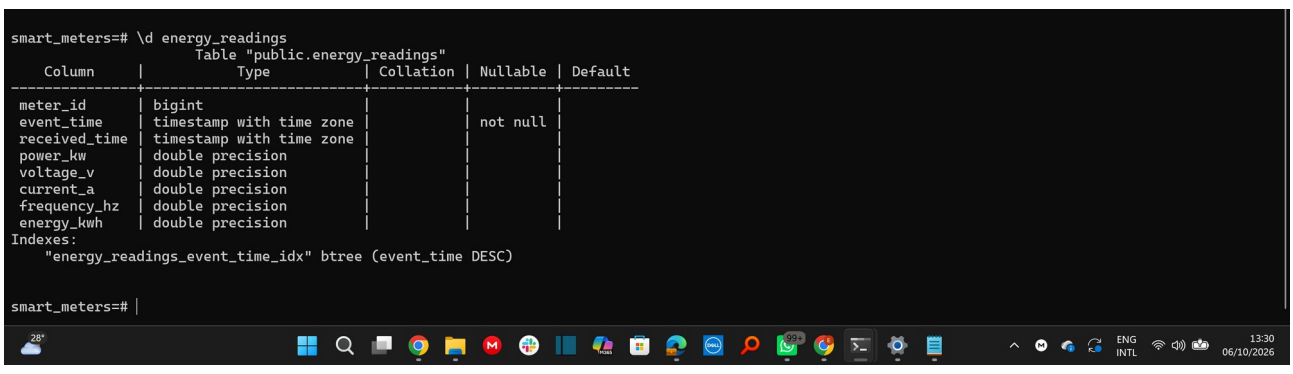
Table	Column	PostgreSQL type	Meaning	Unit
energy_live, energy_readings	meter_id	BIGINT	Meter identifier (1000000000 to 1000003999)	-
	event_time	TIMESTAMPTZ	Start of the 15-minute reading interval	CAT, +02:00
	received_time	TIMESTAMPTZ	Simulated arrival time of the reading (not insertion time)	CAT, +02:00
	power_kw	DOUBLE PRECISION	Average power over the interval	kW
	voltage_v	DOUBLE PRECISION	Voltage	V
	current_a	DOUBLE PRECISION	Current	A
	frequency_hz	DOUBLE PRECISION	Grid frequency	Hz
	energy_kwh	DOUBLE PRECISION	Energy in the interval, about power_kw x 0.25 h	kWh
meters	meter_id	BIGINT (primary key)	Meter identifier	-
	region	TEXT	Region 1 to Region 10 (400 meters each)	-
	customer_type	TEXT	residential, commercial or industrial	-
	base_power_kw	DOUBLE PRECISION	Base power level of the meter	kW
	power_factor	DOUBLE PRECISION	Power factor (0.95)	none



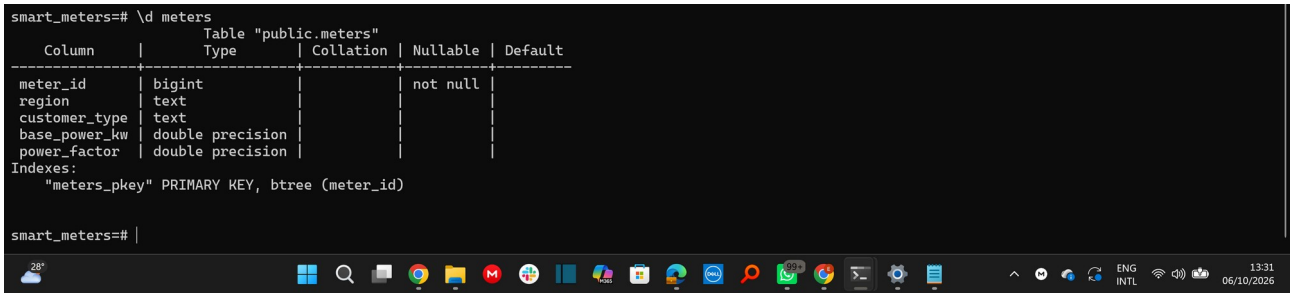
Screenshot 1.2-1 (1-1-2-1-run_create_tables_script.png)



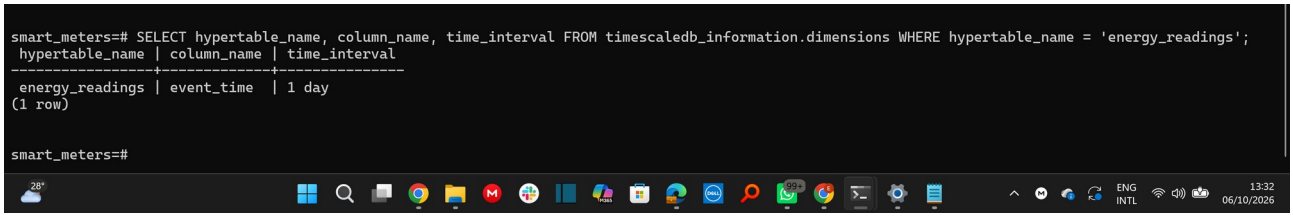
Screenshot 1.2-2 (1-1-2-2-describe_energy_live.png)



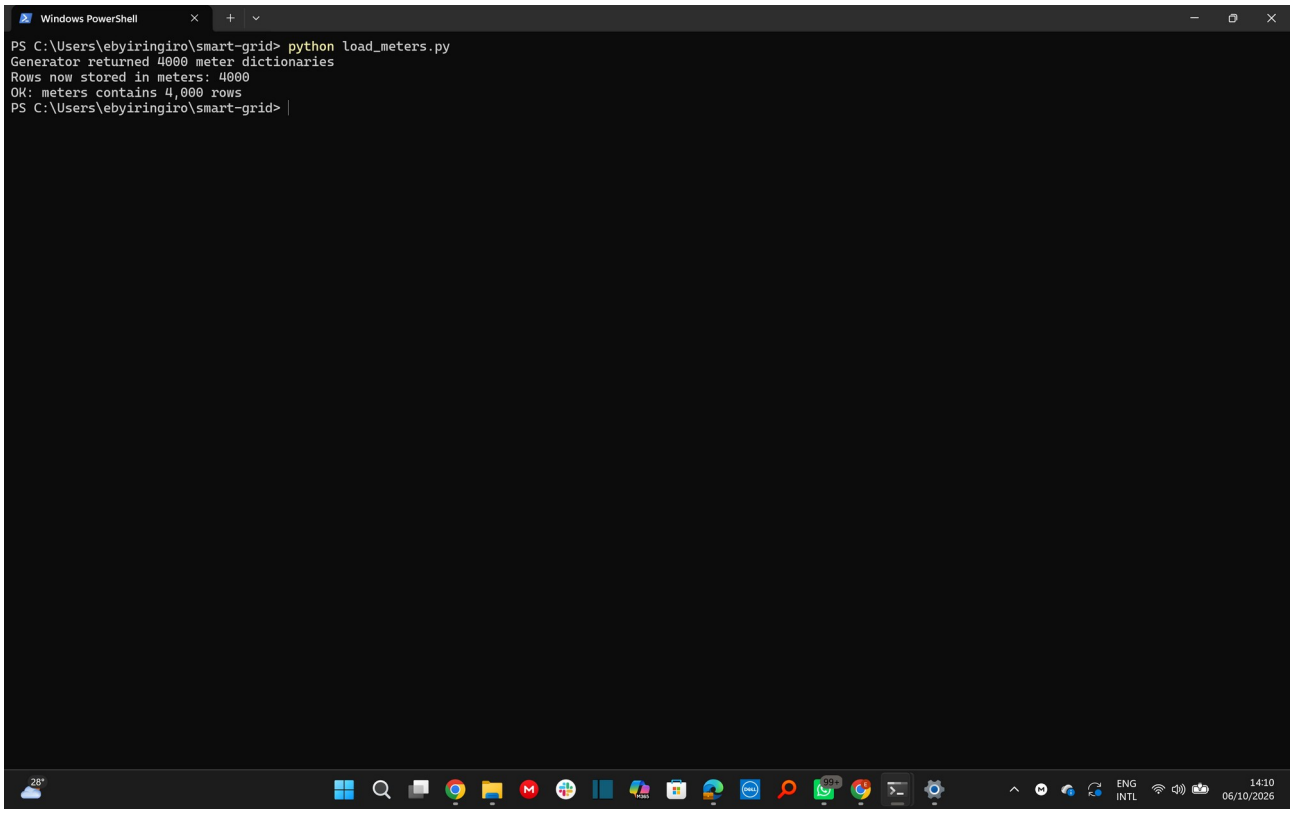
Screenshot 1.2-3 (1-1-2-2-describe_energy_readings.png)



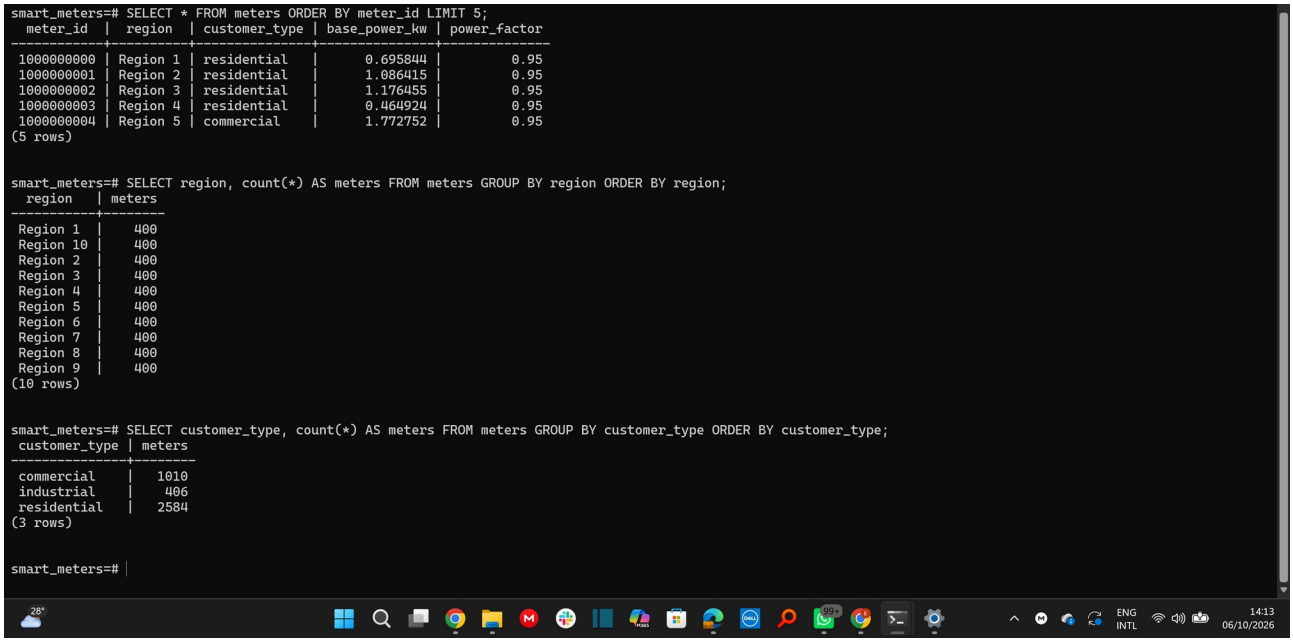
Screenshot 1.2-4 (1-1-2-3-describe_meters.png)



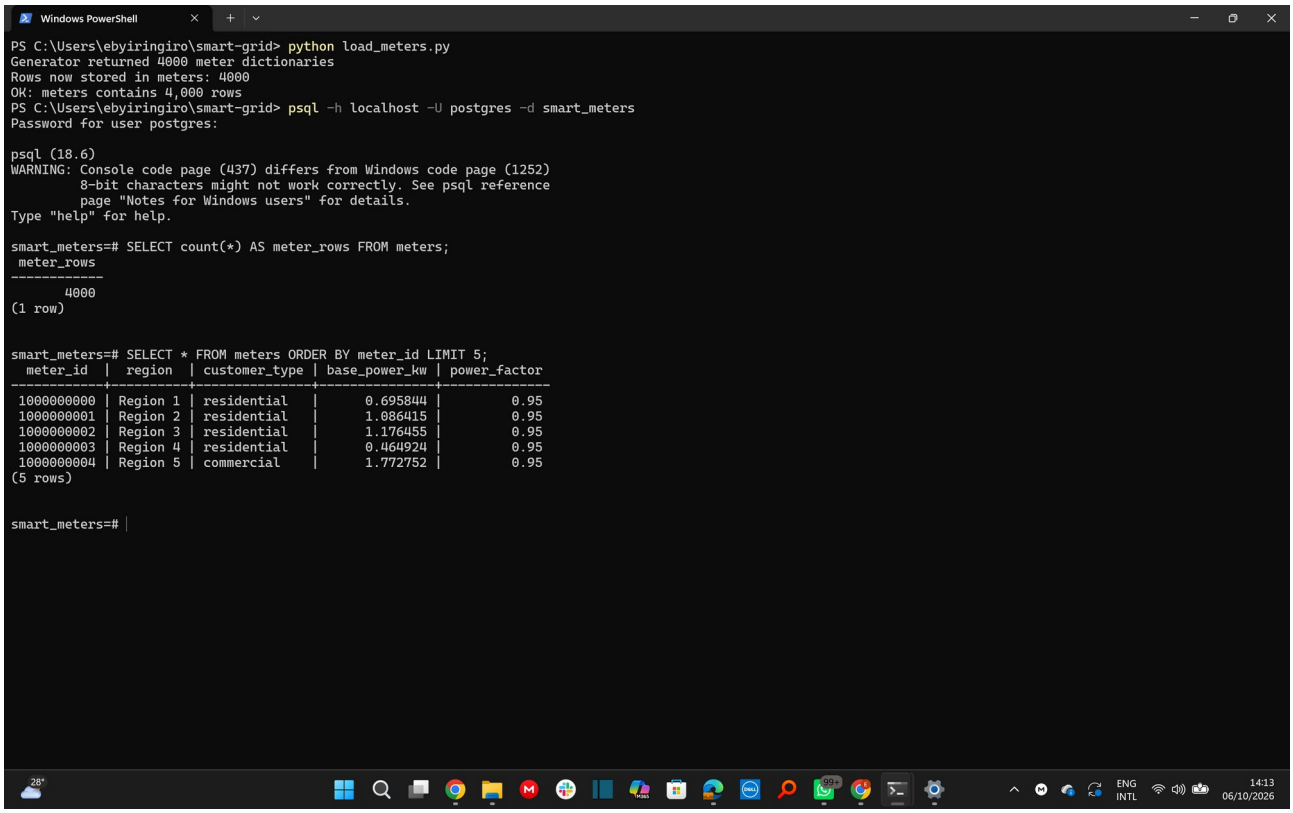
Screenshot 1.2-5 (1-2-5-chunk-interval.png)



Screenshot 1.2-6 (1-1-2-5-confirming 4,000 metadata rows.png)



Screenshot 1.2-7 (1-1-2-6-Sample data.png)



Screenshot 1.2-8 (1-1-2-7-Group by.png)

1.3 Capture and publish a pilot stream

pilot_publisher.py imports the unchanged generator, takes the first 2,000 dictionaries from iter_readings(), converts each to JSON and publishes one message to energy/meters/{meter_id} with MQTT QoS 1 and no retained messages. pilot_subscriber.py subscribes to energy/meters/# at QoS 1, parses each JSON message and inserts its eight fields into energy_live. energy_live was empty and the subscriber was started before the publisher.

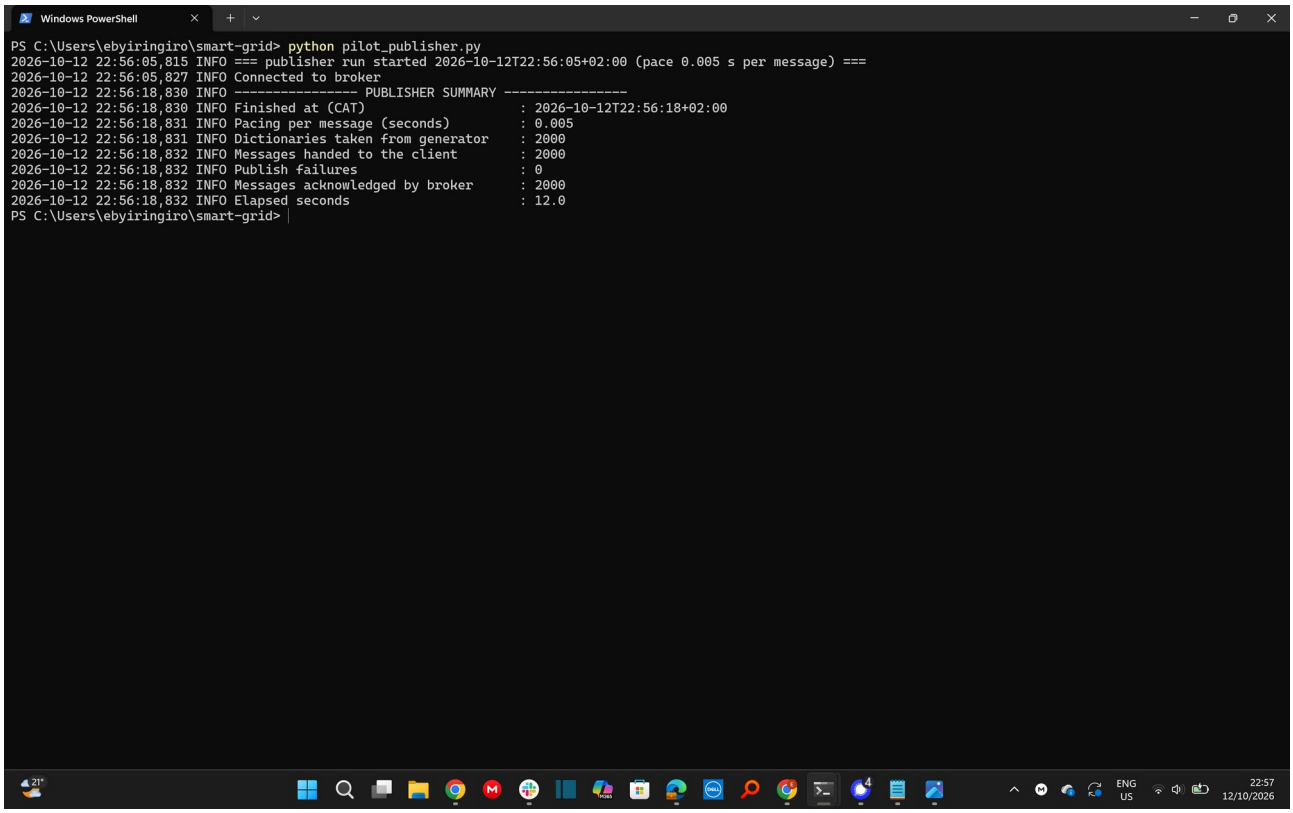
Pilot counts (target: 2,000 at every stage)

Stage	Target	Run 1 (11:32)	Final run
Dictionaries taken from the generator	2,000	2,000	2000
Messages acknowledged by the broker	2,000	2,000	2000
Messages received by the subscriber	2,000	1,876 (124 missing)	2000

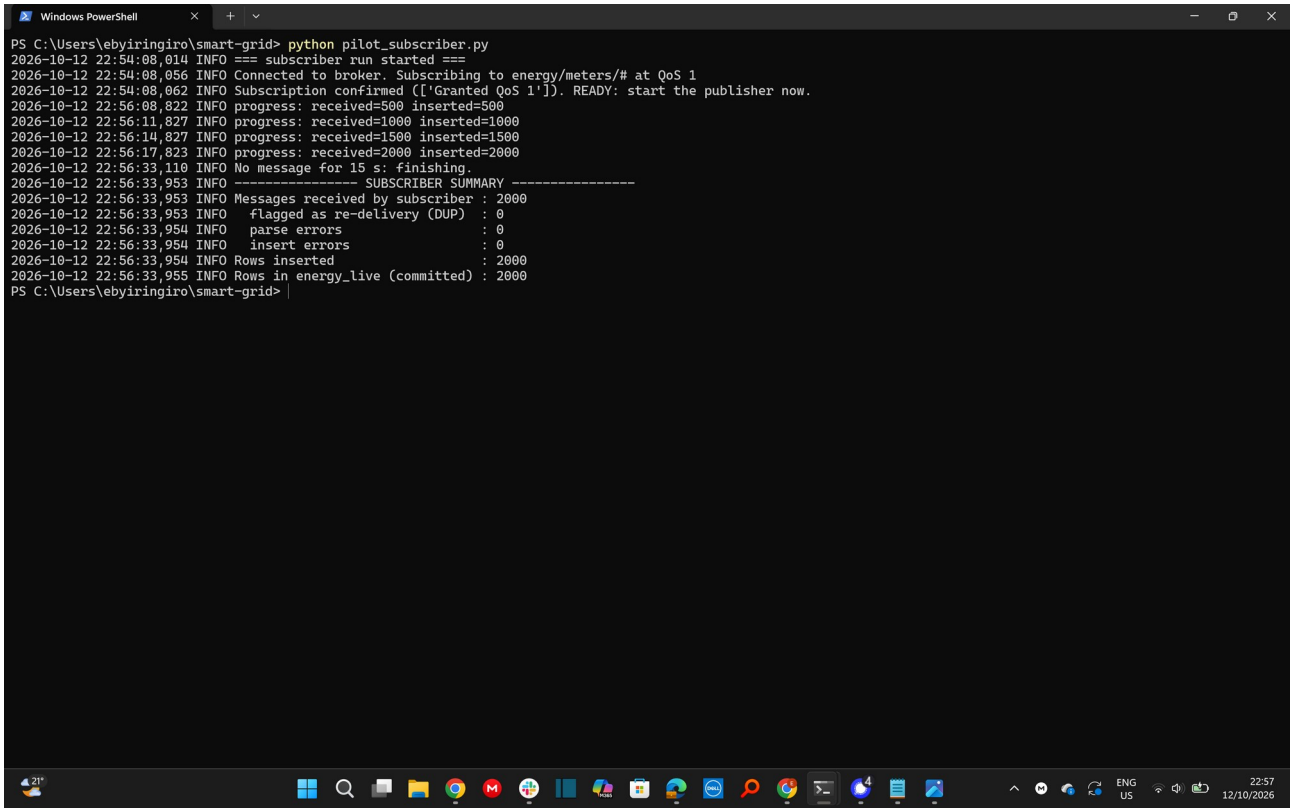
Rows in energy_live	2,000	1,876	2,000 (query on 12 Oct 2026)

Observation (run 1). On 12 Oct 2026 (publisher from 11:32:12, subscriber to 11:32:29 CAT) the publisher log shows all 2,000 messages acknowledged by the broker (2,000 messages in 0.8 s, about 2,500 per second), but the subscriber received only 1,876. analyse_losses.py compared the two logs: 124 of the 2,000 sent messages were missing at the subscriber, in 12 runs of consecutive send positions between 778 and 999 (for example 778-784, 833-844, 851-872 and 955-980). There were 0 arrivals without a matching send, 0 DUP (re-delivery) flags, 0 parse errors and 0 insert errors, and rows in energy_live (1,876) equalled messages received. A second unpaced run at 17:10 CAT gave the same kind of loss (2,000 acknowledged, 1,753 received, 1,753 rows).

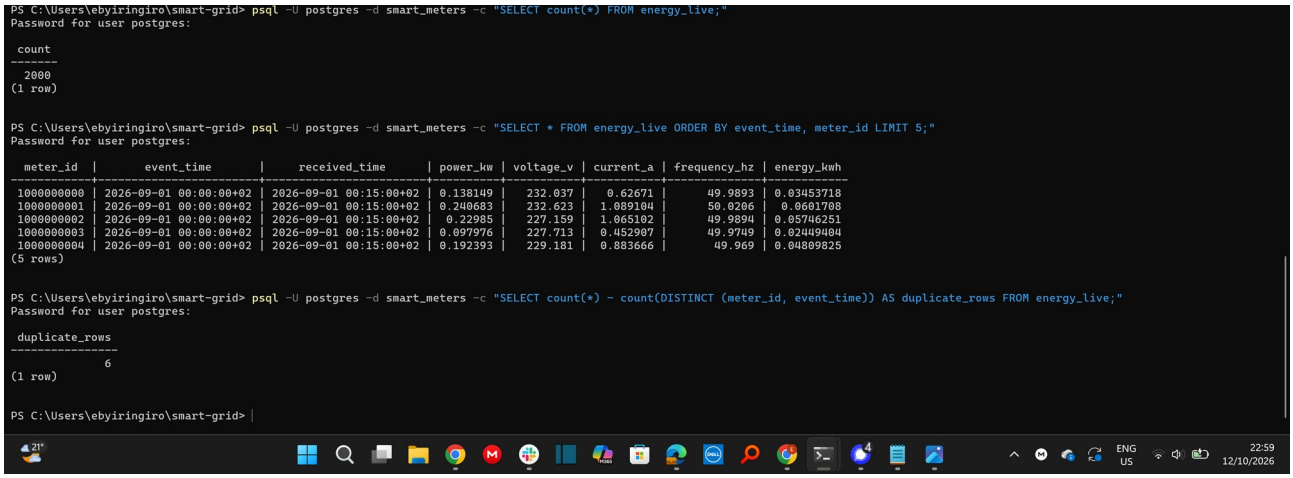
Cause and correction. (a) Cause: the loss is on the broker-to-subscriber leg under an unpaced burst. The publisher side lost nothing, and the logs rule out redelivery, parsing errors and failed inserts. The gaps are bursts of consecutive messages after roughly the first 780, which fits a single subscriber (one autocommitted INSERT per message) falling behind a ~2,500 msg/s burst until the broker's per-subscriber queue overflowed. The queue overflow is our inference from this pattern; the EMQX drop counters were not captured. (b) Correction: pilot_publisher.py now pauses PACE_SECONDS = 0.005 s after each message (about 200 messages per second; 12.0 s for 2,000 messages). The subscriber was not changed. The final run (22:56 CAT) gave 2,000 taken, 2,000 acknowledged, 2,000 received and 2,000 rows in energy_live.



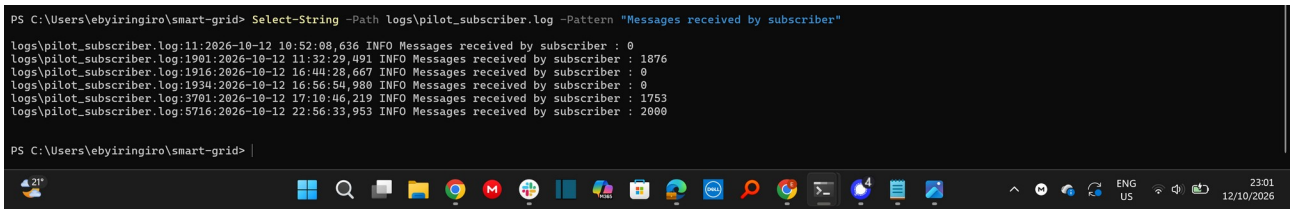
Screenshot 1.3-1 (Screenshot 2026-10-12 225743.png)



Screenshot 1.3-2 (Screenshot 2026-10-12 225731.png)



Screenshot 1.3-3 (Screenshot 2026-10-12 230006.png)



Screenshot 1.3-4 (Screenshot 2026-10-12 230108.png)

2 Capture and prepare the historical dataset

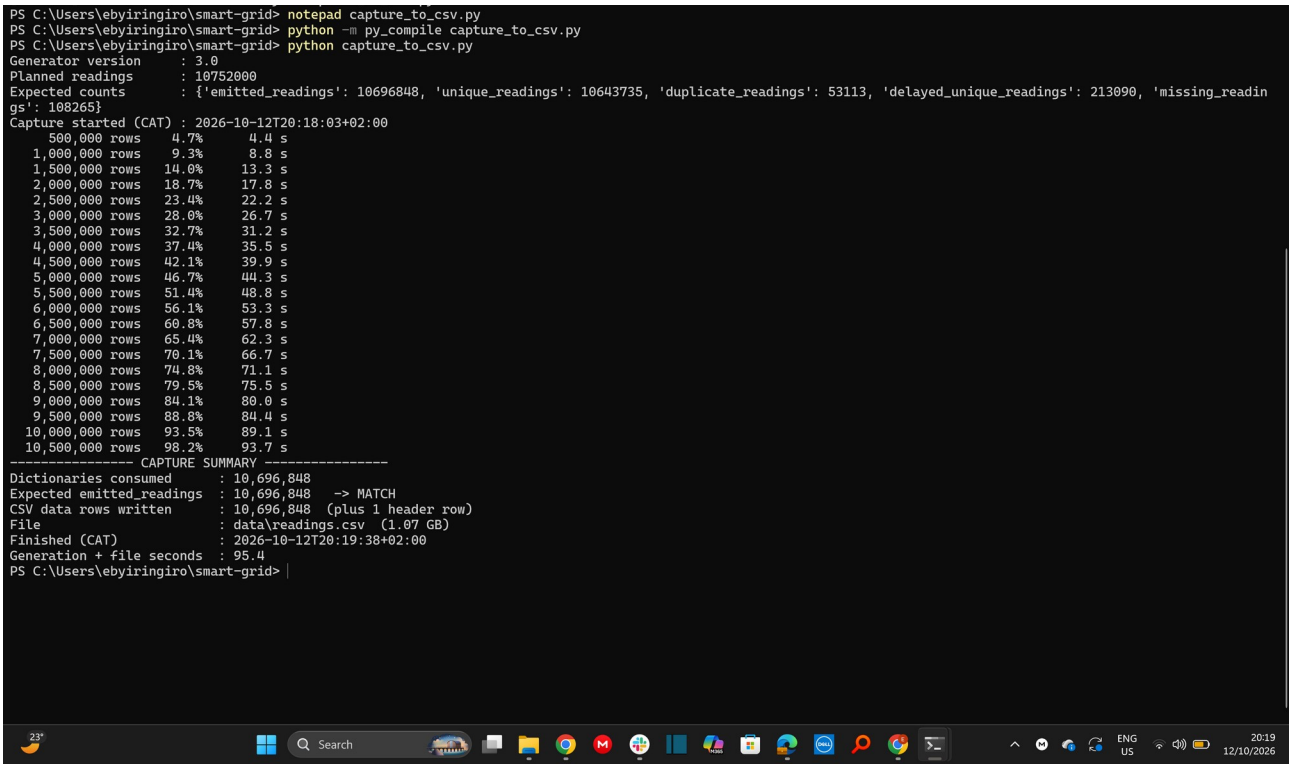
2.1 Capture and load the complete stream

Route: CSV + COPY. capture_to_csv.py iterates iter_readings() progressively and writes data\readings.csv; load_csv.py reads the file in 16 MB pieces and bulk-loads it into the empty energy_readings hypertable, including duplicates. The pilot rows remain in energy_live.

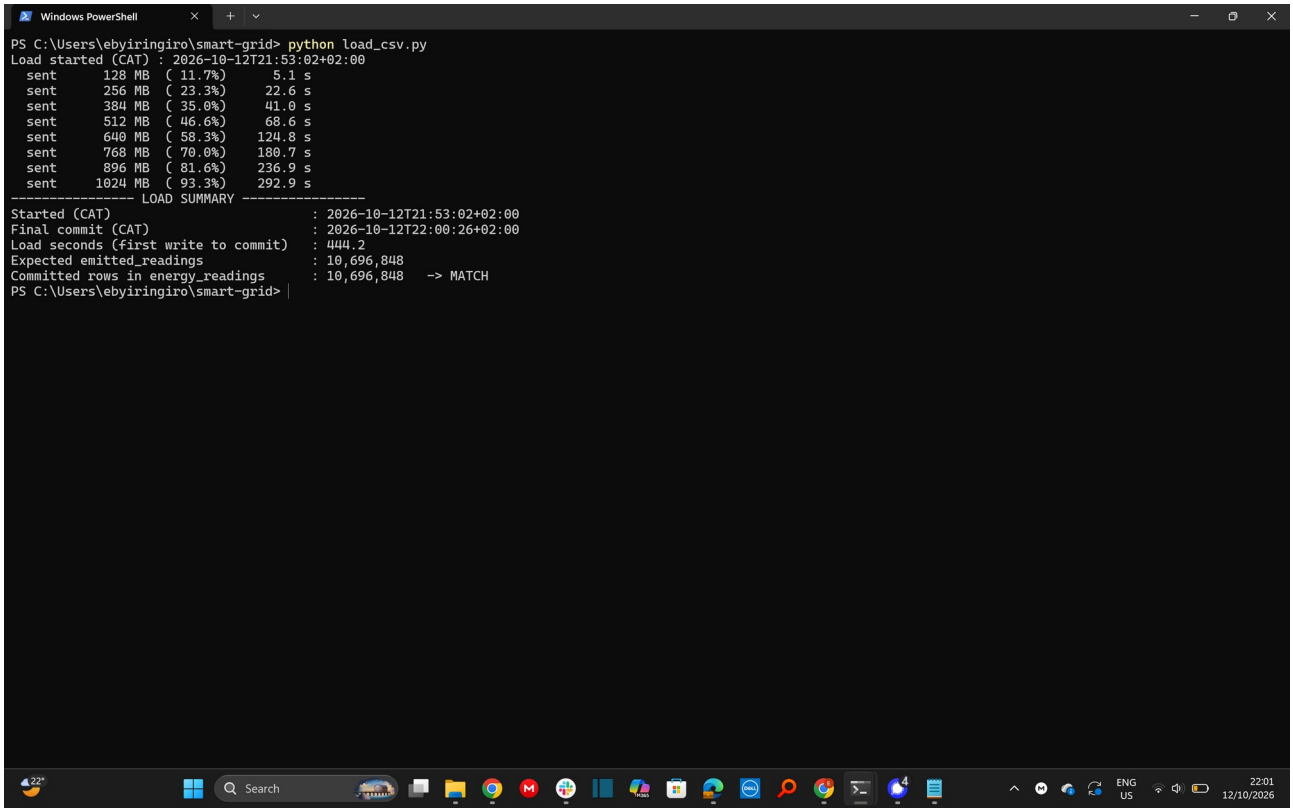
Check	Value
Planned readings (dataset_info)	10,752,000
expected_counts.emitted_readings	10,696,848
Dictionaries consumed from the generator	10,696,848
Rows committed to energy_readings	10,696,848
Data rows in the CSV file	10,696,848 (plus 1 header row)
Generation (start 2026-10-12 20:18:03, end 20:19:38 CAT)	95.4 s
Database load (start 2026-10-12 21:53:02, end 22:00:26 CAT)	444.2 s
Active total (generation + load)	539.6 s
Wall-clock span, first generation to final commit	6,143 s (the two steps ran separately; about 93 minutes of this was idle time between them)

All three counts (consumed, committed and expected) agree at 10,696,848 before cleaning.

Database checks on energy_readings: 4,000 distinct meters; earliest event_time 2026-09-01 00:00:00+02; latest 2026-09-28 23:45:00+02 (these limits were re-checked after cleaning in 3.1).Size of energy_readings including all chunks, measured after cleaning (the pre-cleaning size was not captured): data 950.9 MB, indexes 405.8 MB, total 1,356.7 MB.



Screenshot 2.1-1 (Screenshot 2026-10-12 201956.png)



Screenshot 2.1-2 (Screenshot 2026-10-12 220117.png)

2.2 Investigate data quality

Entry	Reference	Measured	Percentage
planned_readings	10,752,000	10,752,000	-
emitted_readings	10,696,848	10,696,848	-
unique_readings	10,643,735	10,643,735	completeness 98.9931%
duplicate_readings (extra copies)	53,113	53,113	0.4990% of unique pairs
delayed_unique_readings	213,090	213,090	2.0020% of unique pairs
missing_readings	108,265	108,265	1.0069% of planned

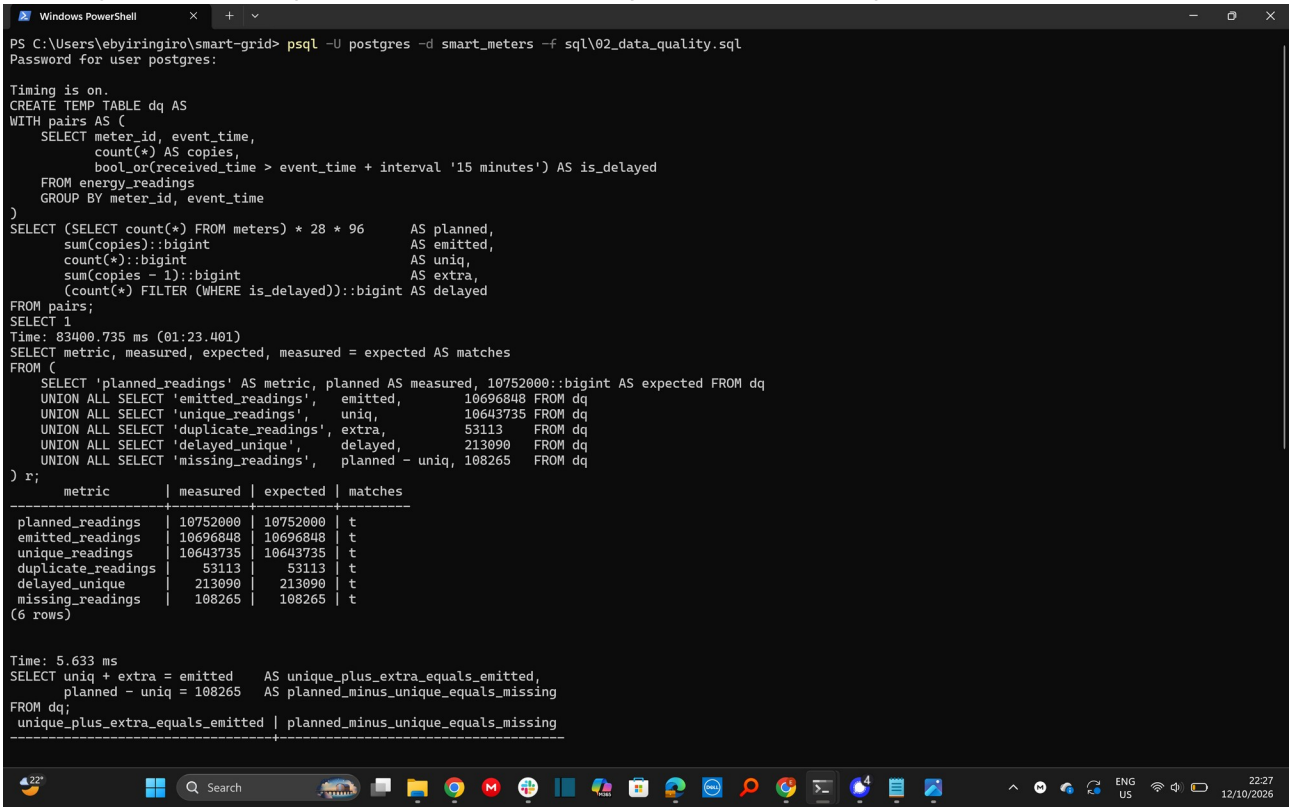
Consistency checks: unique + extra copies = 10,643,735 + 53,113 = 10,696,848 = emitted readings; planned - unique = 10,752,000 - 10,643,735 = 108,265 = missing readings.

The Measured column matches the 2.2 screenshots: every measured count equals its reference value.

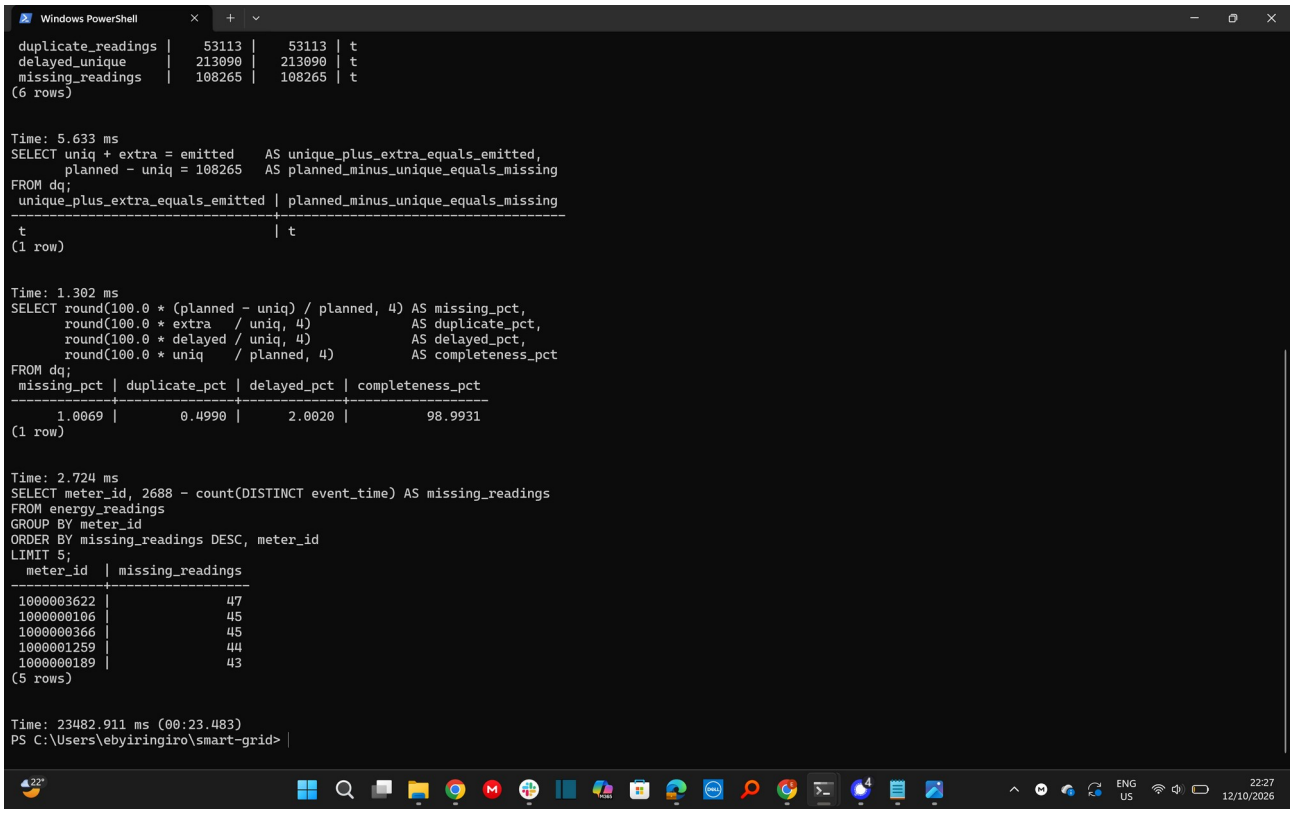
Five meters with the most missing readings

Rank	meter_id	Missing readings (of 2,688)
1	1000003622	47
2	1000000106	45
3	1000000366	45
4	1000001259	44
5	1000000189	43

missing 1.0069%, duplicates 0.4990%, delayed 2.0020%, completeness 98.9931%



Screenshot 2.2-1 (Screenshot 2026-10-12 222714.png)



Screenshot 2.2-2 (Screenshot 2026-10-12 222736.png)

2.3 Clean and preserve a consistent benchmark

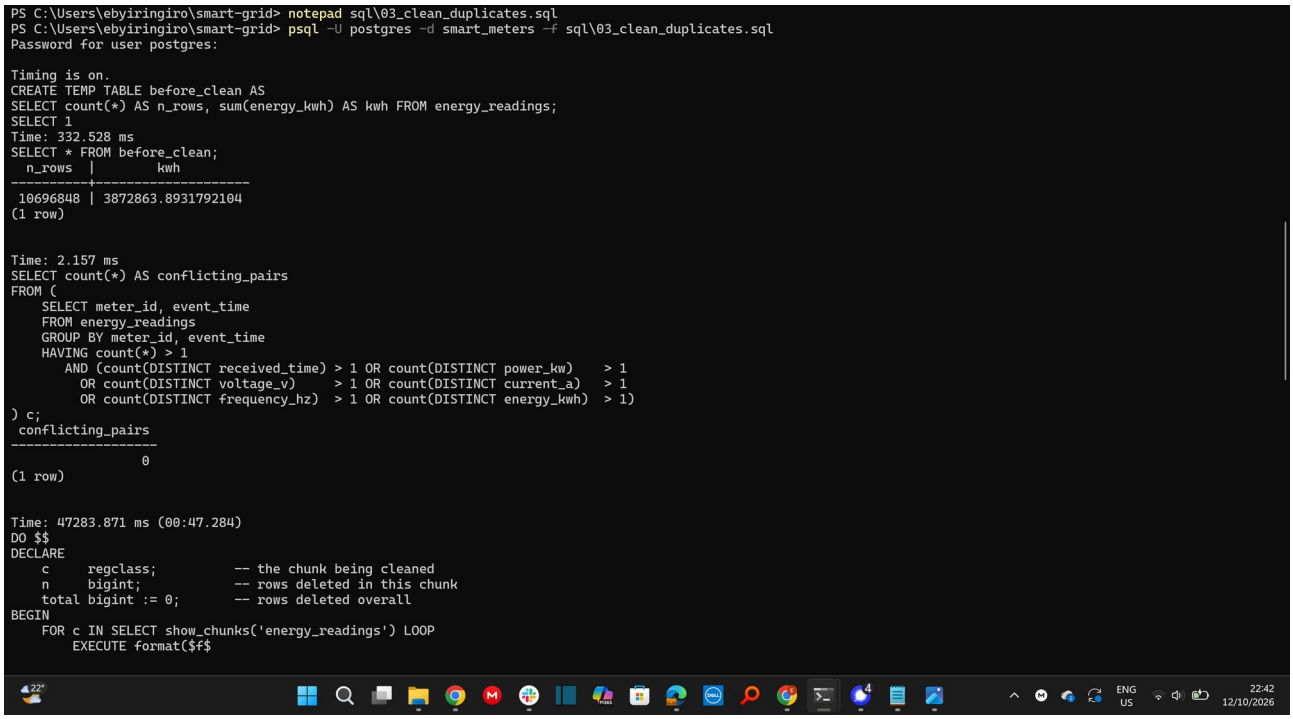
Rows sharing meter_id and event_time were compared on received_time, power_kw, voltage_v, current_a, frequency_hz and energy_kwh. Exact copies were reduced to one row by sql\03_clean_duplicates.sql (repeatable). Missing intervals were kept as gaps.

Measure	Before cleaning	After cleaning
Row count	10,696,848	10,643,735

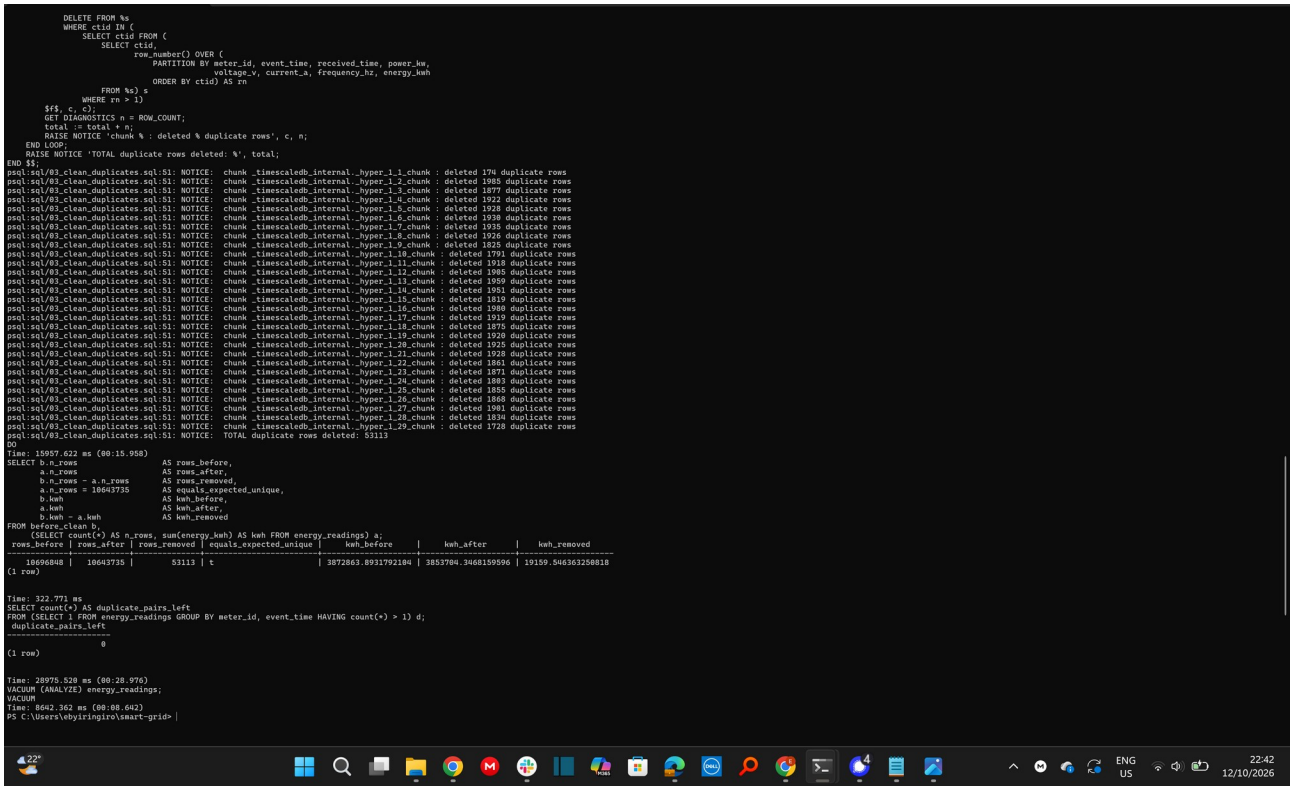
Rows deleted	-	53,113
Sum of energy_kwh	3,872,863.89 kWh	3,853,704.3468
Duplicate (meter_id, event_time) pairs	53,113 extra copies (pairs affected not separately recorded)	0

The final row count equals expected_counts.unique_readings (10,643,735). All 53,113 extra copies were exact duplicates (the conflict query returned 0 conflicting pairs), so one row per pair was kept and the rest deleted; no duplicate pair remains.

Why missing readings are not zero. A missing reading means no measurement arrived, not that the meter used no energy. Replacing gaps with zero would invent consumption of exactly 0 kWh, bias totals and averages downward and hide communication faults. Gaps are therefore reported as missing and counted in completeness.



Screenshot 2.3-1 (Screenshot 2026-10-12 224232.png)



Screenshot 2.3-2 (Screenshot 2026-10-12 224308.png)

3 Compare chunk intervals

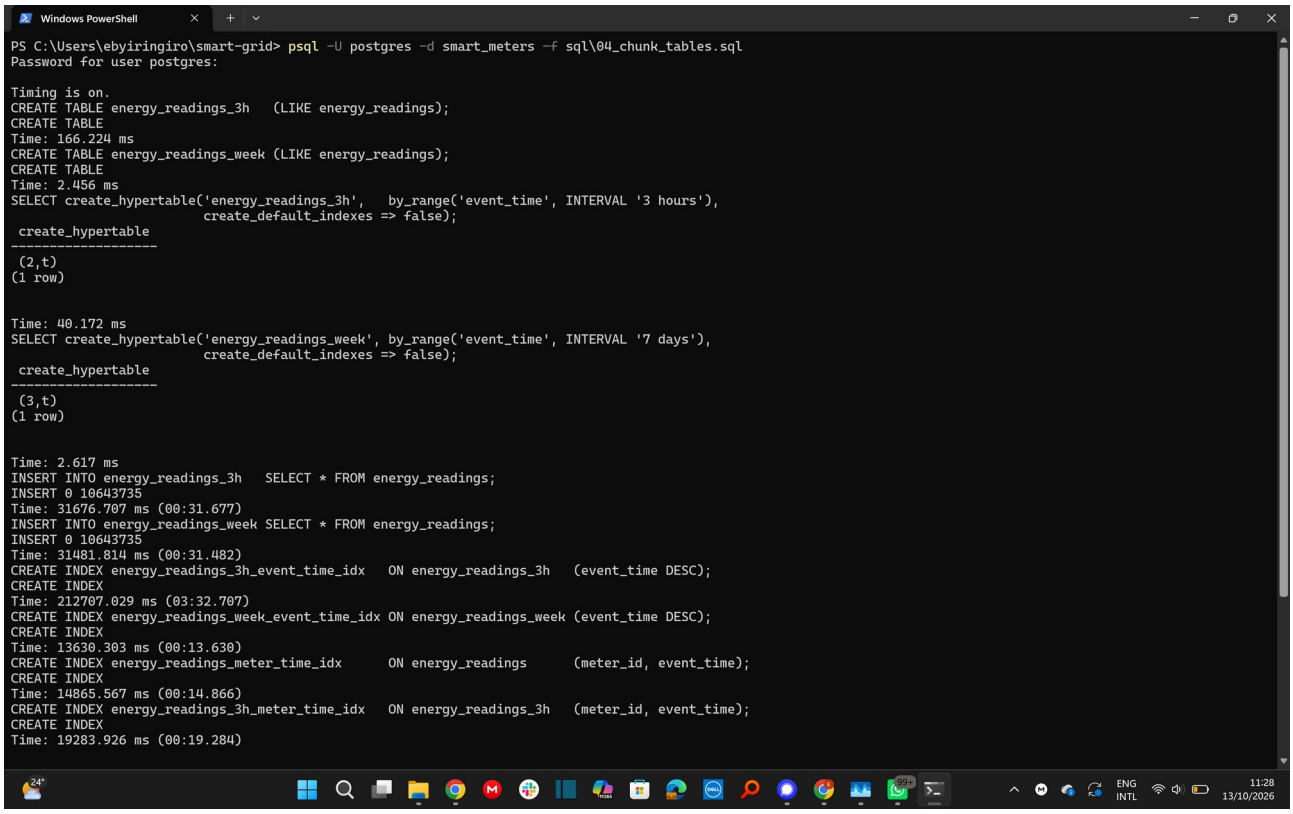
3.1 Create and verify three configurations

Table	Chunk interval	Chunks	First chunk start	Last chunk end
energy_readings	1 day	29	2026-08-31 02:00 +02	2026-09-29 02:00 +02
energy_readings_3h	3 hours	225	2026-08-31 23:00 +02	2026-09-29 02:00 +02
energy_readings_week	7 days	5	2026-08-27 02:00 +02	2026-10-01 02:00 +02

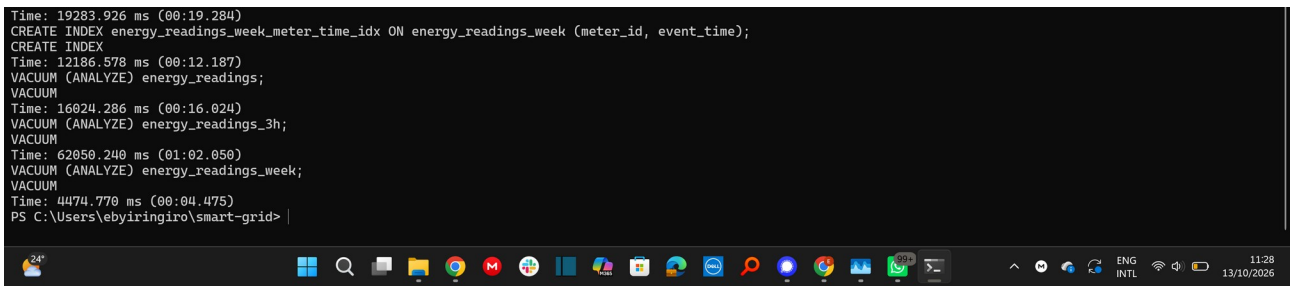
Each table has two matching non-unique indexes: (event_time DESC) and (meter_id, event_time).

Table	Rows	Meters	Earliest	Latest	sum(power_kw)	sum(energy_kwh)	Diff kW / kWh vs daily
energy_readings	10,643,735	4,000	2026-09-01 00:00	2026-09-28 23:45	15,414,817.3878	3,853,704.3468	0 / 0
energy_readings_3h	10,643,735	4,000	2026-09-01 00:00	2026-09-28 23:45	15,414,817.3878	3,853,704.3468	1.9e-9 / 1.4e-9
energy_readings_week	10,643,735	4,000	2026-09-01 00:00	2026-09-28 23:45	15,414,817.3878	3,853,704.3468	2.0e-8 / 3.3e-9

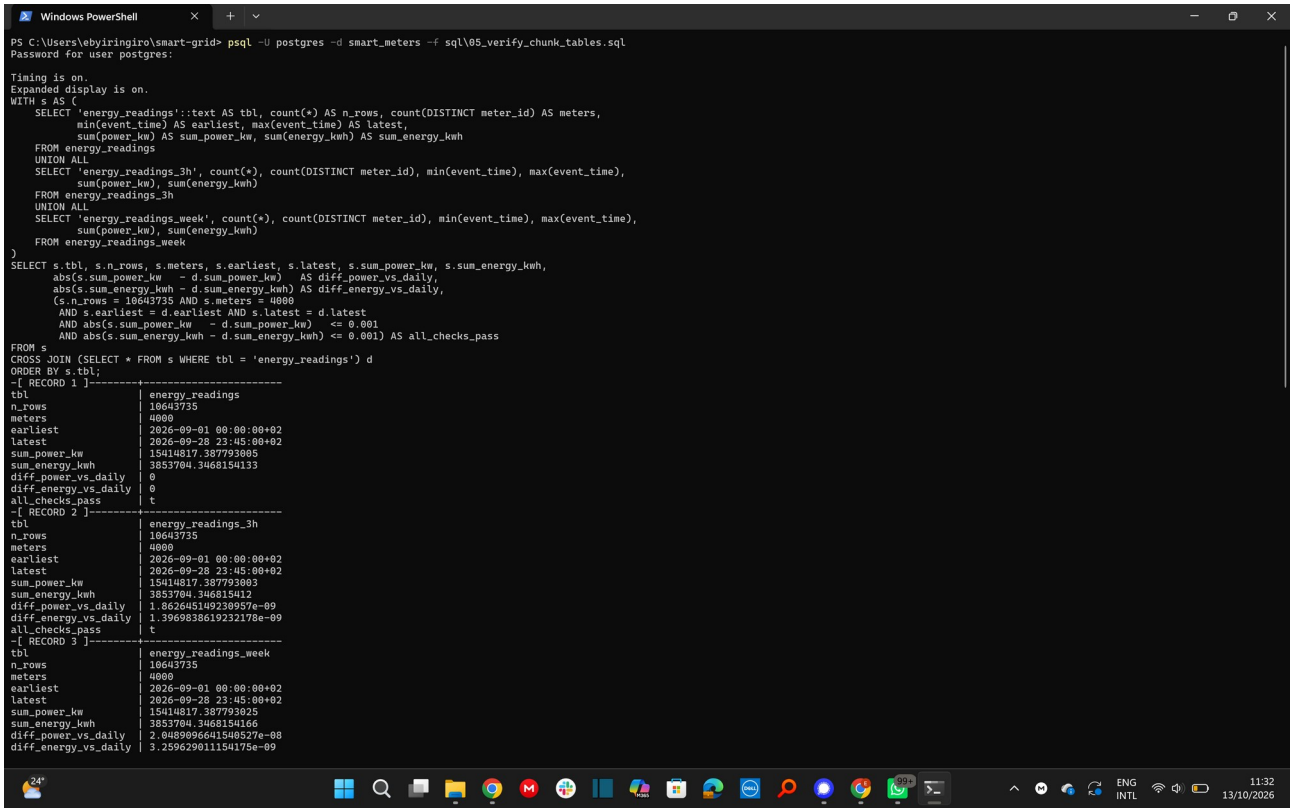
All three tables pass: 10,643,735 rows, 4,000 meters, identical dates and sums within the 0.001 tolerance.



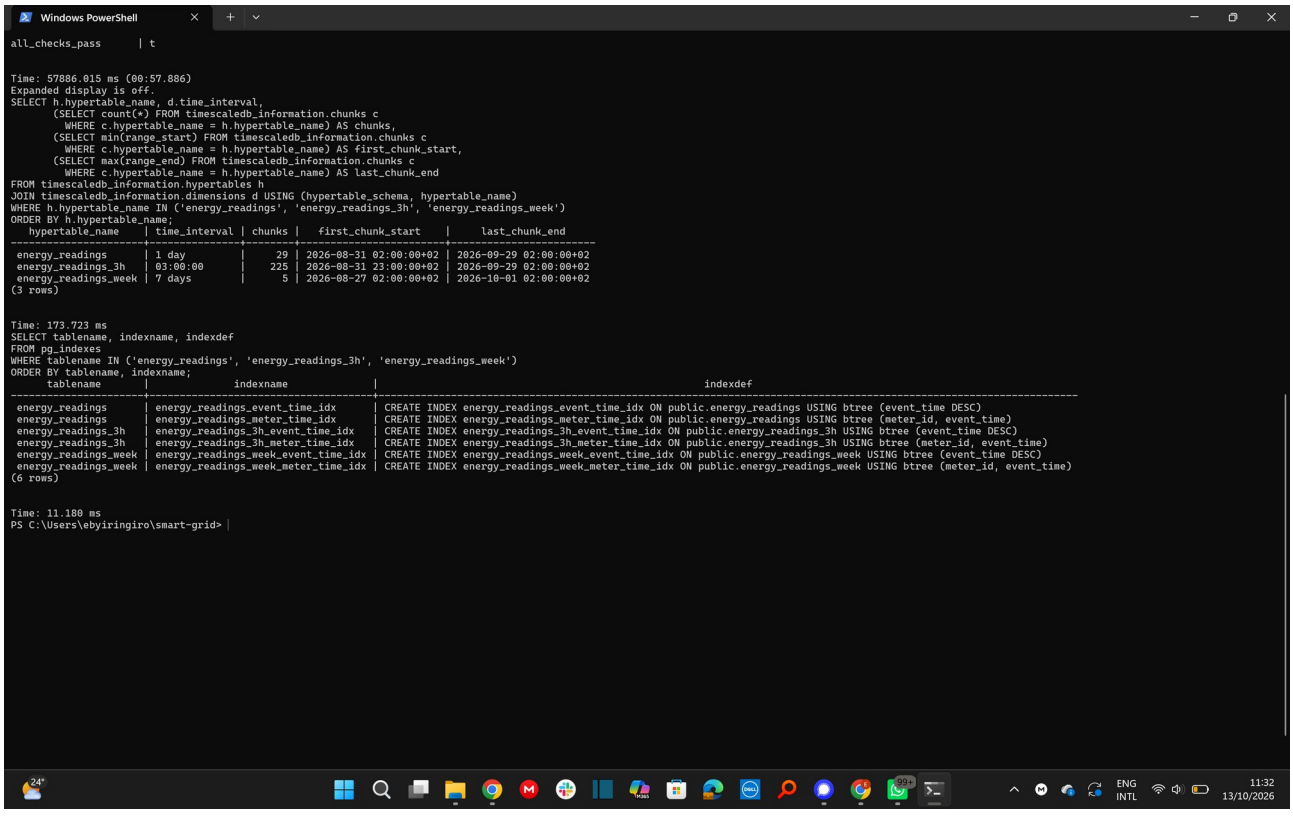
Screenshot 3.1-1 (Screenshot 2026-10-13 112818.png)



Screenshot 3.1-2 (Screenshot 2026-10-13 112839.png)



Screenshot 3.1-3 (Screenshot 2026-10-13 113242.png)



Screenshot 3.1-4 (Screenshot 2026-10-13 113301.png)

3.2 Implement and time the five workloads

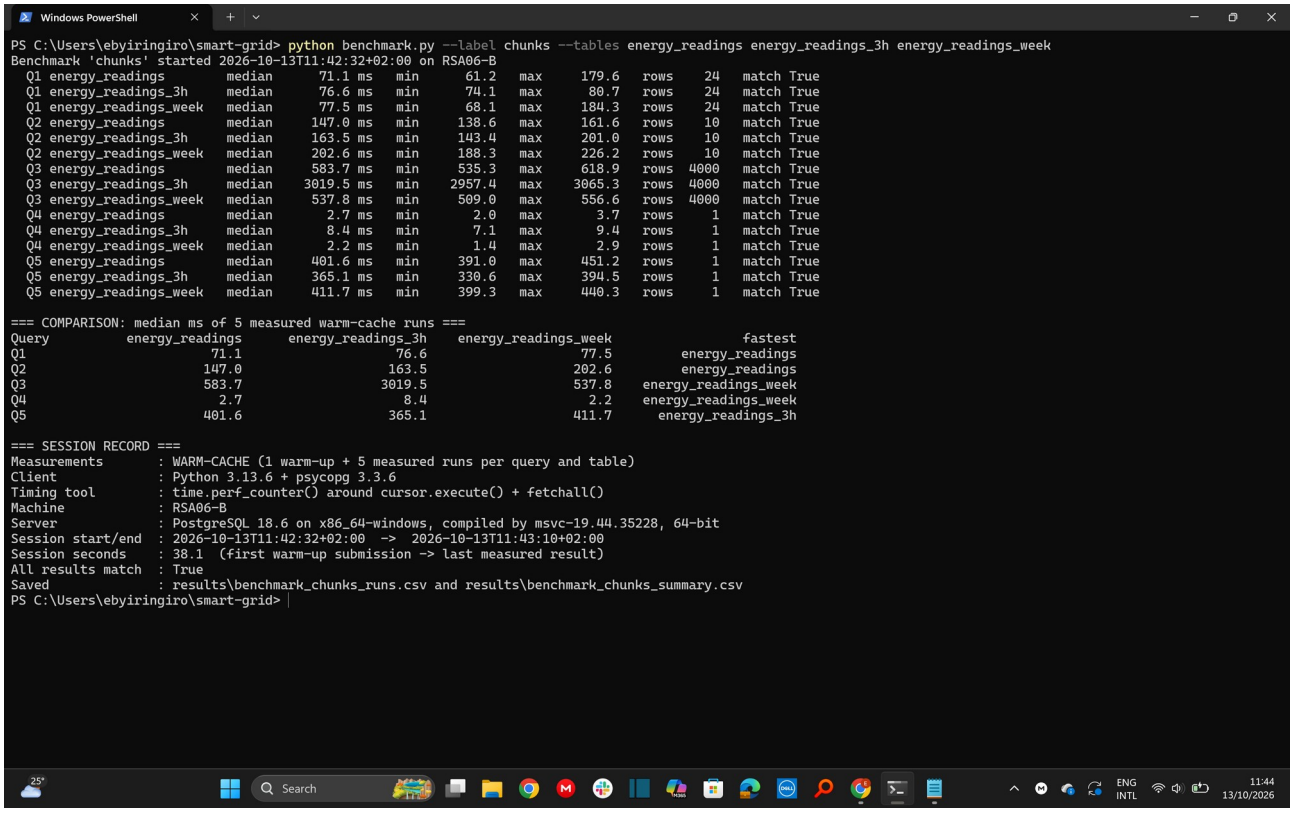
Query	Workload
Q1	Average power_kw per CAT hour on 21 September across all meters (24 rows)
Q2	Ten 15-minute intervals from 22 to 28 September with the highest summed power_kw, ties included
Q3	Total energy per meter over the full period (4,000 rows)
Q4	Energy total, average power and peak power for meter 1000000000, 22 to 28 September
Q5	Full scan: row count, total energy, average, minimum and maximum power

Method: one machine (RSA06-B), one client (benchmark.py using Python 3.13.6 and psycopg 3.3.6), PostgreSQL never restarted and no cache cleared. Each query and table had one warm-up and five measured executions, timed from statement submission to the last result row received. All results are therefore warm-cache measurements. Results were checked against the one-day table (all matched within 0.001). Per-run CAT start and end times, durations and machine identifier are in results\ benchmark_chunks_runs.csv.

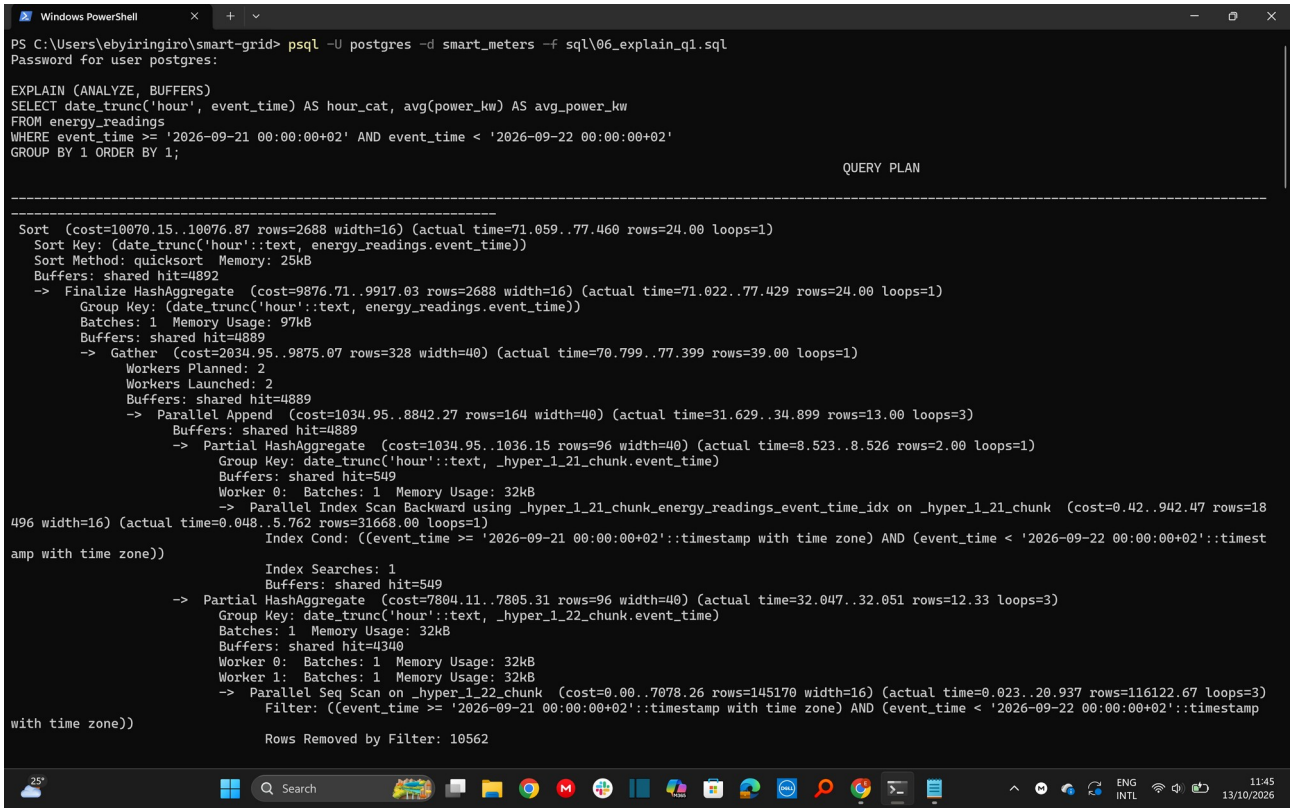
Timing tool: time.perf_counter() around cursor.execute() and fetchall(), client Python 3.13.6 with psycopg 3.3.6. The session ran 38.1 s from the first warm-up submission to the last measured result.

Query	1 day: median (min-max) ms	3 hours: median (min-max) ms	1 week: median (min-max) ms
Q1	71.1 (61.2-179.6)	76.6 (74.1-80.7)	77.5 (68.1-184.3)
Q2	147.0 (138.6-161.6)	163.5 (143.4-201.0)	202.6 (188.3-226.2)
Q3	583.7 (535.3-618.9)	3,019.5 (2,957.4-3,065.3)	537.8 (509.0-556.6)
Q4	2.7 (2.0-3.7)	8.4 (7.1-9.4)	2.2 (1.4-2.9)
Q5	401.6 (391.0-451.2)	365.1 (330.6-394.5)	411.7 (399.3-440.3)

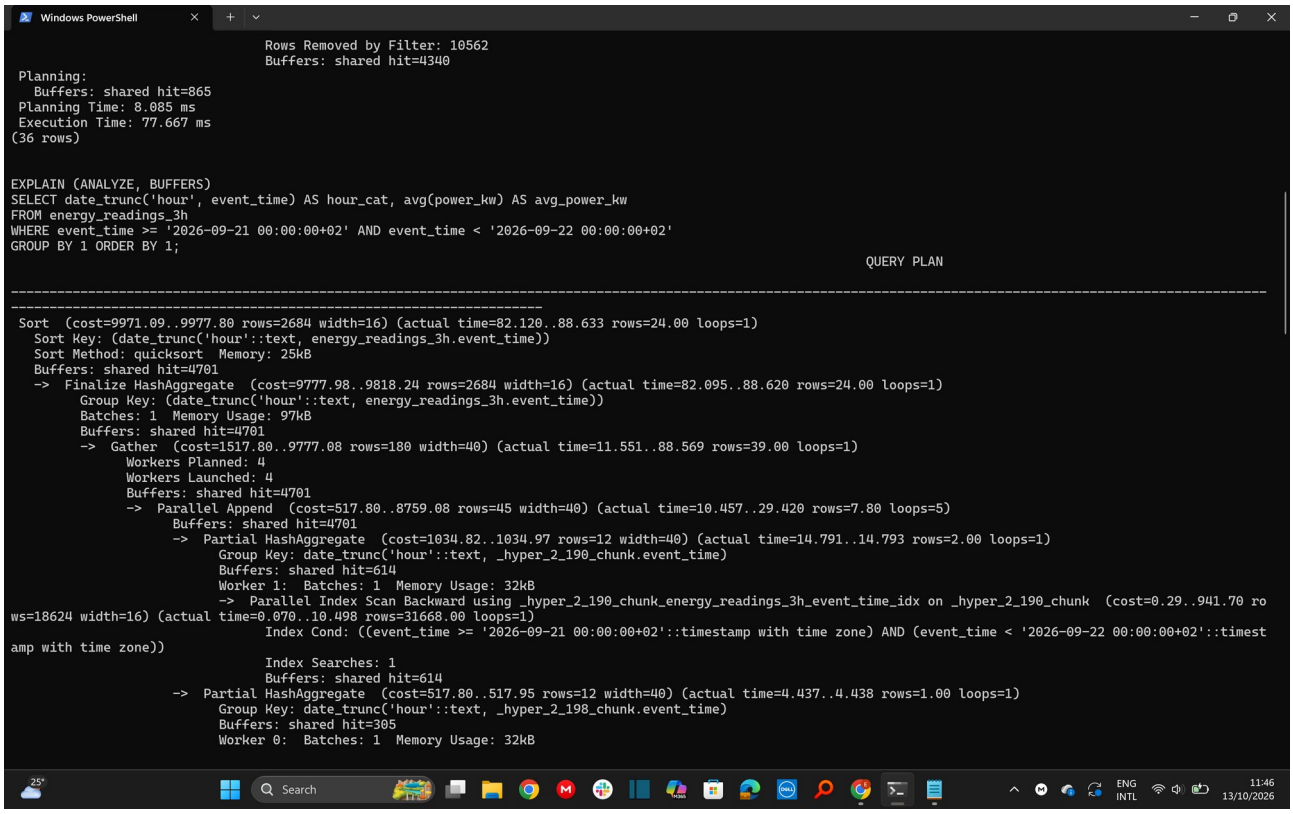
Execution plans with actual timing and buffer statistics for each configuration are in the screenshots below (sql\06_explain_q1.sql).



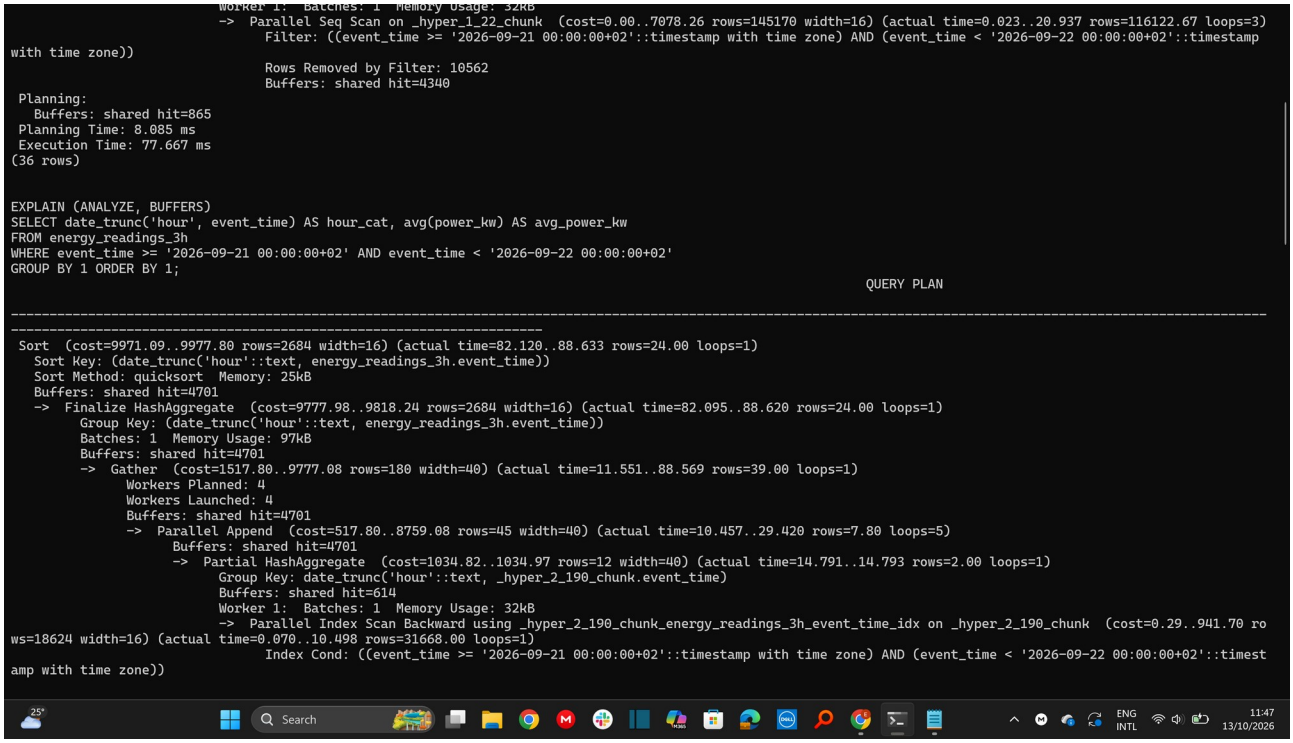
Screenshot 3.2-1 (Screenshot 2026-10-13 114407.png)



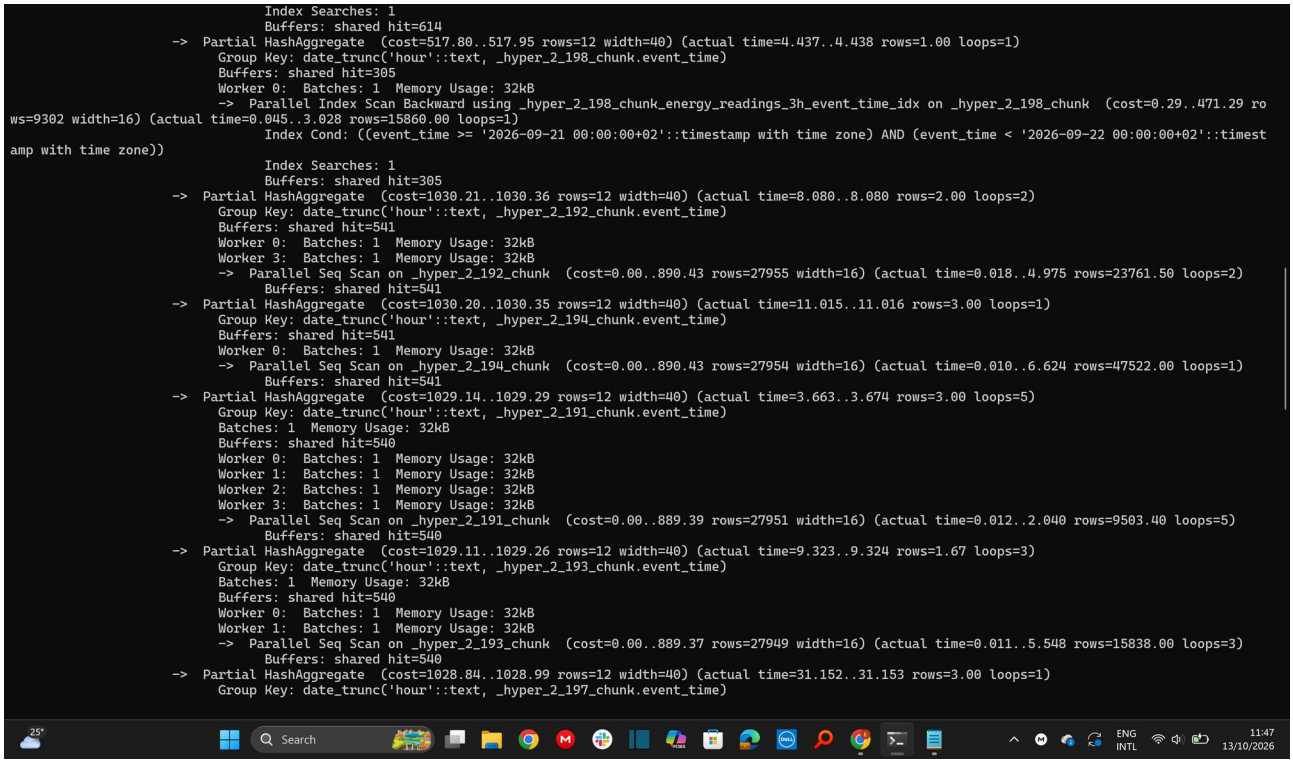
Screenshot 3.2-2 (Screenshot 2026-10-13 114543.png)



Screenshot 3.2-3 (Screenshot 2026-10-13 114608.png)



Screenshot 3.2-4 (Screenshot 2026-10-13 114719.png)



Screenshot 3.2-5 (Screenshot 2026-10-13 114746.png)

```
                        Batches: 1  Memory Usage: 32kB
                        Buffers: shared hit=540
                        Worker 0:  Batches: 1  Memory Usage: 32kB
                        Worker 1:  Batches: 1  Memory Usage: 32kB
                        ->  Parallel Seq Scan on _hyper_2_193_chunk  (cost=0.00..889.37 rows=27949 width=16) (actual time=0.011..5.548 rows=15838.00 loops=3)
                              Buffers: shared hit=540
                  ->  Partial HashAggregate  (cost=1028.84..1028.99 rows=12 width=40) (actual time=31.152..31.153 rows=3.00 loops=1)
                        Group Key: date_trunc('hour'::text, _hyper_2_197_chunk.event_time)
                        Batches: 1  Memory Usage: 32kB
                        Buffers: shared hit=540
                        ->  Parallel Seq Scan on _hyper_2_197_chunk  (cost=0.00..889.17 rows=27934 width=16) (actual time=0.009..16.137 rows=47487.00 loops=1)
                              Buffers: shared hit=540
                  ->  Partial HashAggregate  (cost=1028.72..1028.87 rows=12 width=40) (actual time=12.607..12.609 rows=3.00 loops=1)
                        Group Key: date_trunc('hour'::text, _hyper_2_195_chunk.event_time)
                        Batches: 1  Memory Usage: 32kB
                        Buffers: shared hit=540
                        ->  Parallel Seq Scan on _hyper_2_195_chunk  (cost=0.00..889.09 rows=27927 width=16) (actual time=0.009..7.677 rows=47476.00 loops=1)
                              Buffers: shared hit=540
                  ->  Partial HashAggregate  (cost=1028.65..1028.80 rows=12 width=40) (actual time=10.527..10.529 rows=3.00 loops=1)
                        Group Key: date_trunc('hour'::text, _hyper_2_196_chunk.event_time)
                        Batches: 1  Memory Usage: 32kB
                        Buffers: shared hit=540
                        ->  Parallel Seq Scan on _hyper_2_196_chunk  (cost=0.00..889.04 rows=27923 width=16) (actual time=0.027..6.445 rows=47469.00 loops=1)
                              Buffers: shared hit=540
 Planning:
   Buffers: shared hit=599
 Planning Time: 27.893 ms
 Execution Time: 89.028 ms
(83 rows)


EXPLAIN (ANALYZE, BUFFERS)
SELECT date_trunc('hour', event_time) AS hour_cat, avg(power_kw) AS avg_power_kw
FROM energy_readings_week
WHERE event_time >= '2026-09-21 00:00:00+02' AND event_time < '2026-09-22 00:00:00+02'
GROUP BY 1 ORDER BY 1;
                                                
```

3.3 Explain the chunk results

Observation = measured medians. Explanation = our reasoning about the cause; where no plan was captured the explanation is a hypothesis.

Q1 - fastest: 1 day (71.1 ms). 3 hours was 7.7% slower (76.6 ms) and 1 week 9.0% slower (77.5 ms); the differences are small and the 1-day and 1-week maxima (179.6 and 184.3 ms) show run-to-run noise. Explanation: Q1 filters a single day, which maps to one 1-day chunk, eight 3-hour chunks or one weekly chunk holding 7 days of data.

Q2 - fastest: 1 day (147.0 ms). 3 hours was 11.2% slower (163.5 ms) and 1 week 37.8% slower (202.6 ms). Explanation: the 22-28 September filter touches 7 daily chunks, 56 three-hour chunks, or 2 weekly chunks (weekly boundaries fall on 17 and 24 September, so those two chunks cover 14 days for 7 requested), so weekly chunks read and filter about twice the needed rows.

Q3 - fastest: 1 week (537.8 ms). 1 day was 8.5% slower (583.7 ms) and 3 hours 5.6 times slower (3,019.5 ms; 5.2 times the daily time). Explanation (hypothesis, no plan captured): a full scan touches all 225 three-hour chunks, so per-chunk overhead and merging of 225 partial aggregates over 4,000 meters dominates.

Q4 - fastest: 1 week (2.2 ms). 1 day was 22.7% slower (2.7 ms) and 3 hours 3.8 times slower (8.4 ms). Explanation: the (meter_id, event_time) index makes the query cheap, so planning and chunk-exclusion overhead, which grows with the number of chunks touched (56 for 3 hours, 7 daily, 2 weekly), is the main cost; absolute differences are at most 6.2 ms.

Q5 - fastest: 3 hours (365.1 ms). 1 day was 10.0% slower (401.6 ms) and 1 week 12.8% slower (411.7 ms). Explanation (hypothesis): with 11 parallel workers allowed, many small chunks share the scan more evenly than five large weekly chunks.

Overall: no single interval wins every workload. The one-day interval is the best compromise (fastest on Q1 and Q2, within 10% on Q3 and Q5, and 0.5 ms from the best on Q4) and was used as the baseline for Section 4.

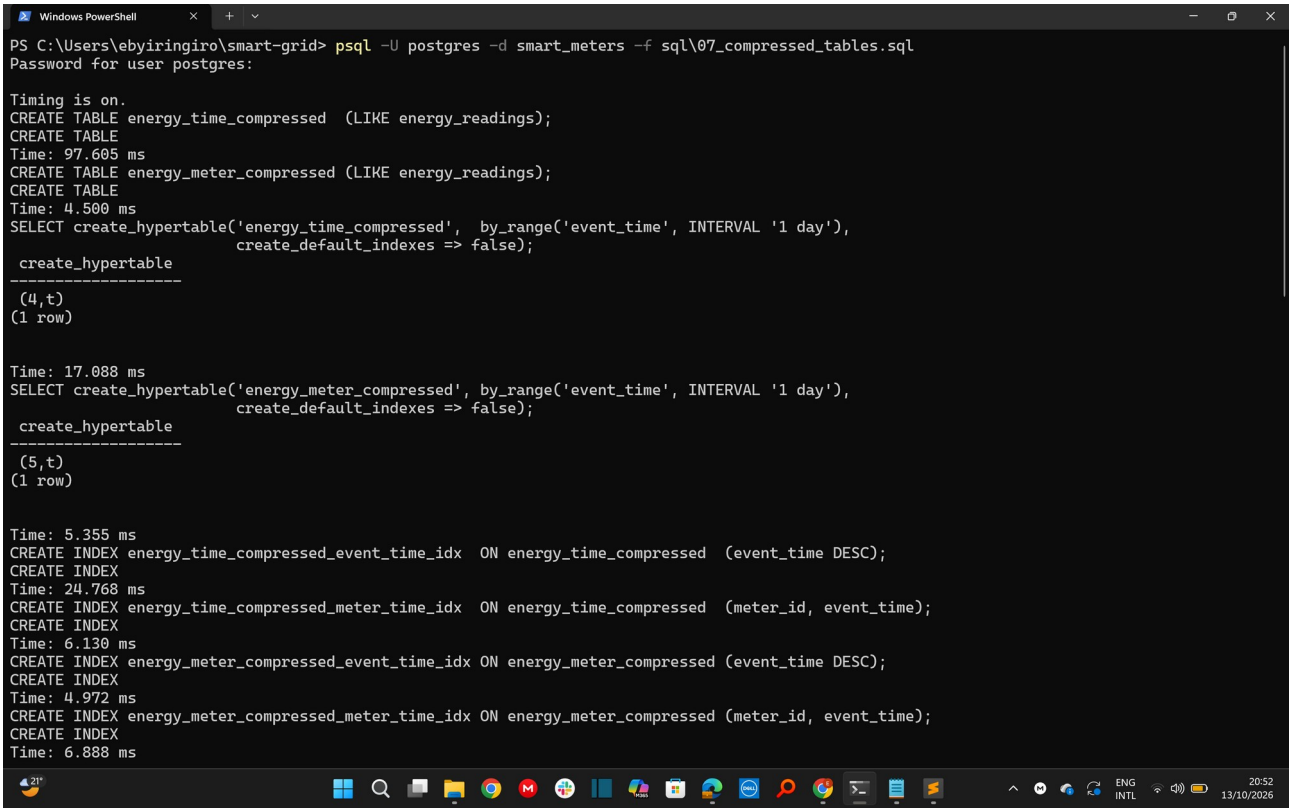
4 Compare compression strategies

4.1 Create the two compressed configurations

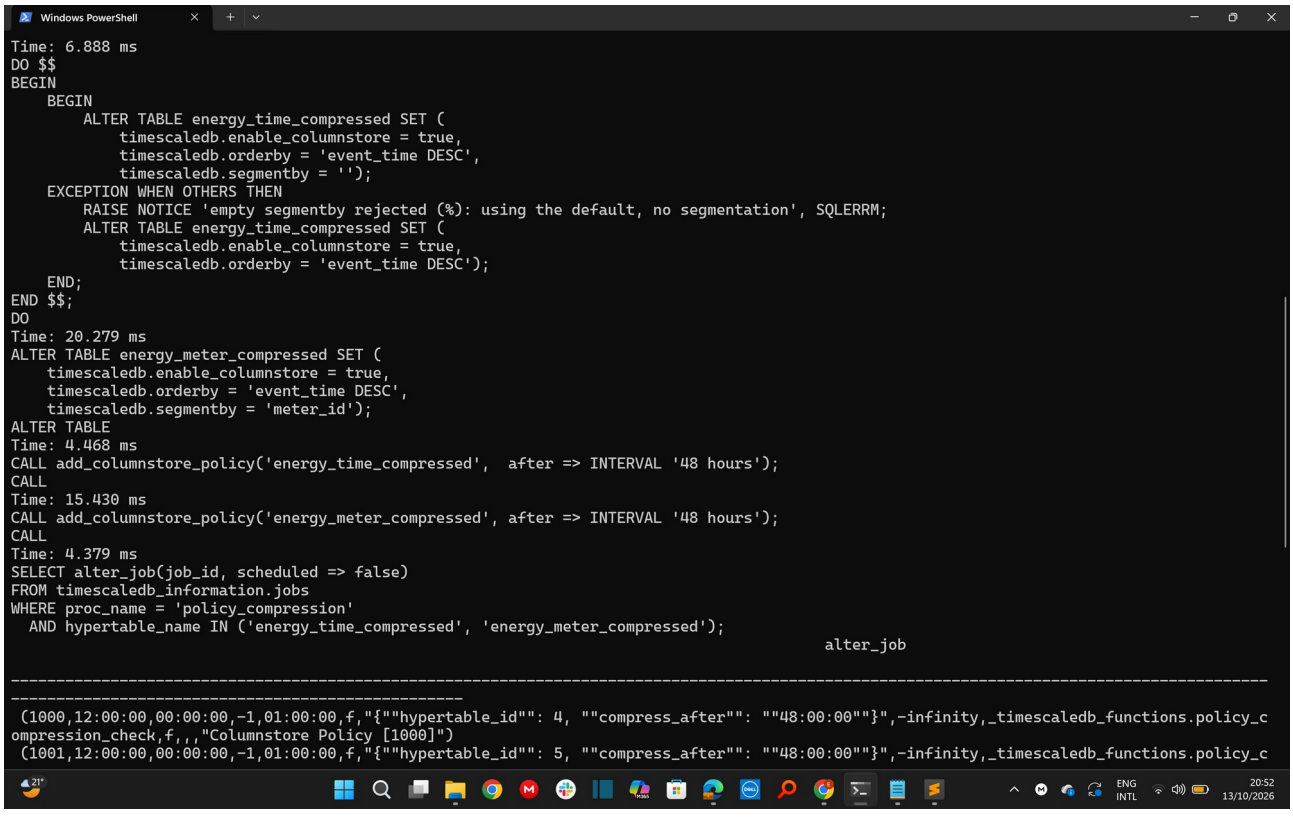
Table	Chunk interval	Order by	Segment by
energy_time_compressed	1 day	event_time DESC	none
energy_meter_compressed	1 day	event_time DESC	meter_id

The one-day interval was chosen from Section 3. Both tables have the same eight columns and matching indexes. A compression policy for chunks whose end is more than 48 hours old was added to each table and paused until the 4.3 measurements were complete. Experiment start: 2026-10-13 20:55:26 CAT; cutoff (start - 48 h): 2026-10-11 20:55:26 CAT, the same for both tables. compress_load.py copied the cleaned data one chunk at a time; after each chunk was fully loaded it was compressed manually because its end preceded the cutoff, and load and compression were timed separately (results\compression_load_log.csv).

All 29 chunks of both tables were compressed (every chunk ends before the cutoff), covering the same time ranges. Load time was 209.9 s (time-ordered) and 204.3 s (meter-segmented), excluded from compression cost. Data checks on both new tables passed: 10,643,735 rows, 4,000 meters, earliest event_time 2026-09-01 00:00:00+02, latest 2026-09-28 23:45:00+02, power sum difference below 0.000001 kW and energy difference 0 kWh against the daily table (all_checks_pass = true).



Screenshot 4.1-1 (Screenshot 2026-10-13 205227.png)



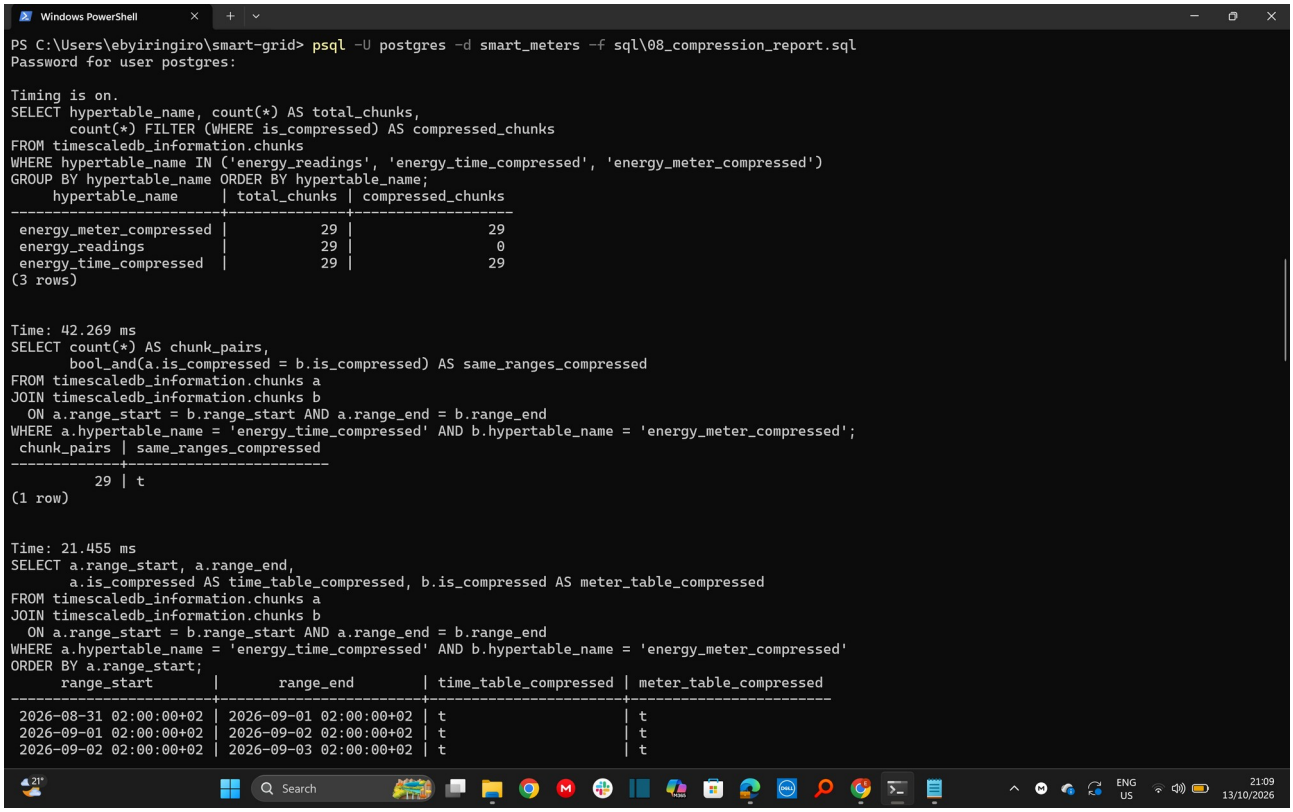
Screenshot 4.1-2 (Screenshot 2026-10-13 205249.png)

```
-----------------------------------------------------------------------------------------------------------------------------------------------------------
-------------------------------------------------
 (1000,12:00:00,00:00:00,-1,01:00:00,f,"{""hypertable_id"": 4, ""compress_after"": ""48:00:00""}",-infinity,_timescaledb_functions.policy_c
ompression_check,f,,,"Columnstore Policy [1000]")
 (1001,12:00:00,00:00:00,-1,01:00:00,f,"{""hypertable_id"": 5, ""compress_after"": ""48:00:00""}",-infinity,_timescaledb_functions.policy_c
ompression_check,f,,,"Columnstore Policy [1001]")
(2 rows)


Time: 27.408 ms
SELECT hypertable, segmentby, orderby
FROM timescaledb_information.hypertable_columnstore_settings
WHERE hypertable::text IN ('energy_time_compressed', 'energy_meter_compressed')
ORDER BY hypertable::text;
       hypertable        | segmentby |     orderby
-------------------------+-----------+-----------------
 energy_meter_compressed | meter_id  | event_time DESC
 energy_time_compressed  |           | event_time DESC
(2 rows)


Time: 17.223 ms
SELECT job_id, hypertable_name, proc_name, scheduled, schedule_interval, config
FROM timescaledb_information.jobs
WHERE hypertable_name IN ('energy_time_compressed', 'energy_meter_compressed')
ORDER BY hypertable_name;
 job_id |     hypertable_name     |     proc_name      | scheduled | schedule_interval |                        config

--------+-------------------------+--------------------+-----------+-------------------+-----------------------------------------------------------
-
   1001 | energy_meter_compressed | policy_compression | f         | 12:00:00          | {"hypertable_id": 5, "compress_after": "48:00:00"}
   1000 | energy_time_compressed  | policy_compression | f         | 12:00:00          | {"hypertable_id": 4, "compress_after": "48:00:00"}
(2 rows)


Time: 2.267 ms
PS C:\Users\ebyiringiro\smart-grid>
```

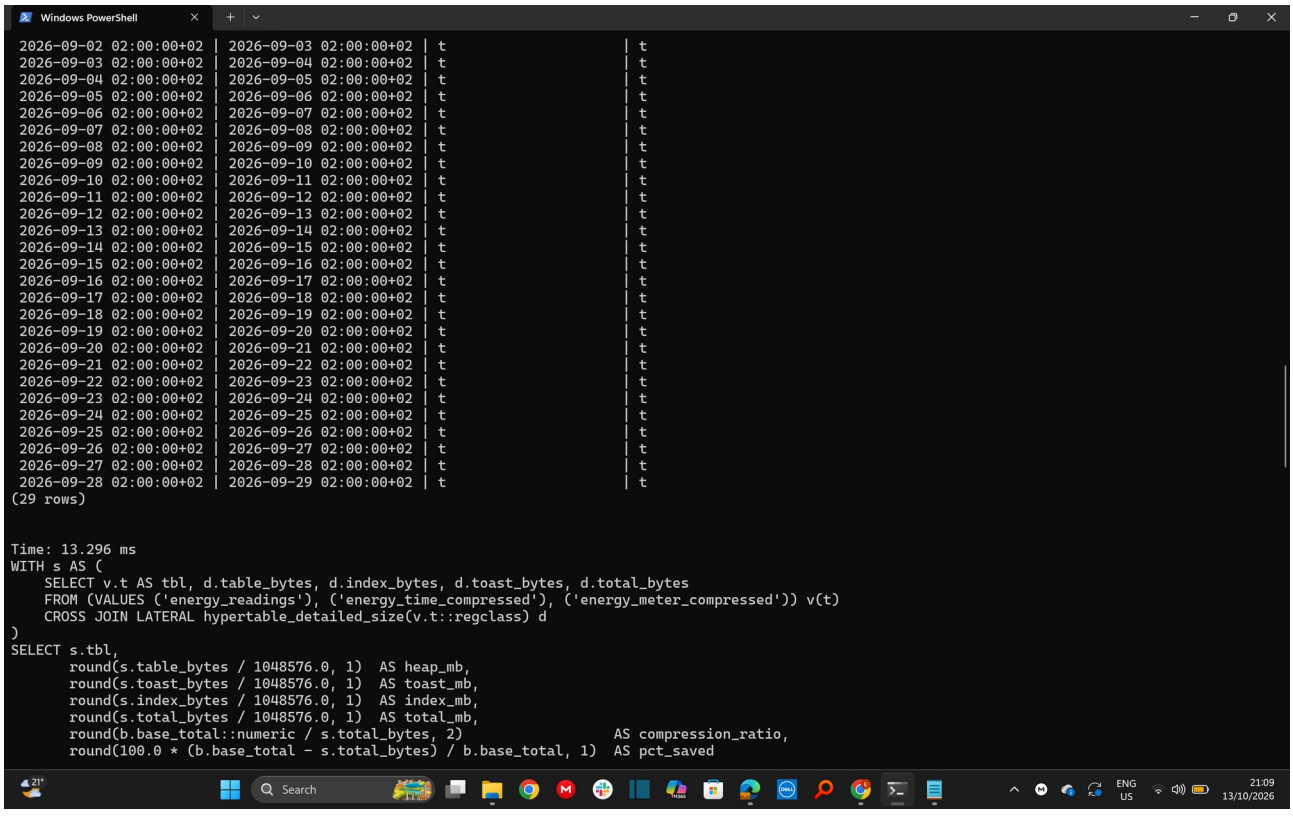
Screenshot 4.1-3 (Screenshot 2026-10-13 205304.png)

```
Windows PowerShell
PS C:\Users\ebyiringiro\smart-grid> python compress_load.py
Experiment start (CAT) : 2026-10-13T20:55:26+02:00
Cutoff (start - 48 h)  : 2026-10-11T20:55:26+02:00
Source chunks          : 29

=== energy_time_compressed ===
  2026-08-31 02:00  rows   31,717  load    0.3 s  compress    0.2 s
  2026-09-01 02:00  rows  380,089  load    2.8 s  compress    1.4 s
  2026-09-02 02:00  rows  380,087  load    7.5 s  compress    1.2 s
  2026-09-03 02:00  rows  380,151  load    7.5 s  compress    1.2 s
  2026-09-04 02:00  rows  380,019  load    7.6 s  compress    1.2 s
  2026-09-05 02:00  rows  380,126  load    7.6 s  compress    1.2 s
  2026-09-06 02:00  rows  380,144  load    7.6 s  compress    1.2 s
  2026-09-07 02:00  rows  380,113  load    7.4 s  compress    1.1 s
  2026-09-08 02:00  rows  380,122  load    7.5 s  compress    1.1 s
  2026-09-09 02:00  rows  380,149  load    7.6 s  compress    1.2 s
  2026-09-10 02:00  rows  380,123  load    8.1 s  compress    1.2 s
  2026-09-11 02:00  rows  380,123  load    7.9 s  compress    1.2 s
  2026-09-12 02:00  rows  380,151  load    7.8 s  compress    1.1 s
  2026-09-13 02:00  rows  380,203  load    7.8 s  compress    1.1 s
  2026-09-14 02:00  rows  380,096  load    9.5 s  compress    1.2 s
  2026-09-15 02:00  rows  380,100  load    7.6 s  compress    1.1 s
  2026-09-16 02:00  rows  380,156  load    7.8 s  compress    1.1 s
  2026-09-17 02:00  rows  380,204  load    7.9 s  compress    1.1 s
  2026-09-18 02:00  rows  380,088  load    7.4 s  compress    1.0 s
  2026-09-19 02:00  rows  380,211  load    7.4 s  compress    1.1 s
  2026-09-20 02:00  rows  380,212  load    7.3 s  compress    1.0 s
  2026-09-21 02:00  rows  380,053  load    7.4 s  compress    1.1 s
  2026-09-22 02:00  rows  380,152  load    7.7 s  compress    1.1 s
  2026-09-23 02:00  rows  380,121  load    7.6 s  compress    1.1 s
  2026-09-24 02:00  rows  380,185  load    7.6 s  compress    1.1 s
  2026-09-25 02:00  rows  380,157  load    7.5 s  compress    1.1 s
  2026-09-26 02:00  rows  380,119  load    7.6 s  compress    1.1 s
  2026-09-27 02:00  rows  380,147  load    7.6 s  compress    1.1 s
  2026-09-28 02:00  rows  348,417  load    7.1 s  compress    1.1 s
--- energy_time_compressed SUMMARY ---
Started / finished (CAT)       : 20:55:26 / 20:59:28
Rows loaded / in table         : 10,643,735 / 10,643,735   (MATCH with 10,643,735)
Load seconds (sum)             : 210.0
Compression seconds (sum)      : 32.3   <- the compression cost for 4.2
Compressed chunks              : 29 of 29
Rows in compressed chunks      : 10,643,735 = 100.00% of the cleaned rows
VACUUM ANALYZE (excluded)      : 2.7 s
```

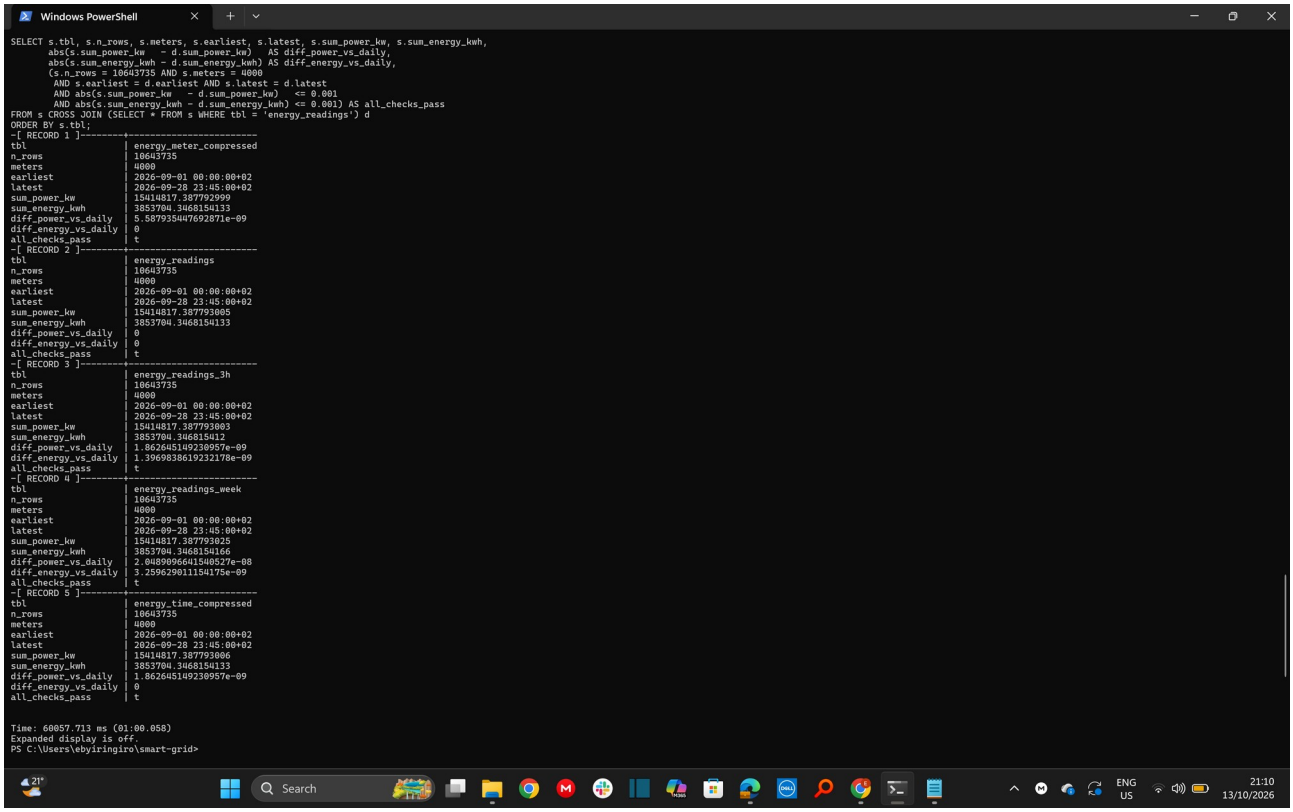
Screenshot 4.1-4 (Screenshot 2026-10-13 210420.png)



Screenshot 4.1-5 (Screenshot 2026-10-13 210938.png)



Screenshot 4.1-6 (Screenshot 2026-10-13 210954.png)

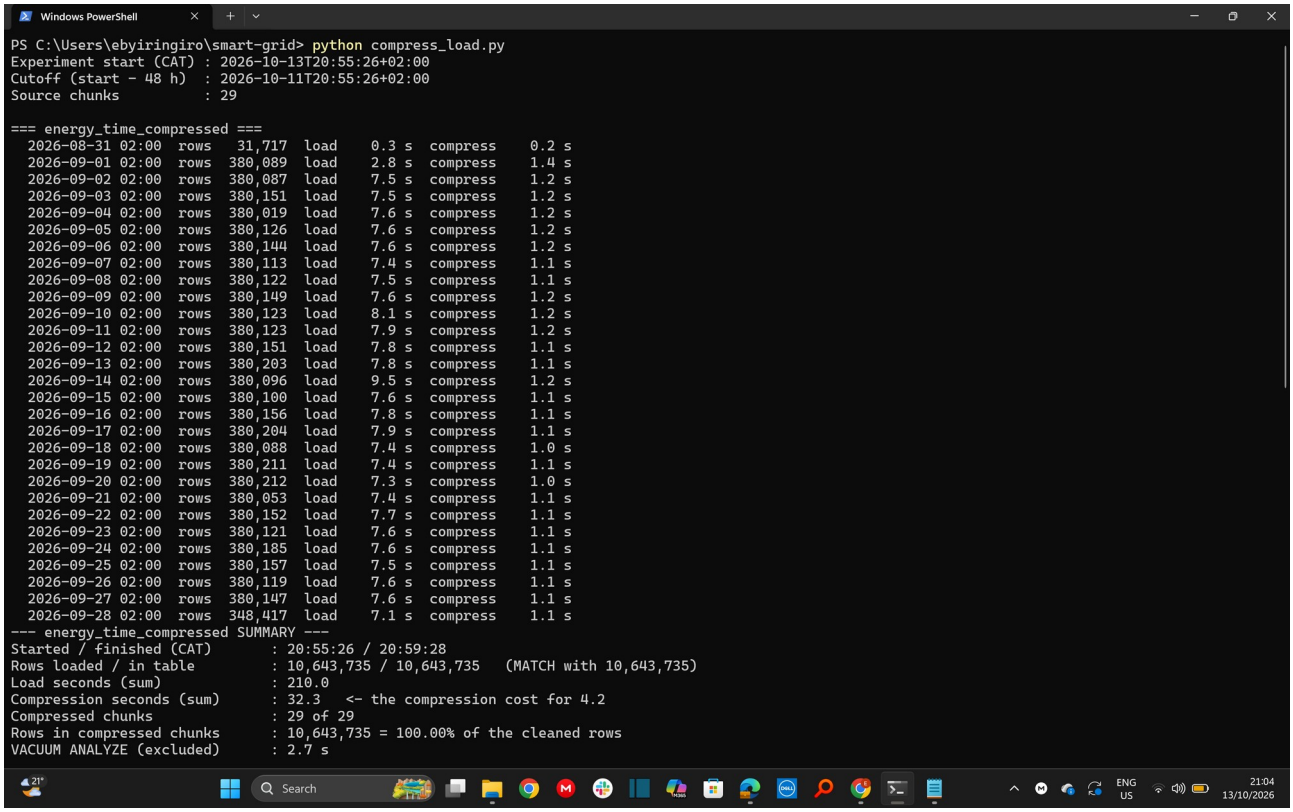


Screenshot 4.1-7 (Screenshot 2026-10-13 211028.png)

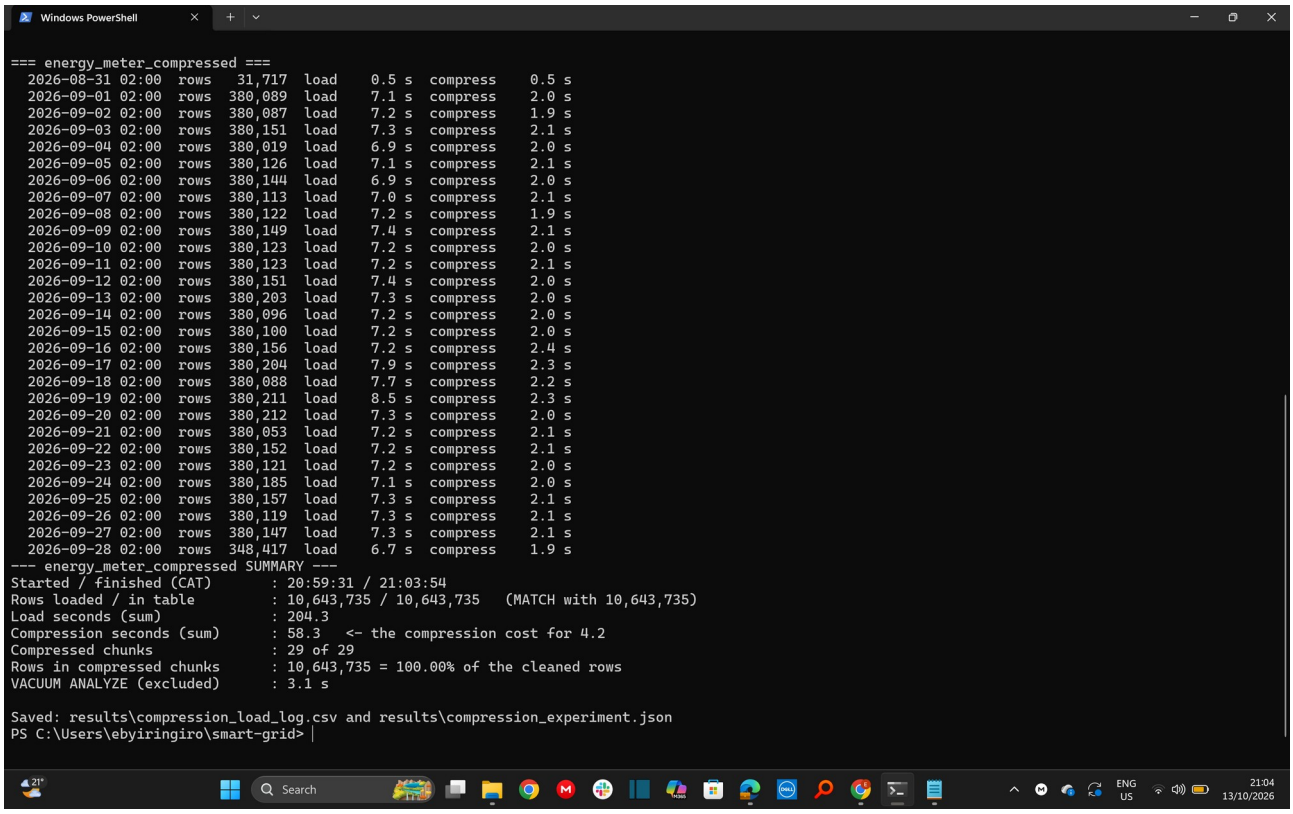
4.2 Measure storage and compression cost

Table	Data MB	Index MB	Total MB	Ratio	Saved %	Compress s	Compressed chunks	Rows in compressed %
energy_readings (baseline)	950.9	405.8	1,356.7	-	-	-	-	-
energy_time_compressed	437.8	0.9	438.7	3.09	67.7	32.3	29 of 29	100.00
energy_meter_compressed	496.1	5.5	501.6	2.70	63.0	58.3	29 of 29	100.00

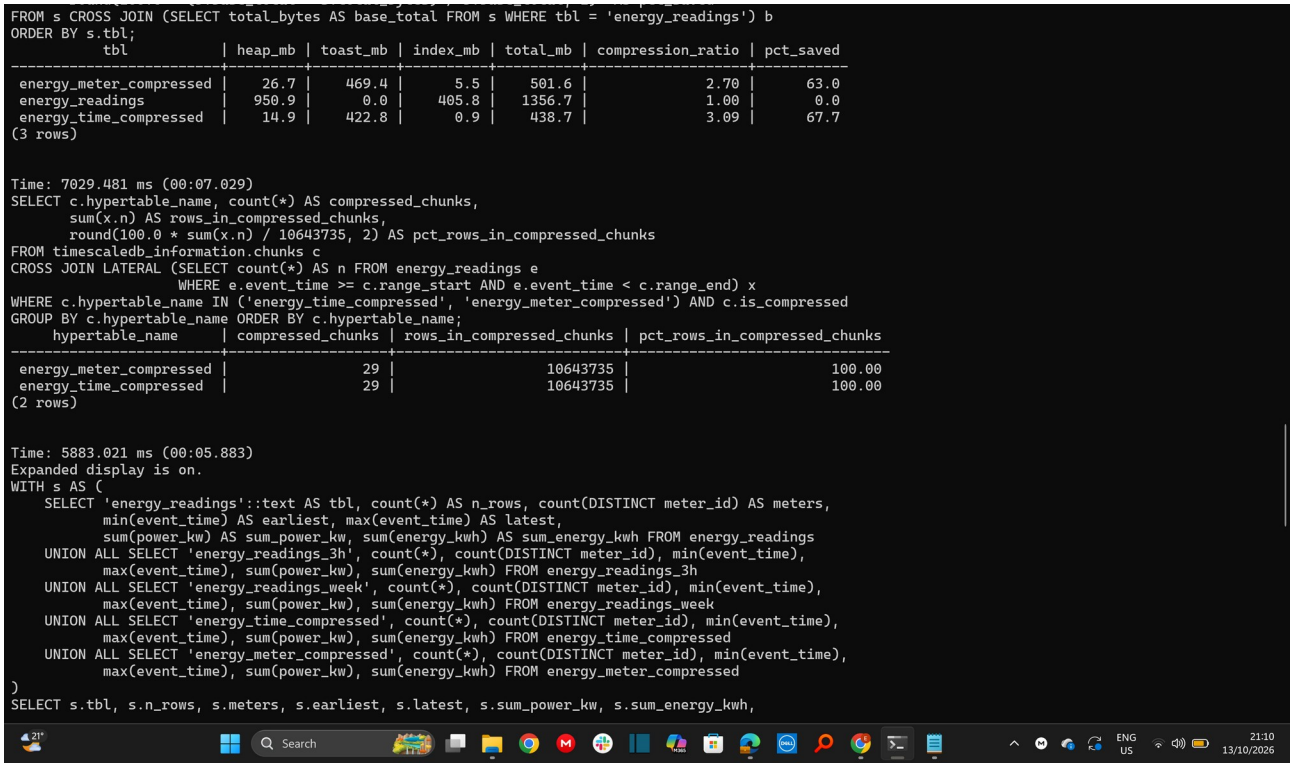
Ratio = baseline total / compressed total; Saved % = 100 x (baseline - compressed) / baseline; compression cost is the sum of the measured chunk compression times, excluding loading.



Screenshot 4.2-1 (Screenshot 2026-10-13 210420.png)



Screenshot 4.2-2 (Screenshot 2026-10-13 210432.png)



Screenshot 4.2-3 (Screenshot 2026-10-13 211009.png)

4.3 Repeat the query comparison

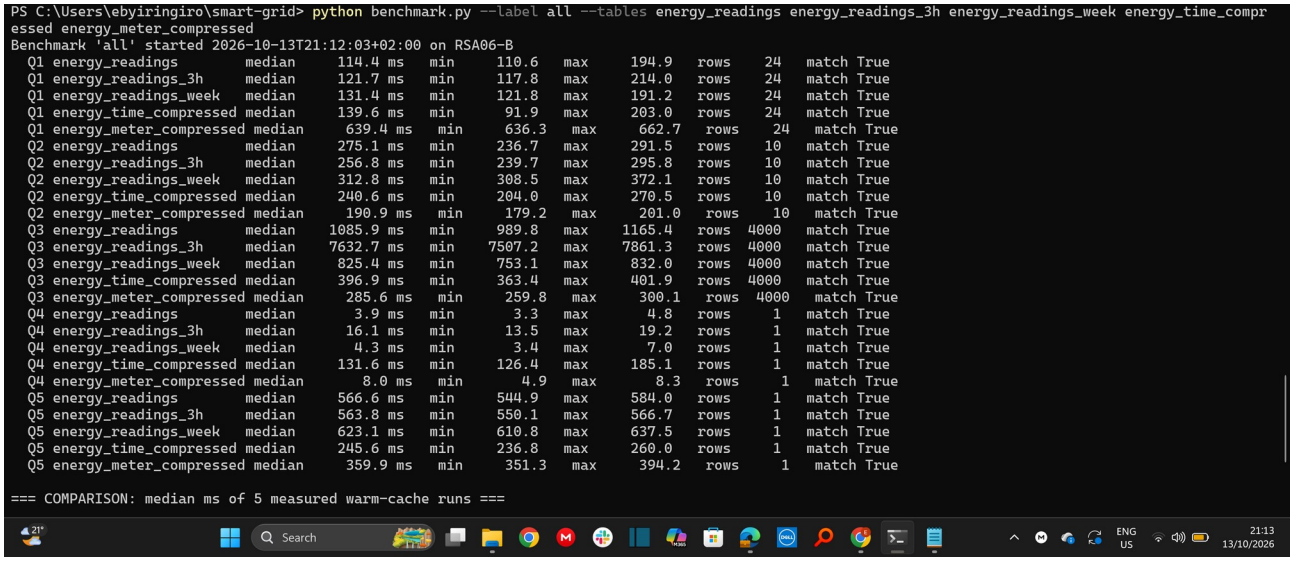
Query	1 day	3 hours	1 week	time-ordered	meter-segmented
Q1	114.4 (110.6-194.9)	121.7 (117.8-214.0)	131.4 (121.8-191.2)	139.6 (91.9-203.0)	639.4 (636.3-662.7)
Q2	275.1 (236.7-291.5)	256.8 (239.7-295.8)	312.8 (308.5-372.1)	240.6 (204.0-270.5)	190.9 (179.2-201.0)
Q3	1,085.9 (989.8-1,165.4)	7,632.7 (7,507.2-7,861.3)	825.4 (753.1-832.0)	396.9 (363.4-401.9)	285.6 (259.8-300.1)
Q4	3.9 (3.3-4.8)	16.1 (13.5-19.2)	4.3 (3.4-7.0)	131.6 (126.4-185.1)	8.0 (4.9-8.3)
Q5	566.6 (544.9-584.0)	563.8 (550.1-566.7)	623.1 (610.8-637.5)	245.6 (236.8-260.0)	359.9 (351.3-394.2)

Median (min-max) in ms, warm cache, results matched the reference for every query and table.

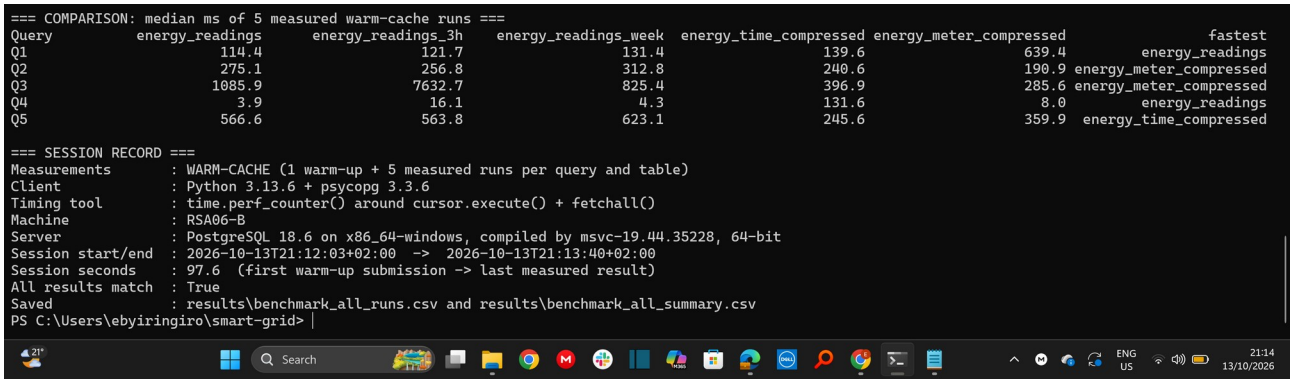
Query	Baseline (1 day) ms	time-ordered ms	Improvement %	meter-segmented ms	Improvement %
Q1	114.4	139.6	-22.0	639.4	-458.9
Q2	275.1	240.6	+12.5	190.9	+30.6
Q3	1,085.9	396.9	+63.4	285.6	+73.7
Q4	3.9	131.6	-3,274.4	8.0	-105.1
Q5	566.6	245.6	+56.7	359.9	+36.5

Improvement % = 100 x (baseline median - compressed median) / baseline median; positive means faster. The sum of the five medians is 2,045.9 ms (baseline), 1,154.3 ms (time-ordered) and 1,483.8 ms (meter-segmented).

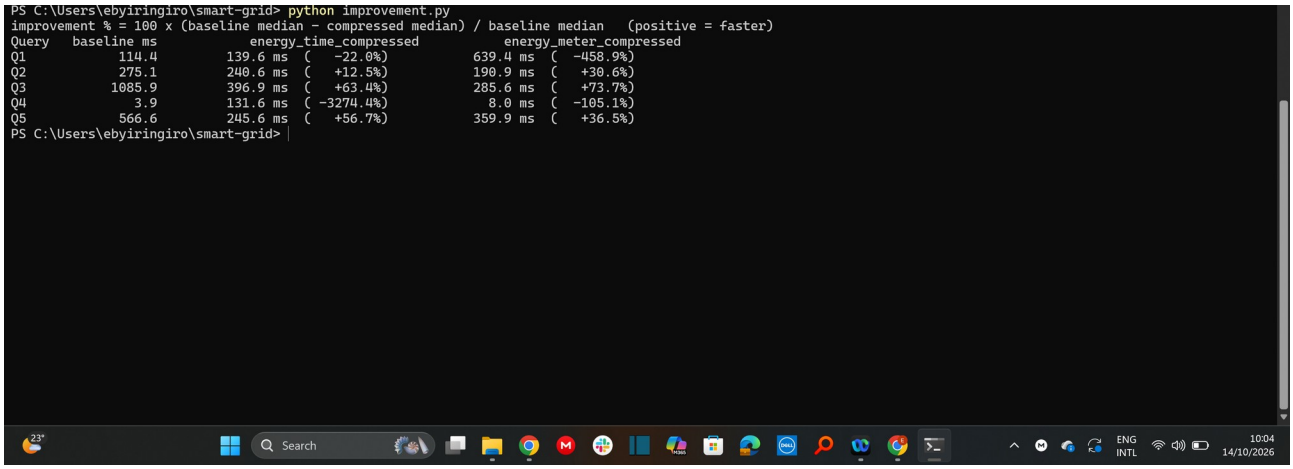
Compression policies were resumed after the measurements; both show as enabled (jobs 1000 and 1001, next run 2026-10-14 21:20 CAT at the time of the pause/resume check).



Screenshot 4.3-1 (Screenshot 2026-10-13 211358.png)

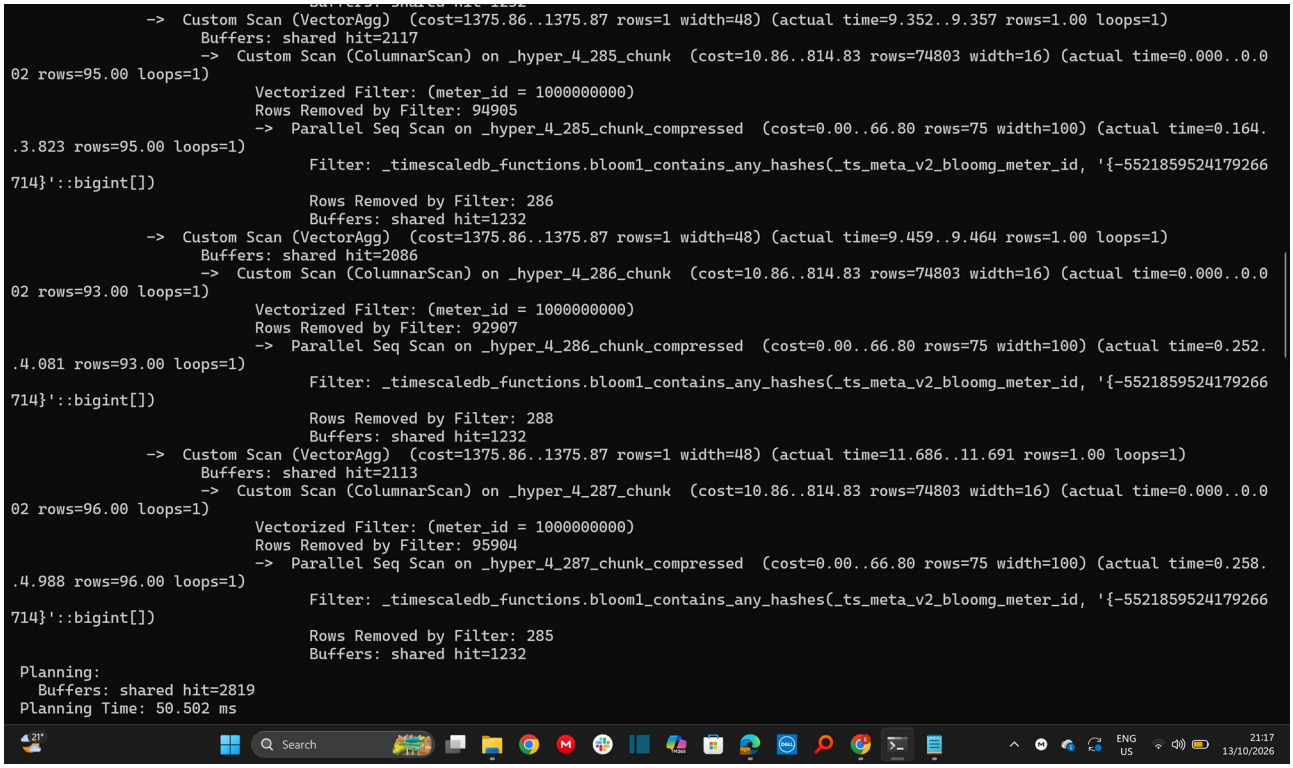


Screenshot 4.3-2 (Screenshot 2026-10-13 211412.png)

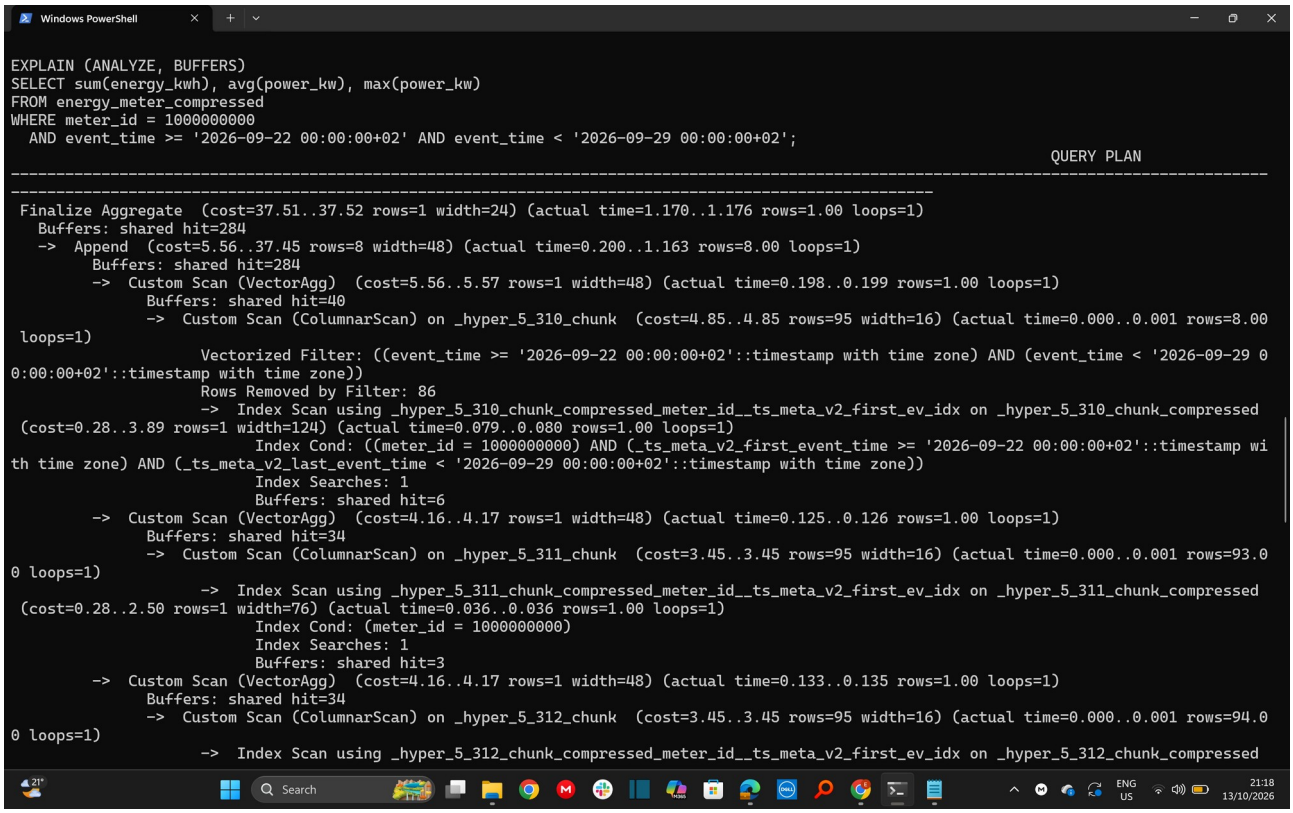


Screenshot 4.3-3 (Screenshot 2026-10-14 100413.png)

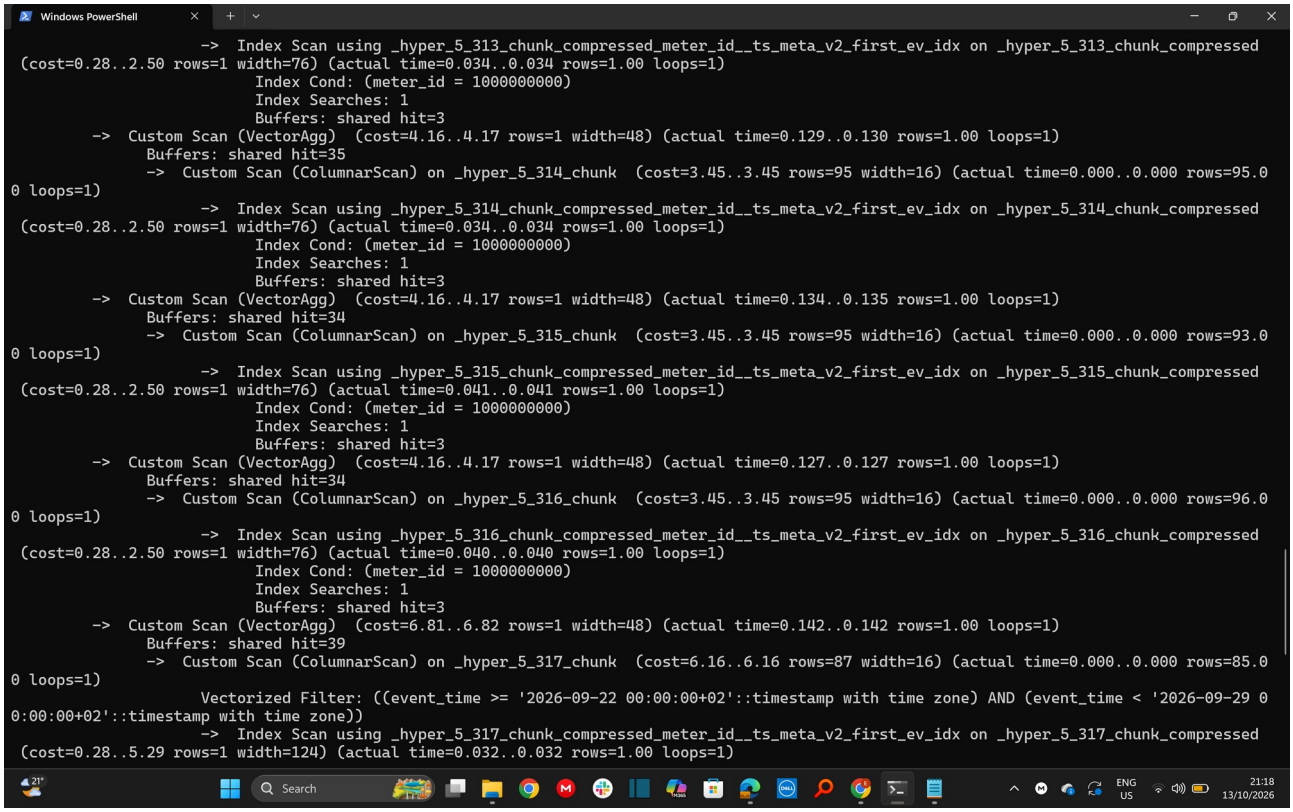
```
                     QUERY PLAN
------------------
```



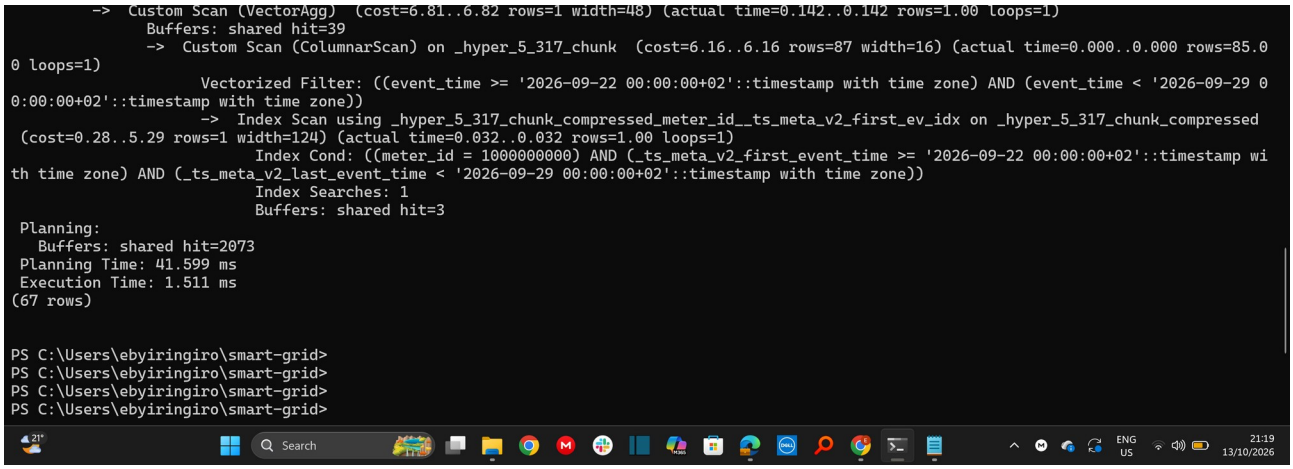
Screenshot 4.3-6 (Screenshot 2026-10-13 211803.png)



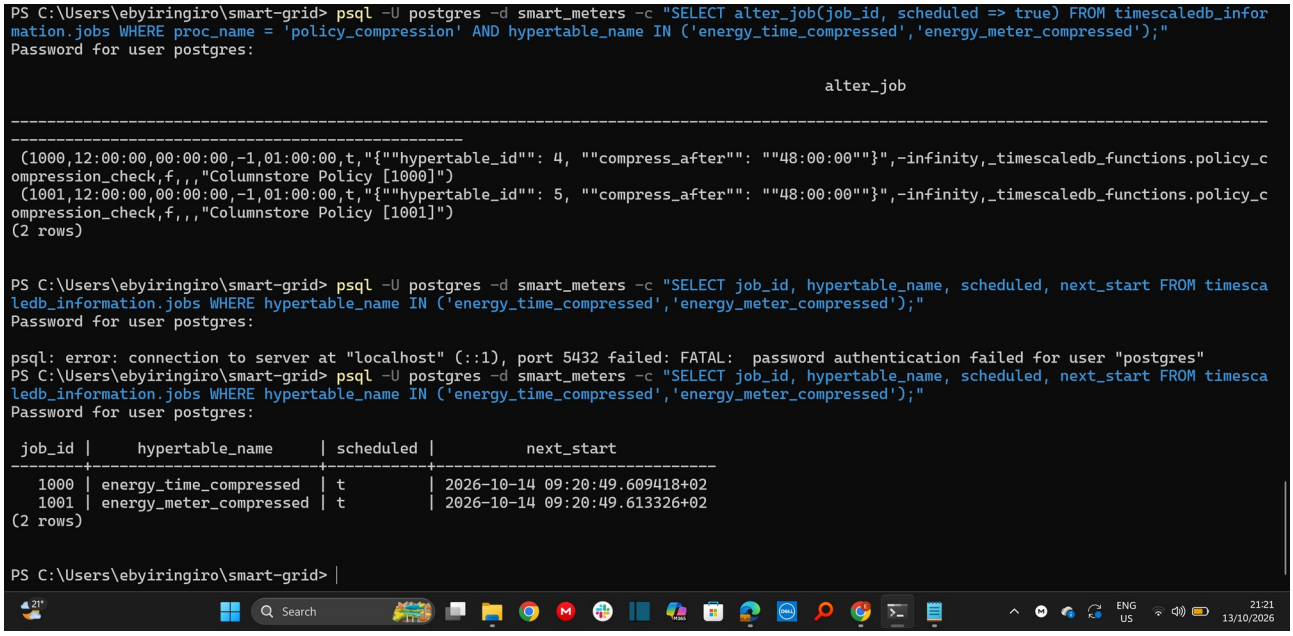
Screenshot 4.3-7 (Screenshot 2026-10-13 211821.png)



Screenshot 4.3-8 (Screenshot 2026-10-13 211900.png)



Screenshot 4.3-9 (Screenshot 2026-10-13 211921.png)



Screenshot 4.3-10 (Screenshot 2026-10-13 212211.png)

4.4 Select a storage configuration

Effect of meter segmentation (meter-segmented versus time-ordered table)

Workload	meter-segmented ms	time-ordered ms	Effect
Q4 single meter	8.0	131.6	93.9% faster
Q1 all meters, one day	639.4	139.6	358.0% slower
Q2 all meters, 7 days	190.9	240.6	20.7% faster
Q3 all meters, full period	285.6	396.9	28.0% faster
Q5 full scan	359.9	245.6	46.5% slower

The single-meter workload gains most from segmentation because the segment key lets the engine read only that meter's batch in each chunk instead of decompressing and filtering every batch; the segmented plan (screenshot 4.3-9) uses the meter_id and event_time index on the compressed chunk and shows 1.5 ms execution (41.6 ms planning). Use the medians, not single plan runs, for speed comparisons. Workloads across all meters gain or lose depending on whether they aggregate over time (Q2, Q3 faster) or must touch every segment of one day or the whole table (Q1, Q5 slower).

Recommendation. Use 1-day chunks with columnstore compression ordered by event_time DESC and segmented by meter_id.

Evidence. Total size 501.6 MB against 1,356.7 MB uncompressed (2.70 times, 63.0% saved); Q4 8.0 ms against 131.6 ms for the time-ordered table (3.9 ms uncompressed); Q3 285.6 ms against 1,085.9 ms uncompressed.

Tradeoff. Compression took 58.3 s against 32.3 s (1.80 times), the table is 14.3% larger than the time-ordered one (501.6 against 438.7 MB), Q1 is 5.6 times slower than uncompressed (639.4 against 114.4 ms) and the five medians sum to 1,483.8 ms against 1,154.3 ms for the time-ordered table. Stated assumption: per-meter drill-downs matter and the Section 6 aggregates serve network-wide queries; if network-wide scans dominated, the time-ordered table would be the better choice.

5 Investigate the utility data

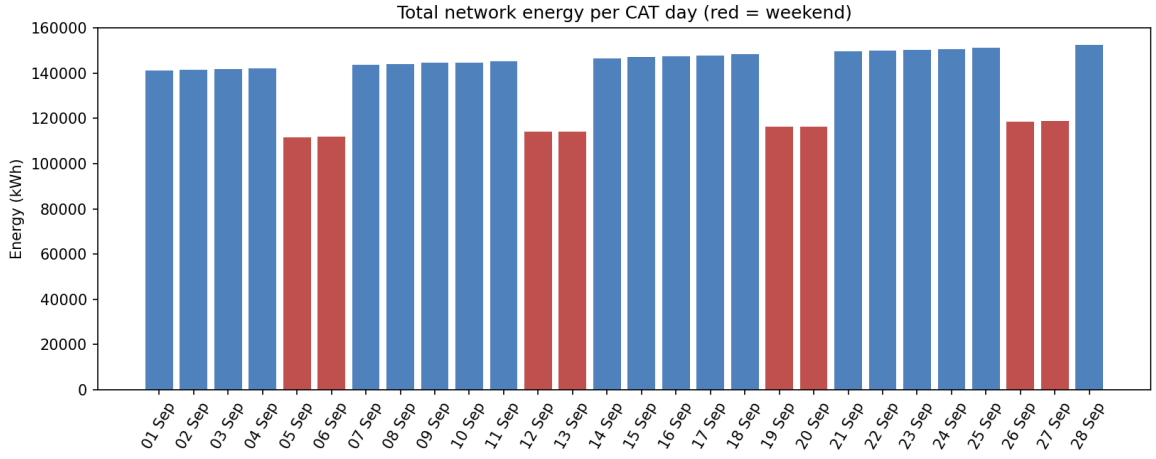
5.1 Identify demand patterns

Total network energy per CAT day (kWh), sum of energy_kwh

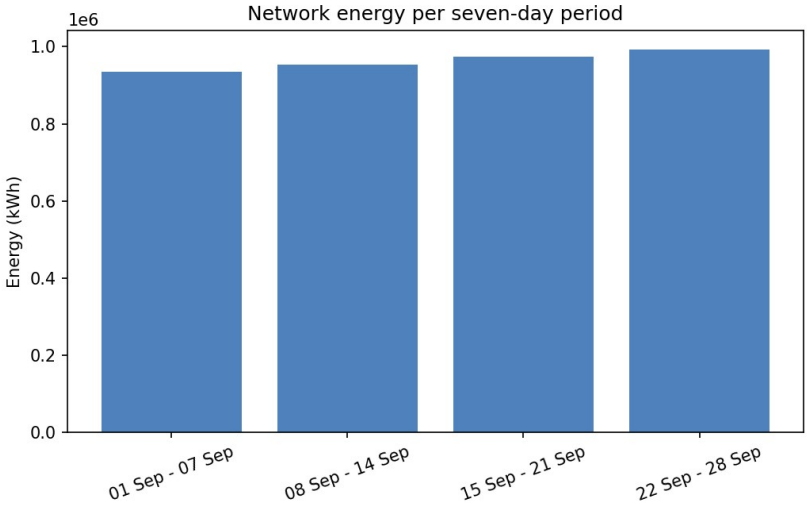
Day		kWh	Day		kWh
01 Sep	Tue	141,195.24	02 Sep	Wed	141,533.90
03 Sep	Thu	141,938.49	04 Sep	Fri	142,301.48
05 Sep	Sat	111,680.48	06 Sep	Sun	111,950.74
07 Sep	Mon	143,598.74	08 Sep	Tue	144,127.00

09 Sep	Wed	144,519.80	10 Sep	Thu	144,818.08
11 Sep	Fri	145,348.89	12 Sep	Sat	114,025.49
13 Sep	Sun	114,264.09	14 Sep	Mon	146,611.63
15 Sep	Tue	147,144.62	16 Sep	Wed	147,508.12
17 Sep	Thu	147,968.92	18 Sep	Fri	148,320.20
19 Sep	Sat	116,242.45	20 Sep	Sun	116,493.00
21 Sep	Mon	149,552.72	22 Sep	Tue	150,056.53
23 Sep	Wed	150,430.65	24 Sep	Thu	150,800.61
25 Sep	Fri	151,190.78	26 Sep	Sat	118,623.96
27 Sep	Sun	118,993.60	28 Sep	Mon	152,464.16

Seven-day period	Energy (kWh)
01-07 Sep	934,199.06
08-14 Sep	953,714.97
15-21 Sep	973,230.03
22-28 Sep	992,560.29



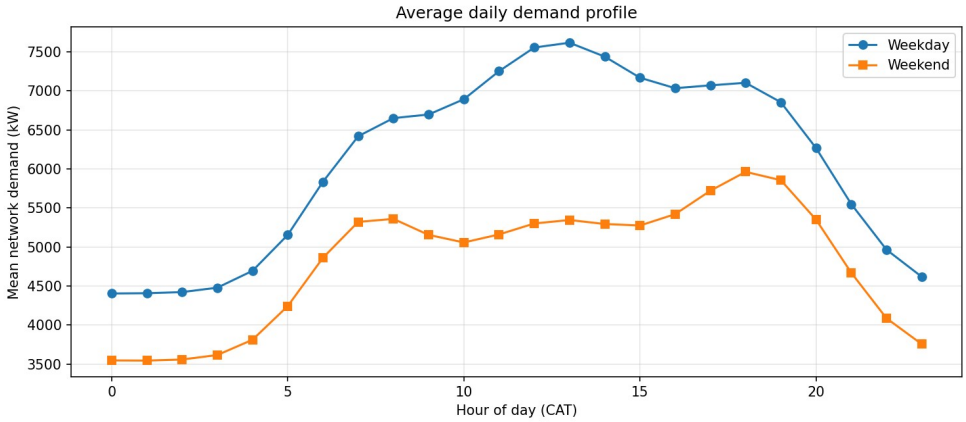
Total network energy per CAT day (red = weekend)



Network energy per seven-day period

Three highest 15-minute network demand intervals (sum of power_kw across meters)

Interval start (CAT)	Demand (kW)	Readings in interval
2026-09-28 13:00	7,993.69	3,965
2026-09-28 12:45	7,943.95	3,969
2026-09-28 13:15	7,943.64	3,952



Average network demand by CAT hour, weekday against weekend

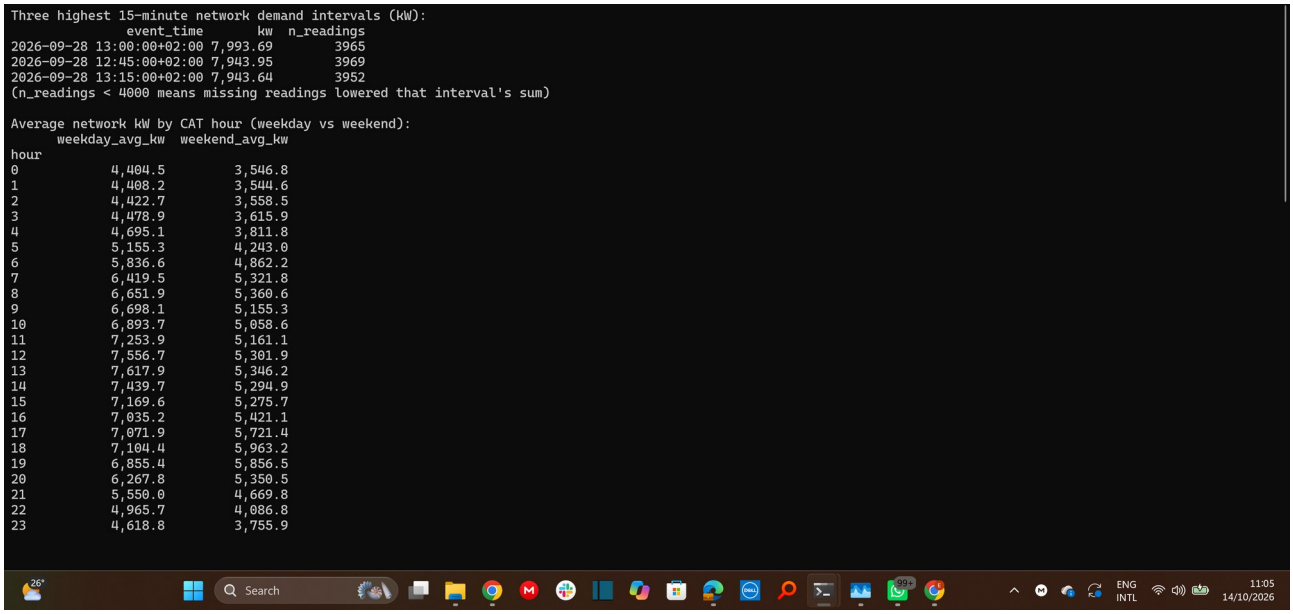
Patterns. Weekday energy rises about 0.3% per day and the seven-day totals grow about 2% a week (934,199 to 992,560 kWh, +6.2% from the first to the fourth week). Weekend days are about 21% lower (for example 111,680 kWh on Saturday 5 September against 141,195 kWh on Tuesday 1 September). Weekday demand averages 7,618 kW at 13:00 and falls to about 4,400 kW overnight; weekends peak later and lower (5,963 kW at 18:00).

Operational implications. All three peaks fall on Monday 28 September between 12:45 and 13:15, consistent with the weekday midday profile and the growth trend. Capacity and storage should be sized against the weekday midday kW peak, whose headroom shrinks about 2% a week; overnight base load is about 58% of the peak. The peak intervals have 3,952-3,969 of 4,000 readings, so missing readings slightly understate them.

kW versus kWh. kW is power, the rate of use at an instant (here averaged over 15 minutes); kWh is energy, power multiplied by time. A 15-minute interval at 7,993.69 kW is about 1,998 kWh. Capacity is limited by kW, while consumption and billing use kWh.

```
PS C:\Users\ebyiringiro\smart-grid> python section5_analysis.py
Section 5 analysis started 2026-10-14 11:03:08 CAT

================================================================================
5.1 Network totals per 15-minute interval (one scan)   [2026-10-14 11:03:08 CAT]
================================================================================
Intervals returned: 2688 (planned 2688)

Daily network energy (kWh):
         day         kwh    weekday
2026-09-01 141,195.24   Tuesday
2026-09-02 141,533.90 Wednesday
2026-09-03 141,938.49  Thursday
2026-09-04 142,301.48    Friday
2026-09-05 111,680.48  Saturday
2026-09-06 111,950.74    Sunday
2026-09-07 143,598.74    Monday
2026-09-08 144,127.00   Tuesday
2026-09-09 144,519.80 Wednesday
2026-09-10 144,818.08  Thursday
2026-09-11 145,348.89    Friday
2026-09-12 114,025.49  Saturday
2026-09-13 114,264.09    Sunday
2026-09-14 146,611.63    Monday
2026-09-15 147,144.62   Tuesday
2026-09-16 147,508.12 Wednesday
2026-09-17 147,968.92  Thursday
2026-09-18 148,320.20    Friday
2026-09-19 116,242.45  Saturday
2026-09-20 116,493.00    Sunday
2026-09-21 149,552.72    Monday
2026-09-22 150,056.53   Tuesday
2026-09-23 150,430.65 Wednesday
2026-09-24 150,800.61  Thursday
2026-09-25 151,190.78    Friday
2026-09-26 118,623.96  Saturday
2026-09-27 118,993.60    Sunday
2026-09-28 152,464.16    Monday

Seven-day period totals (kWh):
period_start period_end         kwh
      01 Sep     07 Sep 934,199.06
      08 Sep     14 Sep 953,714.97
      15 Sep     21 Sep 973,230.03
      22 Sep     28 Sep 992,560.29
```

Screenshot 5.1-1 (Screenshot 2026-10-14 110452.png)



Screenshot 5.1-2 (Screenshot 2026-10-14 110512.png)

5.2 Compare regions and customer types

Region	Meters	Total kWh	kWh per meter
Region 1	400	383,539.81	958.85
Region 2	400	378,448.89	946.12
Region 3	400	392,376.59	980.94
Region 4	400	324,404.48	811.01
Region 5	400	411,040.39	1,027.60
Region 6	400	346,512.84	866.28
Region 7	400	362,288.54	905.72
Region 8	400	503,475.83	1,258.69
Region 9	400	400,467.20	1,001.17
Region 10	400	351,149.78	877.87

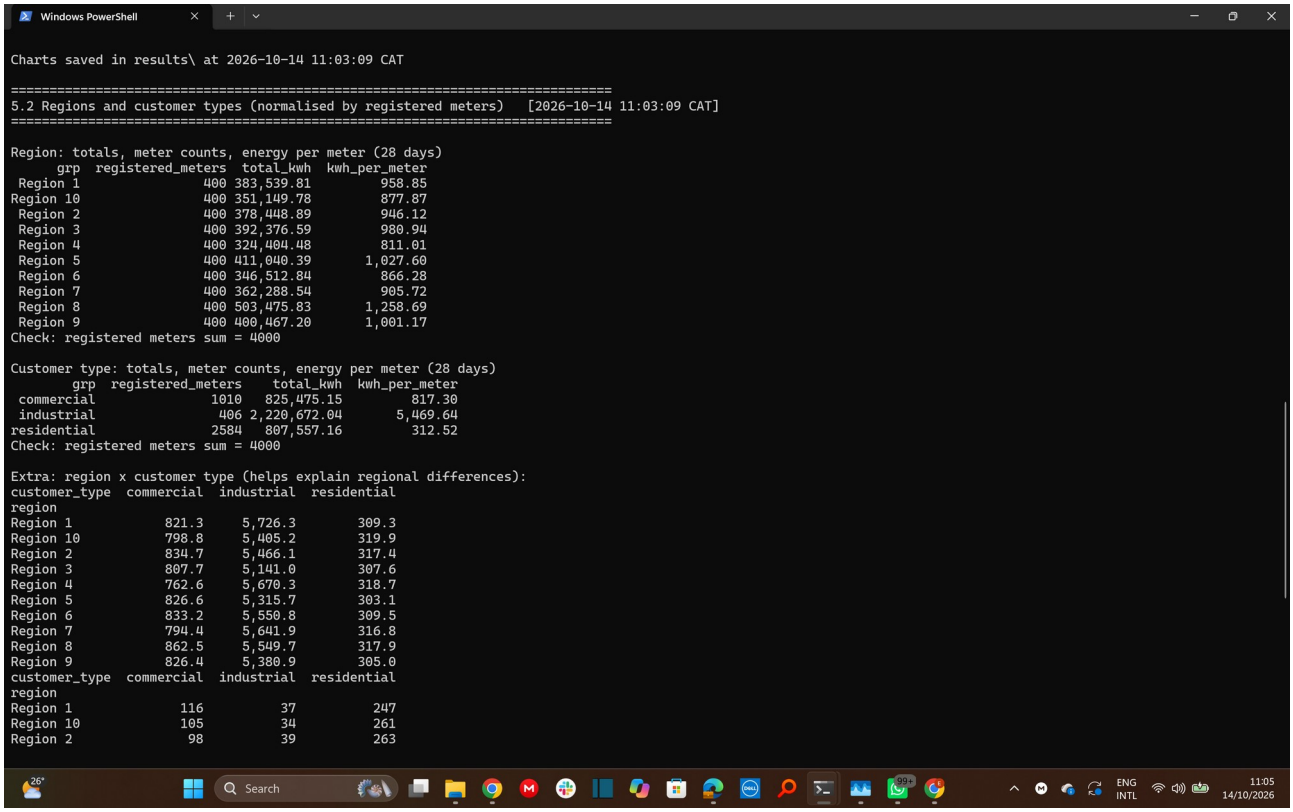
Customer type	Meters	Total kWh	kWh per meter
commercial	1,010	825,475.15	817.30
industrial	406	2,220,672.04	5,469.64
residential	2,584	807,557.16	312.52

Region x customer type: meters and kWh per meter

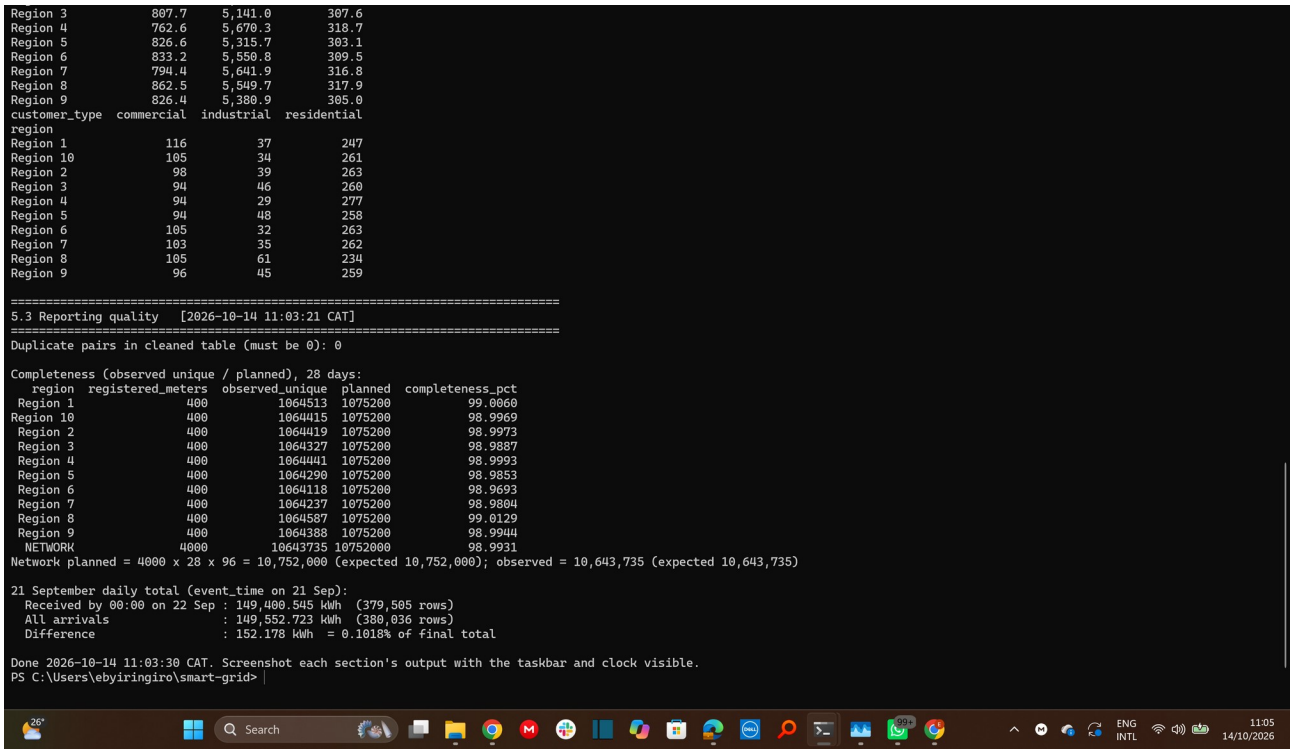
Region	comm. meters	ind. meters	res. meters	comm. kWh/m	ind. kWh/m	res. kWh/m
Region 1	116	37	247	821.3	5,726.3	309.3
Region 2	98	39	263	834.7	5,466.1	317.4
Region 3	94	46	260	807.7	5,141.0	307.6
Region 4	94	29	277	762.6	5,670.3	318.7
Region 5	94	48	258	826.6	5,315.7	303.1
Region 6	105	32	263	833.2	5,550.8	309.5
Region 7	103	35	262	794.4	5,641.9	316.8
Region 8	105	61	234	862.5	5,549.7	317.9
Region 9	96	45	259	826.4	5,380.9	305.0
Region 10	105	34	261	798.8	5,405.2	319.9

What remains after normalization. Customer type dominates: industrial meters are 10.2% of meters but use 57.6% of energy (5,470 kWh per meter against 313 for residential, about 17 times). Regions still differ per meter (Region 8 1,258.69 kWh, Region 4 811.01 kWh), but this is largely a customer-mix effect: Region 8 has 61 industrial meters (15.3%) against 29 (7.3%) in Region 4, and within each customer type the regions are much closer (residential 303-320, commercial 763-863, industrial 5,141-5,726 kWh per meter).

Implication. Do not read a regional gap as inherently higher demand; compare regions within a customer type and plan capacity around industrial customer counts.



Screenshot 5.2-1 (Screenshot 2026-10-14 110535.png)



Screenshot 5.2-2 (Screenshot 2026-10-14 110604.png)

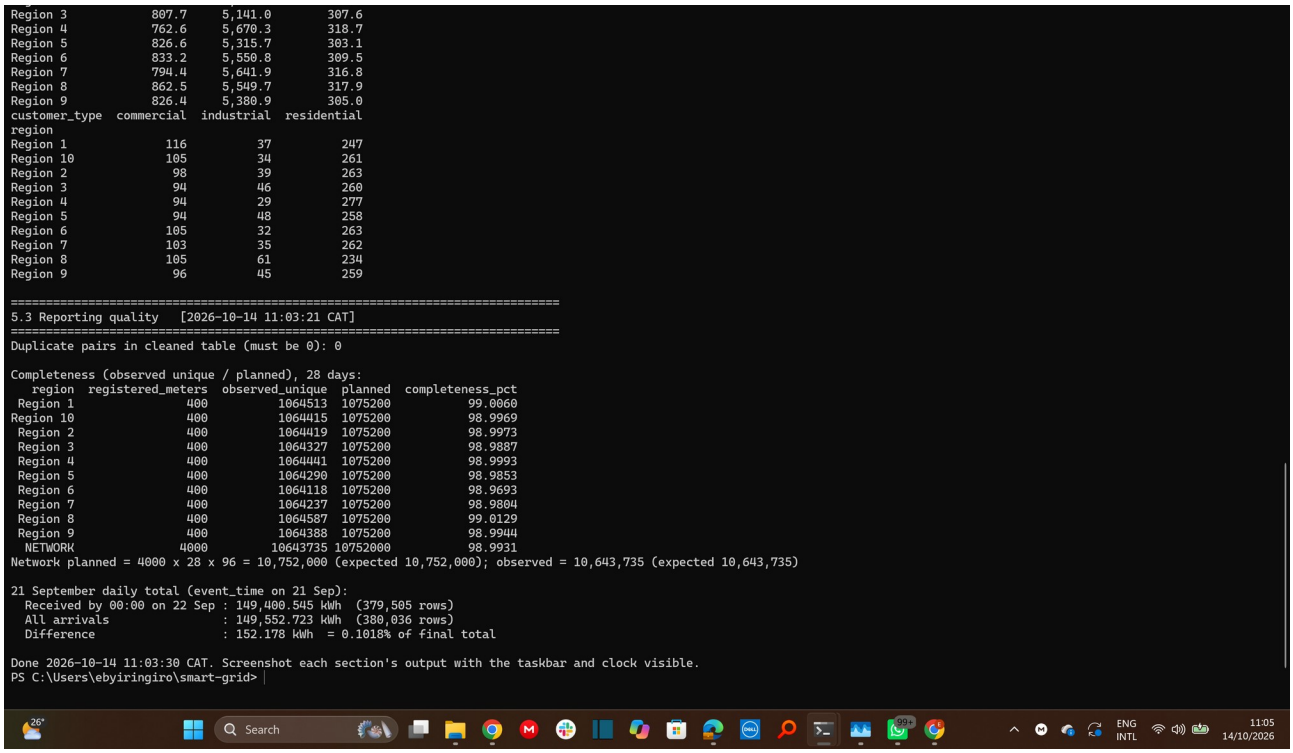
5.3 Assess reporting quality

Region	Registered meters	Planned readings	Observed unique	Completeness %
Region 1	400	1,075,200	1,064,513	99.0060
Region 2	400	1,075,200	1,064,419	98.9973
Region 3	400	1,075,200	1,064,327	98.9887
Region 4	400	1,075,200	1,064,441	98.9993
Region 5	400	1,075,200	1,064,290	98.9853
Region 6	400	1,075,200	1,064,118	98.9693
Region 7	400	1,075,200	1,064,237	98.9804
Region 8	400	1,075,200	1,064,587	99.0129
Region 9	400	1,075,200	1,064,388	98.9944

Region 10	400	1,075,200	1,064,415	98.9969
NETWORK	4,000	10,752,000	10,643,735	98.9931

21 September (events on that CAT day)	Energy (kWh)	Rows
Received by 00:00 on 22 September	149,400.545	379,505
All arrivals	149,552.723	380,036
Difference	152.178 kWh (0.1018% of the final total)	531

Effects. Completeness is about 99% everywhere (98.969% to 99.013%), so missing readings are spread evenly and no region is disproportionately affected. They are permanent gaps: the final total is a lower bound (21 September has 380,036 of 384,000 planned rows). Delayed readings are recoverable once they arrive, so a report issued at midnight understates the day by 0.10%.



Screenshot 5.3-1 (Screenshot 2026-10-14 110604.png)

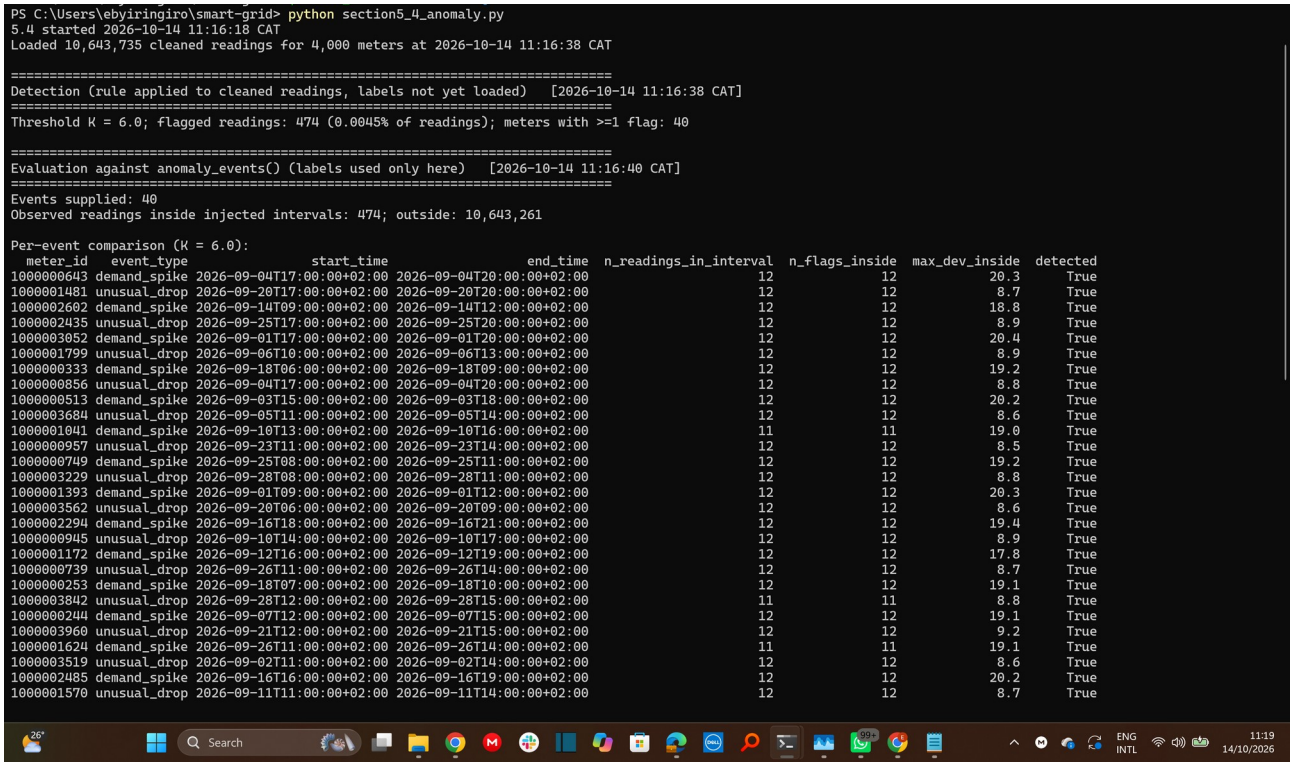
5.4 Investigate abnormal consumption

Rule. For each customer type and event_time the reference is the mean power_kw of that group. Each meter reading is divided by that reference (removing the shared daily, weekly and growth pattern). For each meter the median ratio and the MAD (scaled by 1.4826) are computed over its own ratios; a reading is flagged when |ratio - median| / MAD > 6 (two-sided, so spikes and drops are both caught). K = 6 is a conservative cut-off (a deviation of six robust standard deviations). In section5_4_anomaly.py it is set as a constant and the detection step runs on the cleaned readings before anomaly_events() is imported; the labels are used only in the evaluation step.

Result (K = 6)	Value
Flagged readings	474 (0.0045% of readings), on 40 meters
Detected events	40 of 40
Missed events	0 of 40
False flags (outside injected intervals)	0 = 0.00000% of 10,643,261 observed readings
Meters with at least one false flag	0 of 4,000 = 0.000% of meters affected by false flags
Readings inside injected intervals	474 observed, all flagged (6 events had one missing reading: 40 x 12 = 480 planned)

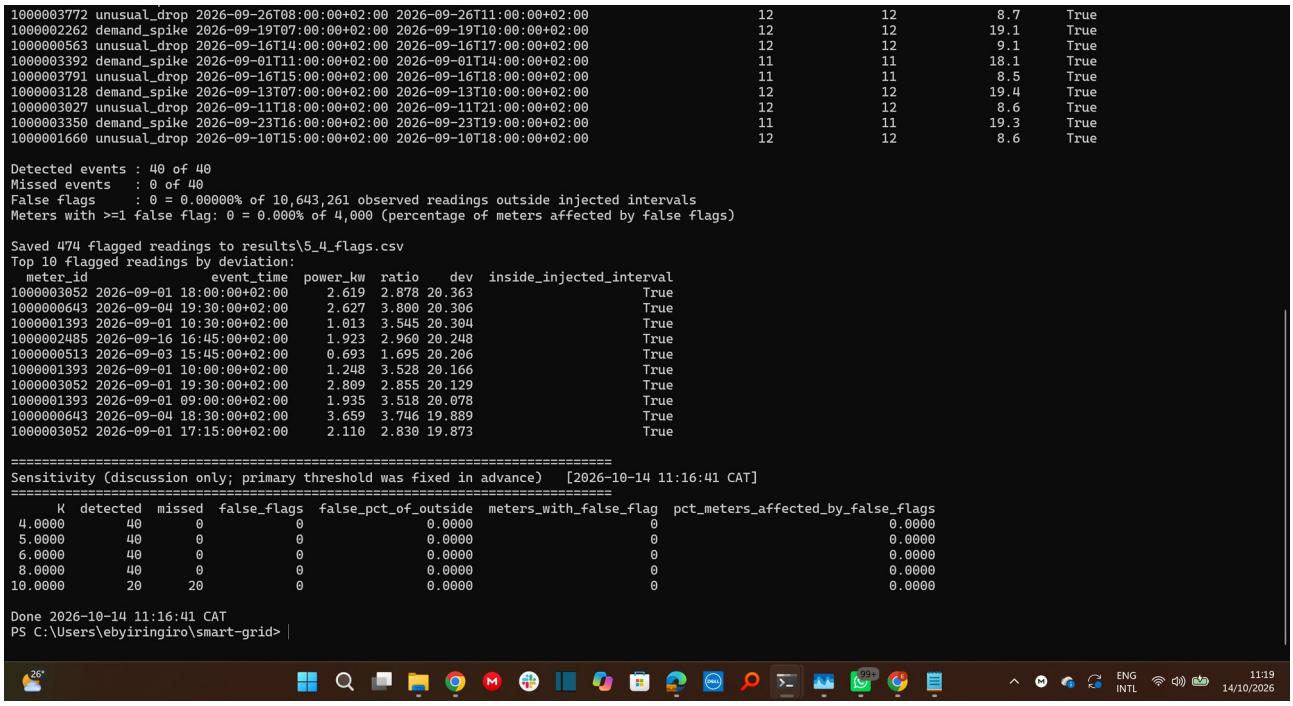
Threshold K	Detected	Missed	False flags
4	40	0	0
5	40	0	0
6 (chosen)	40	0	0
8	40	0	0
10	20	20	0

Explanation. Demand spikes scored 17.7-20.4 and unusual drops 8.5-9.2. A drop is bounded below at zero, so its deviation cannot grow like a spike's; at K = 10 exactly the 20 drop events are missed. There were no false alarms: normal readings stay below a deviation of 4 (K = 4 also gives 0 false flags), so there is a wide margin. This clean result reflects simulated data with well-behaved noise; on real data holidays, outages and new customers would cause false alarms, and an event whose readings were all missing could not be detected. The sensitivity table is for discussion only: the evaluated detection uses the pre-set K = 6, and the other values show how robust that choice is.



```
PS C:\Users\ebyiringiro\smart-grid> python section5_4_anomaly.py
5.4 started 2026-10-14 11:16:18 CAT
Loaded 10,643,735 cleaned readings for 4,000 meters at 2026-10-14 11:16:38 CAT

======================================================================
Detection (rule applied to cleaned readings, labels not yet loaded)   [2026-10-14 11:16:38 CAT]
======================================================================
Threshold K = 6.0; flagged readings: 474 (0.0045% of readings); meters with >=1 flag: 40

======================================================================
Evaluation against anomaly_events() (labels used only here)   [2026-10-14 11:16:40 CAT]
======================================================================
Events supplied: 40
Observed readings inside injected intervals: 474; outside: 10,643,261

Per-event comparison (K = 6.0):
  meter_id   event_type              start_time                end_time  n_readings_in_interval  n_flags_inside  max_dev_inside  detected
1000000643 demand_spike 2026-09-04T17:00:00+02:00 2026-09-04T20:00:00+02:00                      12              12            20.3      True
1000001481 unusual_drop 2026-09-20T17:00:00+02:00 2026-09-20T20:00:00+02:00                      12              12             8.7      True
1000002602 demand_spike 2026-09-14T09:00:00+02:00 2026-09-14T12:00:00+02:00                      12              12            18.8      True
1000002435 unusual_drop 2026-09-25T17:00:00+02:00 2026-09-25T20:00:00+02:00                      12              12             8.9      True
1000003052 demand_spike 2026-09-01T17:00:00+02:00 2026-09-01T20:00:00+02:00                      12              12            20.4      True
1000001799 unusual_drop 2026-09-06T10:00:00+02:00 2026-09-06T13:00:00+02:00                      12              12             8.9      True
1000000333 demand_spike 2026-09-18T06:00:00+02:00 2026-09-18T09:00:00+02:00                      12              12            19.2      True
1000000856 unusual_drop 2026-09-04T17:00:00+02:00 2026-09-04T20:00:00+02:00                      12              12             8.8      True
1000000513 demand_spike 2026-09-03T15:00:00+02:00 2026-09-03T18:00:00+02:00                      12              12            20.2      True
1000003684 unusual_drop 2026-09-05T11:00:00+02:00 2026-09-05T14:00:00+02:00                      12              12             8.6      True
1000001041 demand_spike 2026-09-10T13:00:00+02:00 2026-09-10T16:00:00+02:00                      11              11            19.0      True
1000000957 unusual_drop 2026-09-23T11:00:00+02:00 2026-09-23T14:00:00+02:00                      12              12             8.5      True
1000000749 demand_spike 2026-09-25T08:00:00+02:00 2026-09-25T11:00:00+02:00                      12              12            19.2      True
1000003229 unusual_drop 2026-09-28T08:00:00+02:00 2026-09-28T11:00:00+02:00                      12              12             8.8      True
1000001393 demand_spike 2026-09-01T09:00:00+02:00 2026-09-01T12:00:00+02:00                      12              12            20.3      True
1000003562 unusual_drop 2026-09-20T06:00:00+02:00 2026-09-20T09:00:00+02:00                      12              12             8.6      True
1000002294 demand_spike 2026-09-16T18:00:00+02:00 2026-09-16T21:00:00+02:00                      12              12            19.4      True
1000000945 unusual_drop 2026-09-10T14:00:00+02:00 2026-09-10T17:00:00+02:00                      12              12             8.9      True
1000001172 demand_spike 2026-09-12T16:00:00+02:00 2026-09-12T19:00:00+02:00                      12              12            17.8      True
1000000739 unusual_drop 2026-09-26T11:00:00+02:00 2026-09-26T14:00:00+02:00                      12              12             8.7      True
1000000253 demand_spike 2026-09-18T07:00:00+02:00 2026-09-18T10:00:00+02:00                      12              12            19.1      True
1000003842 unusual_drop 2026-09-28T12:00:00+02:00 2026-09-28T15:00:00+02:00                      11              11             8.8      True
1000002044 demand_spike 2026-09-07T12:00:00+02:00 2026-09-07T15:00:00+02:00                      12              12            19.1      True
1000003960 unusual_drop 2026-09-21T12:00:00+02:00 2026-09-21T15:00:00+02:00                      12              12             9.2      True
1000001624 demand_spike 2026-09-26T11:00:00+02:00 2026-09-26T14:00:00+02:00                      11              11            19.1      True
1000003519 unusual_drop 2026-09-02T11:00:00+02:00 2026-09-02T14:00:00+02:00                      12              12             8.6      True
1000002485 demand_spike 2026-09-16T16:00:00+02:00 2026-09-16T19:00:00+02:00                      12              12            20.2      True
1000001570 unusual_drop 2026-09-11T11:00:00+02:00 2026-09-11T14:00:00+02:00                      12              12             8.7      True
```

Screenshot 5.4-1 (Screenshot 2026-10-14 111935.png)



```
1000003772 unusual_drop 2026-09-26T08:00:00+02:00 2026-09-26T11:00:00+02:00                      12              12             8.7      True
1000002262 demand_spike 2026-09-19T07:00:00+02:00 2026-09-19T10:00:00+02:00                      12              12            19.1      True
1000000563 unusual_drop 2026-09-16T14:00:00+02:00 2026-09-16T17:00:00+02:00                      12              12             9.1      True
1000003392 demand_spike 2026-09-01T11:00:00+02:00 2026-09-01T14:00:00+02:00                      11              11            18.1      True
1000003791 unusual_drop 2026-09-16T15:00:00+02:00 2026-09-16T18:00:00+02:00                      11              11             8.5      True
1000003128 demand_spike 2026-09-13T07:00:00+02:00 2026-09-13T10:00:00+02:00                      12              12            19.4      True
1000003027 unusual_drop 2026-09-11T18:00:00+02:00 2026-09-11T21:00:00+02:00                      12              12             8.6      True
1000003350 demand_spike 2026-09-23T16:00:00+02:00 2026-09-23T19:00:00+02:00                      11              11            19.3      True
1000001660 unusual_drop 2026-09-10T15:00:00+02:00 2026-09-10T18:00:00+02:00                      12              12             8.6      True

Detected events : 40 of 40
Missed events   : 0 of 40
False flags     : 0 = 0.00000% of 10,643,261 observed readings outside injected intervals
Meters with >=1 false flag: 0 = 0.000% of 4,000 (percentage of meters affected by false flags)

Saved 474 flagged readings to results\5_4_flags.csv
Top 10 flagged readings by deviation:
  meter_id           event_time  power_kw  ratio    dev  inside_injected_interval
1000003052 2026-09-01 18:00:00+02:00     2.619  2.878 20.363                      True
1000000643 2026-09-04 19:30:00+02:00     2.627  3.800 20.306                      True
1000001393 2026-09-01 10:30:00+02:00     1.013  3.545 20.304                      True
1000002485 2026-09-16 16:45:00+02:00     1.923  2.960 20.248                      True
1000000513 2026-09-03 15:45:00+02:00     0.693  1.695 20.206                      True
1000001393 2026-09-01 10:00:00+02:00     1.248  3.528 20.166                      True
1000003052 2026-09-01 19:30:00+02:00     2.809  2.855 20.129                      True
1000001393 2026-09-01 09:00:00+02:00     1.935  3.518 20.078                      True
1000000643 2026-09-04 18:30:00+02:00     3.659  3.746 19.889                      True
1000003052 2026-09-01 17:15:00+02:00     2.110  2.830 19.873                      True

======================================================================
Sensitivity (discussion only; primary threshold was fixed in advance)   [2026-10-14 11:16:41 CAT]
======================================================================
      K  detected  missed  false_flags  false_pct_of_outside  meters_with_false_flag  pct_meters_affected_by_false_flags
 4.0000        40       0            0                 0.0000                       0                              0.0000
 5.0000        40       0            0                 0.0000                       0                              0.0000
 6.0000        40       0            0                 0.0000                       0                              0.0000
 8.0000        40       0            0                 0.0000                       0                              0.0000
10.0000        20      20            0                 0.0000                       0                              0.0000

Done 2026-10-14 11:16:41 CAT
PS C:\Users\ebyiringiro\smart-grid>
```

Screenshot 5.4-2 (Screenshot 2026-10-14 111957.png)

5.5 Management recommendation

Operations. Weekday demand peaks around 13:00; the highest 15-minute interval was 7,993.69 kW on Monday 28 September, and weekly energy grew about 2% a week (934,199 to 992,560 kWh over four weeks). Plan capacity against the weekday midday peak and review headroom monthly. Weekend demand is about 21% lower, so schedule planned maintenance then. Industrial customers are 10.2% of meters but 57.6% of energy (5,470 kWh per meter against 313 for residential), so monitoring and demand-response effort should start there. Regional differences mostly reflect customer mix, so compare regions within a customer type.

Data quality. About 1.0% of planned readings never arrived and about 2.0% arrived late; network completeness was 98.99%. Report missing readings as gaps, never as zero. Figures issued at midnight understate the day slightly (21 September: 152.178 kWh, 0.10%), so label them provisional. The anomaly rule found all 40 injected events with no false flags, but this is simulated data, so run it alert-only on real data first.

Storage and reporting. Use one-day chunks with compression ordered by event_time and segmented by meter_id. It cut storage from 1,356.7 MB to 501.6 MB (2.70 times) and the single-meter query Q4 from 131.6 ms to 8.0 ms against the unsegmented compressed table. The cost is 58.3 s of compression time against 32.3 s, and network-wide Q1 ran 5.6 times slower than on the uncompressed table. For dashboards, keep the hourly and 15-minute regional aggregates: dashboard refresh fell from 1,065 ms to 108 ms (9.8 times) for 18.0 MB of storage. The daily per-meter aggregate used 70% of that storage and saved about 2 ms, so drop it unless per-meter history becomes a heavy workload.

Forecasting. Adopt the Ridge model for day-ahead hourly regional planning, keeping the previous-day baseline as a fallback. On the test week (1,680 hourly predictions) its MAE was 4.55 kWh against 42.71 kWh for the baseline (89% lower; RMSE 5.79 against 79.40 kWh). Most of the gain comes from Saturday and Monday, where the baseline copies a weekend day onto a weekday. Validate on more weeks, including holidays, before relying on it.

(346 words)

6 Create and investigate continuous aggregates

6.1 Create the three aggregate views

View	Bucket and key	Stored columns
cagg_15min_region	15-minute bucket, region (inner join to meters)	sum_energy_kwh, sum_power_kw, reading_count
cagg_hourly_region	CAT hour (time_bucket with Africa/Kigali), region (inner join to meters)	same
cagg_daily_meter	CAT day (time_bucket with Africa/Kigali), meter_id	same

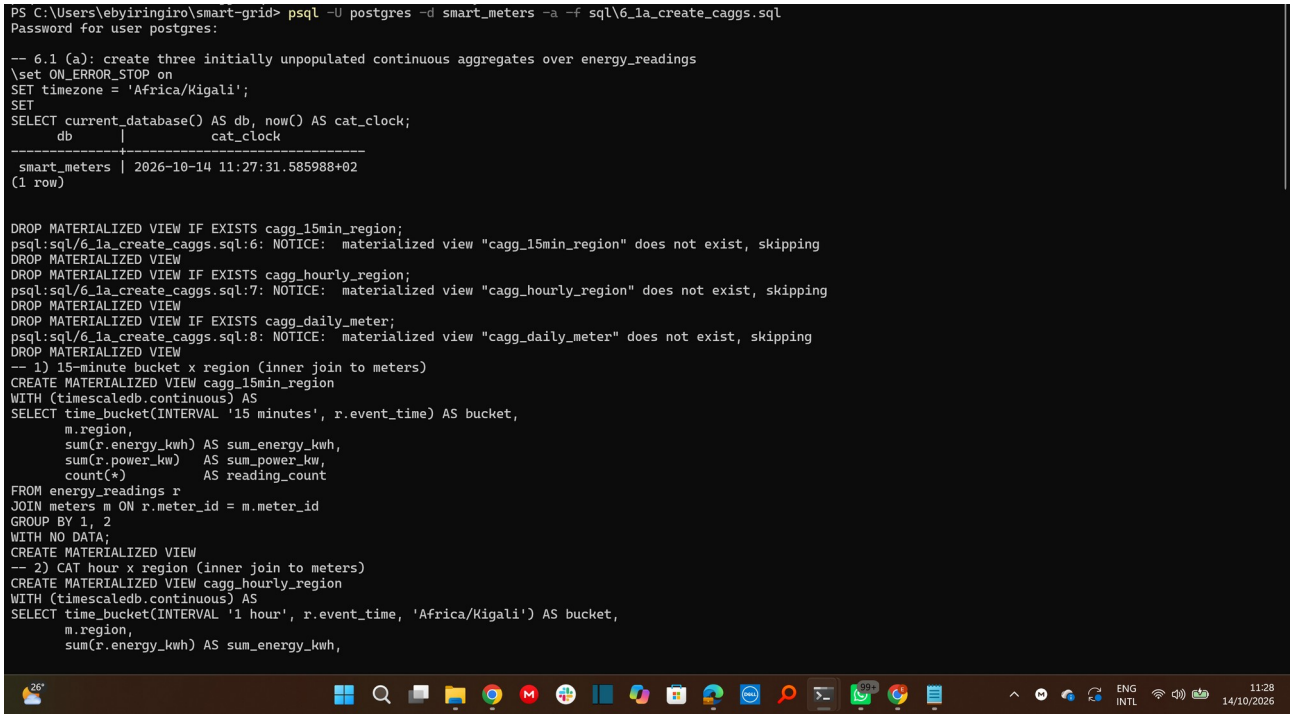
The views were created WITH NO DATA over energy_readings, and timescaledb.materialized_only is true on all three (real-time aggregation disabled), confirmed in timescaledb_information.continuous_aggregates.

View	Initial refresh (s)	Rows	Data size	Index size	Total size (bytes)
cagg_15min_region	28.708	26,880	2,152 kB	2,088 kB	4,382,720
cagg_hourly_region	46.787	6,720	640 kB	608 kB	1,318,912
cagg_daily_meter	32.326	112,000	7,584 kB	5,272 kB	13,164,544
All three	107.821				18,866,176 (about 18.0 MB)

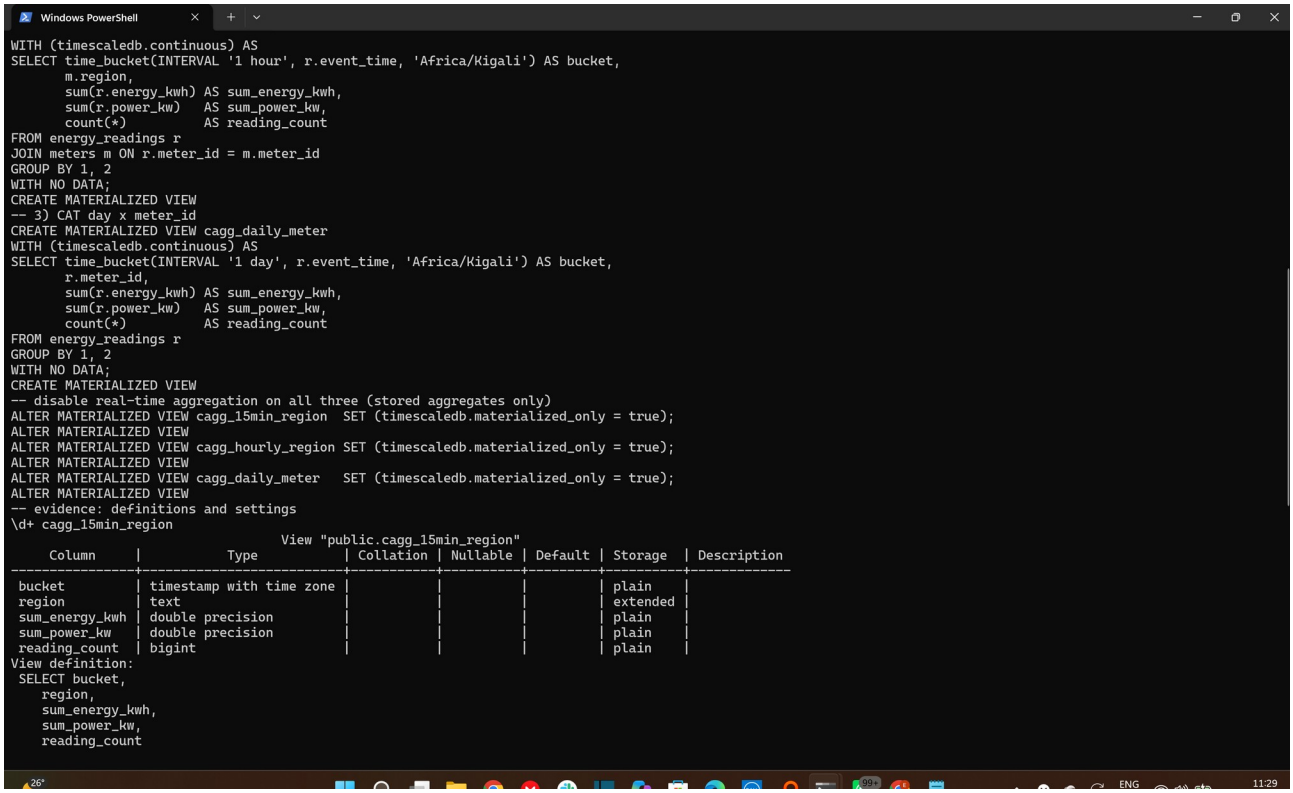
Each refresh covered 2026-09-01 00:00 to 2026-09-29 00:00 CAT and ran before any policy was added (11:30:14 to 11:32:02 CAT on 14 Oct). Correctness: reading_count totals 10,643,735 in every view and the summed energy equals energy_readings (3,853,704.3468 kWh) to better than 1e-8.

Refresh policies (jobs 1002, 1003 and 1004): start offset 3 days, end offset 1 hour, schedule 15 minutes. Because the offsets are relative to the database clock, the

policies only touch the last three days before the current time and never populate the older September data; each job ran successfully once with 0 failures.



Screenshot 6.1-1 (Screenshot 2026-10-14 112809.png)



Screenshot 6.1-2 (Screenshot 2026-10-14 112912.png)

```
    reading_count
   FROM _timescaledb_internal._materialized_hypertable_6;


\d+ cagg_hourly_region
                                View "public.cagg_hourly_region"
     Column     |           Type           | Collation | Nullable | Default | Storage  | Description
----------------+--------------------------+-----------+----------+---------+----------+-------------
 bucket         | timestamp with time zone |           |          |         | plain    |
 region         | text                     |           |          |         | extended |
 sum_energy_kwh | double precision         |           |          |         | plain    |
 sum_power_kw   | double precision         |           |          |         | plain    |
 reading_count  | bigint                   |           |          |         | plain    |
View definition:
 SELECT bucket,
    region,
    sum_energy_kwh,
    sum_power_kw,
    reading_count
   FROM _timescaledb_internal._materialized_hypertable_7;


\d+ cagg_daily_meter
                                View "public.cagg_daily_meter"
     Column     |           Type           | Collation | Nullable | Default | Storage | Description
----------------+--------------------------+-----------+----------+---------+---------+-------------
 bucket         | timestamp with time zone |           |          |         | plain   |
 meter_id       | bigint                   |           |          |         | plain   |
 sum_energy_kwh | double precision         |           |          |         | plain   |
 sum_power_kw   | double precision         |           |          |         | plain   |
 reading_count  | bigint                   |           |          |         | plain   |
View definition:
 SELECT bucket,
    meter_id,
    sum_energy_kwh,
    sum_power_kw,
    reading_count
   FROM _timescaledb_internal._materialized_hypertable_8;


SELECT view_name, materialized_only, finalized
FROM timescaledb_information.continuous_aggregates
WHERE view_name IN ('cagg_15min_region','cagg_hourly_region','cagg_daily_meter')
ORDER BY view_name;
psql:sql/6_1a_create_caggs.sql:61: ERROR:  column "finalized" does not exist
LINE 1: SELECT view_name, materialized_only, finalized
                                             ^
PS C:\Users\ebyiringiro\smart-grid>
```

Screenshot 6.1-3 (Screenshot 2026-10-14 112931.png)

```
PS C:\Users\ebyiringiro\smart-grid> psql -U postgres -d smart_meters -a -f sql\6_1b_refresh_and_sizes.sql
Password for user postgres:

-- 6.1 (b): manual initial refresh (timed), storage, sample rows, correctness checks
-- Run BEFORE any refresh policy is added. Pause other workloads while this runs.
\set ON_ERROR_STOP on
SET timezone = 'Africa/Kigali';
SET
SELECT now() AS cat_clock_start;
        cat_clock_start
-------------------------------
 2026-10-14 11:30:14.275971+02
(1 row)


\timing on
Timing is on.
\echo '--- refresh cagg_15min_region (whole generated period) ---'
--- refresh cagg_15min_region (whole generated period) ---
CALL refresh_continuous_aggregate('cagg_15min_region',  '2026-09-01 00:00:00+02', '2026-09-29 00:00:00+02');
CALL
Time: 28708.454 ms (00:28.708)
\echo '--- refresh cagg_hourly_region ---'
--- refresh cagg_hourly_region ---
CALL refresh_continuous_aggregate('cagg_hourly_region', '2026-09-01 00:00:00+02', '2026-09-29 00:00:00+02');
CALL
Time: 46787.035 ms (00:46.787)
\echo '--- refresh cagg_daily_meter ---'
--- refresh cagg_daily_meter ---
CALL refresh_continuous_aggregate('cagg_daily_meter',   '2026-09-01 00:00:00+02', '2026-09-29 00:00:00+02');
CALL
Time: 32325.672 ms (00:32.326)
\timing off
Timing is off.
SELECT now() AS cat_clock_end;
         cat_clock_end
-------------------------------
 2026-10-14 11:32:02.157278+02
(1 row)


\echo 'Refresh duration = the "Time:" value above for each CALL (ms; divide by 1000 for seconds)'
Refresh duration = the "Time:" value above for each CALL (ms; divide by 1000 for seconds)
\echo '--- materialized data + index size ---'
```

Screenshot 6.1-4 (Screenshot 2026-10-14 114802.png)

```
--- materialized data + index size ---
SELECT c.view_name,
       pg_size_pretty(d.table_bytes) AS data_size,
       pg_size_pretty(d.index_bytes) AS index_size,
       pg_size_pretty(d.total_bytes) AS total_size,
       d.table_bytes, d.index_bytes, d.total_bytes
FROM timescaledb_information.continuous_aggregates c
CROSS JOIN LATERAL hypertable_detailed_size(
     format('%I.%I', c.materialization_hypertable_schema,
                     c.materialization_hypertable_name)::regclass) d
WHERE c.view_name IN ('cagg_15min_region','cagg_hourly_region','cagg_daily_meter')
ORDER BY c.view_name;
     view_name      | data_size | index_size | total_size | table_bytes | index_bytes | total_bytes
--------------------+-----------+------------+------------+-------------+-------------+-------------
 cagg_15min_region  | 2152 kB   | 2088 kB    | 4280 kB    |     2203648 |     2138112 |     4382720
 cagg_daily_meter   | 7584 kB   | 5272 kB    | 13 MB      |     7766016 |     5398528 |    13164544
 cagg_hourly_region | 640 kB    | 608 kB     | 1288 kB    |      655360 |      622592 |     1318912
(3 rows)


\echo '--- row counts (expect ~26,880 / ~6,720 / ~112,000) ---'
--- row counts (expect ~26,880 / ~6,720 / ~112,000) ---
SELECT 'cagg_15min_region' AS view, count(*) AS rows FROM cagg_15min_region
UNION ALL SELECT 'cagg_hourly_region', count(*) FROM cagg_hourly_region
UNION ALL SELECT 'cagg_daily_meter',   count(*) FROM cagg_daily_meter;
       view        |  rows
-------------------+--------
 cagg_15min_region  |  26880
 cagg_hourly_region |   6720
 cagg_daily_meter   | 112000
(3 rows)


\echo '--- sample populated rows ---'
--- sample populated rows ---
SELECT * FROM cagg_15min_region  ORDER BY bucket, region LIMIT 5;
        bucket          |  region   |   sum_energy_kwh    |    sum_power_kw     | reading_count
------------------------+-----------+---------------------+---------------------+---------------
 2026-09-01 00:00:00+02 | Region 1  |       103.01984575 | 412.07937799999996 |           398
 2026-09-01 00:00:00+02 | Region 10 |  91.36313593000001 | 365.45253400000024 |           397
 2026-09-01 00:00:00+02 | Region 2  | 105.53019479999993 |  422.1207729999998 |           399
 2026-09-01 00:00:00+02 | Region 3  |       113.36917513 |  453.4766989999998 |           399
 2026-09-01 00:00:00+02 | Region 4  |  83.51429352000008 |  334.0571770000001 |           396
(5 rows)
```

Screenshot 6.1-5 (Screenshot 2026-10-14 114823.png)

```
SELECT * FROM cagg_hourly_region ORDER BY bucket, region LIMIT 5;
        bucket          |  region   |   sum_energy_kwh    |    sum_power_kw     | reading_count
------------------------+-----------+---------------------+---------------------+---------------
 2026-09-01 00:00:00+02 | Region 1  | 412.56091079000004 | 1650.2436250000005 |          1585
 2026-09-01 00:00:00+02 | Region 10 |  365.0069615899998 | 1460.0278500000004 |          1584
 2026-09-01 00:00:00+02 | Region 2  | 420.50421976000075 | 1682.0168739999988 |          1589
 2026-09-01 00:00:00+02 | Region 3  |  446.04018479999997 | 1784.1607399999982 |          1589
 2026-09-01 00:00:00+02 | Region 4  | 339.42778507999986 |  1357.711140000003 |          1590
(5 rows)


SELECT * FROM cagg_daily_meter   ORDER BY bucket, meter_id LIMIT 5;
        bucket          |  meter_id  |   sum_energy_kwh    |    sum_power_kw     | reading_count
------------------------+------------+---------------------+---------------------+---------------
 2026-09-01 00:00:00+02 | 1000000000 |         8.82348068 |  35.29392099999999 |            96
 2026-09-01 00:00:00+02 | 1000000001 | 13.382748549999997 |          53.530993 |            95
 2026-09-01 00:00:00+02 | 1000000002 | 14.722062180000004 |  58.88824599999999 |            93
 2026-09-01 00:00:00+02 | 1000000003 |         5.84101962 | 23.364076000000004 |            95
 2026-09-01 00:00:00+02 | 1000000004 |  17.84006471000001 |  71.36026099999998 |            95
(5 rows)


\echo '--- correctness: reading_count and energy must match energy_readings ---'
--- correctness: reading_count and energy must match energy_readings ---
SELECT 'energy_readings' AS source, count(*) AS readings, sum(energy_kwh) AS energy_kwh
FROM energy_readings
UNION ALL SELECT 'cagg_15min_region',  sum(reading_count), sum(sum_energy_kwh) FROM cagg_15min_region
UNION ALL SELECT 'cagg_hourly_region', sum(reading_count), sum(sum_energy_kwh) FROM cagg_hourly_region
UNION ALL SELECT 'cagg_daily_meter',   sum(reading_count), sum(sum_energy_kwh) FROM cagg_daily_meter;
       source       | readings |     energy_kwh
--------------------+----------+--------------------
 energy_readings    | 10643735 |  3853704.346815974
 cagg_15min_region  | 10643735 | 3853704.3468159763
 cagg_hourly_region | 10643735 | 3853704.3468159707
 cagg_daily_meter   | 10643735 |  3853704.346815964
(4 rows)
```

Screenshot 6.1-6 (Screenshot 2026-10-14 114841.png)

```
PS C:\Users\ebyiringiro\smart-grid> psql -U postgres -d smart_meters -a -f sql\6_1c_policies.sql
Password for user postgres:

-- 6.1 (c): add refresh policies (3-day lookback, 1-hour end offset, 15-minute schedule)
-- Run only AFTER 6_1b measurements are complete.
\set ON_ERROR_STOP on
SET timezone = 'Africa/Kigali';
SET
SELECT now() AS cat_clock;
          cat_clock
-------------------------------
 2026-10-14 11:33:30.944205+02
(1 row)


SELECT add_continuous_aggregate_policy('cagg_15min_region',
       start_offset => INTERVAL '3 days', end_offset => INTERVAL '1 hour',
       schedule_interval => INTERVAL '15 minutes');
 add_continuous_aggregate_policy
---------------------------------
                            1002
(1 row)


SELECT add_continuous_aggregate_policy('cagg_hourly_region',
       start_offset => INTERVAL '3 days', end_offset => INTERVAL '1 hour',
       schedule_interval => INTERVAL '15 minutes');
 add_continuous_aggregate_policy
---------------------------------
                            1003
(1 row)


SELECT add_continuous_aggregate_policy('cagg_daily_meter',
       start_offset => INTERVAL '3 days', end_offset => INTERVAL '1 hour',
       schedule_interval => INTERVAL '15 minutes');
 add_continuous_aggregate_policy
---------------------------------
                            1004
(1 row)


\echo '--- policy settings ---'
--- policy settings ---
SELECT j.job_id, c.view_name, j.proc_name, j.schedule_interval, j.config, j.scheduled, j.next_start
```

Screenshot 6.1-7 (Screenshot 2026-10-14 114853.png)

```
SELECT s.job_id, c.view_name, s.last_run_status, s.last_run_started_at,
       s.total_runs, s.total_successes, s.total_failures
FROM timescaledb_information.job_stats s
JOIN timescaledb_information.continuous_aggregates c
  ON s.hypertable_name = c.materialization_hypertable_name
WHERE c.view_name IN ('cagg_15min_region','cagg_hourly_region','cagg_daily_meter')
ORDER BY c.view_name;
 job_id |     view_name      | last_run_status | last_run_started_at | total_runs | total_successes | total_failures
--------+--------------------+-----------------+---------------------+------------+-----------------+----------------
   1002 | cagg_15min_region  |                 |                     |            |                 |
   1004 | cagg_daily_meter   |                 |                     |            |                 |
   1003 | cagg_hourly_region |                 |                     |            |                 |
(3 rows)


\echo '--- policy window now: policies refresh [now-3 days, now-1 hour), so September data is NOT touched ---'
--- policy window now: policies refresh [now-3 days, now-1 hour), so September data is NOT touched ---
SELECT now() - INTERVAL '3 days' AS policy_window_start, now() - INTERVAL '1 hour' AS policy_window_end;
      policy_window_start      |       policy_window_end
-------------------------------+-------------------------------
 2026-10-11 11:33:31.266678+02 | 2026-10-14 10:33:31.266678+02
(1 row)
```

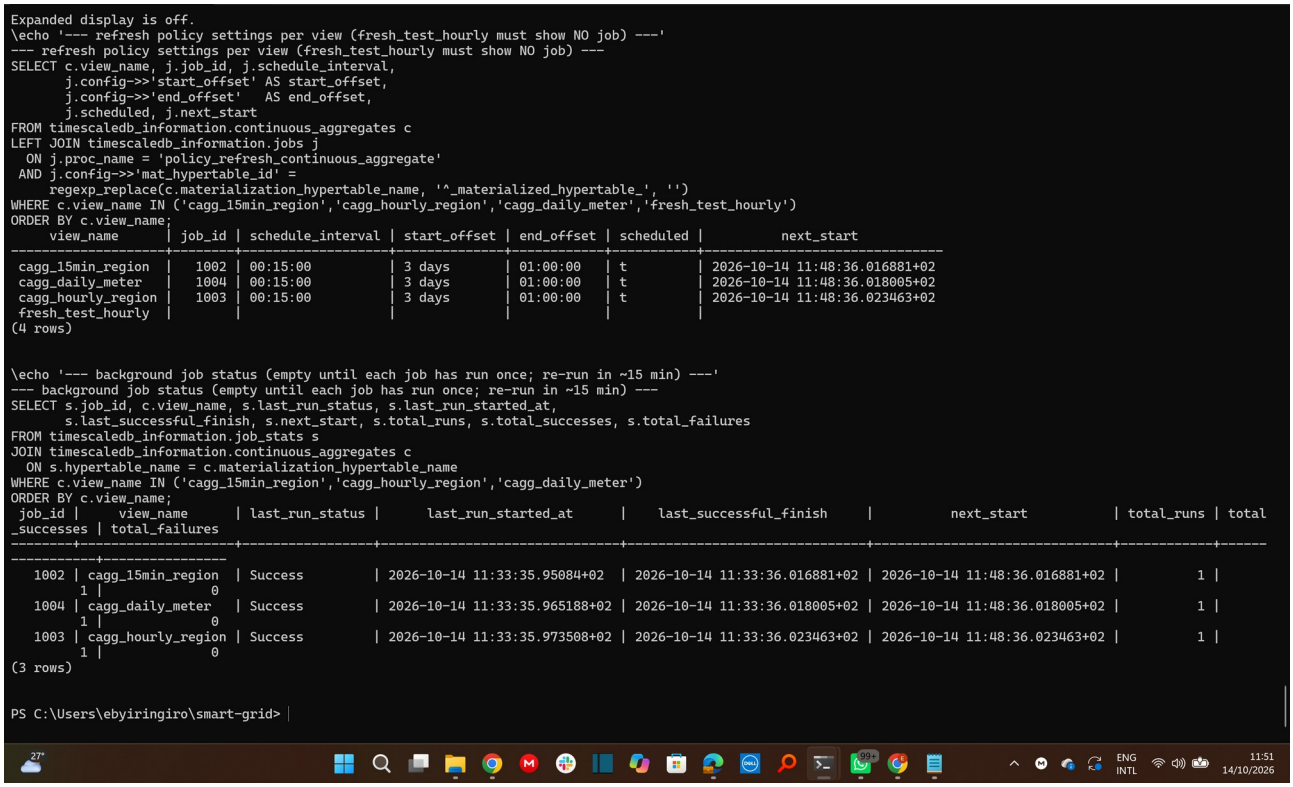
Screenshot 6.1-8 (Screenshot 2026-10-14 114913.png)

```
PS C:\Users\ebyiringiro\smart-grid> psql -U postgres -d smart_meters -a -f sql\6_1_checks.sql
Password for user postgres:

psql: error: connection to server at "localhost" (::1), port 5432 failed: FATAL:  password authentication failed for user "postgres"
PS C:\Users\ebyiringiro\smart-grid> psql -U postgres -d smart_meters -a -f sql\6_1_checks.sql
Password for user postgres:

-- 6.1 / 6.2 evidence checks (read-only). Safe to run repeatedly; does NOT drop anything.
\set ON_ERROR_STOP on
SET timezone = 'Africa/Kigali';
SET
SELECT current_database() AS db, now() AS cat_clock;
      db      |           cat_clock
--------------+-------------------------------
 smart_meters | 2026-10-14 11:42:05.381275+02
(1 row)


\echo '--- real-time aggregation disabled + original view definitions ---'
--- real-time aggregation disabled + original view definitions ---
SELECT view_name, materialized_only
FROM timescaledb_information.continuous_aggregates
WHERE view_name IN ('cagg_15min_region','cagg_hourly_region','cagg_daily_meter','fresh_test_hourly')
ORDER BY view_name;
     view_name      | materialized_only
--------------------+-------------------
 cagg_15min_region  | t
 cagg_daily_meter   | t
 cagg_hourly_region | t
 fresh_test_hourly  | f
(4 rows)


\x on
Expanded display is on.
SELECT view_name, view_definition
FROM timescaledb_information.continuous_aggregates
WHERE view_name IN ('cagg_15min_region','cagg_hourly_region','cagg_daily_meter')
ORDER BY view_name;
-[ RECORD 1 ]---+-----------------------------------------------------------------------------------------------
view_name       | cagg_15min_region
view_definition |  SELECT time_bucket('00:15:00'::interval, r.event_time) AS bucket,                           +
                |     m.region,                                                                               +
                |     sum(r.energy_kwh) AS sum_energy_kwh,                                                    +
                |     sum(r.power_kw) AS sum_power_kw,                                                        +
```

Screenshot 6.1-9 (Screenshot 2026-10-14 115041.png)

```
Windows PowerShell
SELECT view_name, view_definition
FROM timescaledb_information.continuous_aggregates
WHERE view_name IN ('cagg_15min_region','cagg_hourly_region','cagg_daily_meter')
ORDER BY view_name;
-[ RECORD 1 ]---+-----------------------------------------------------------------------------------------------
view_name       | cagg_15min_region
view_definition |  SELECT time_bucket('00:15:00'::interval, r.event_time) AS bucket,                           +
                |     m.region,                                                                               +
                |     sum(r.energy_kwh) AS sum_energy_kwh,                                                    +
                |     sum(r.power_kw) AS sum_power_kw,                                                        +
                |     count(*) AS reading_count                                                               +
                |    FROM (energy_readings r                                                                  +
                |      JOIN meters m ON ((r.meter_id = m.meter_id)))                                           +
                |   GROUP BY (time_bucket('00:15:00'::interval, r.event_time)), m.region;
-[ RECORD 2 ]---+-----------------------------------------------------------------------------------------------
view_name       | cagg_daily_meter
view_definition |  SELECT time_bucket('1 day'::interval, event_time, 'Africa/Kigali'::text) AS bucket,         +
                |     meter_id,                                                                               +
                |     sum(energy_kwh) AS sum_energy_kwh,                                                      +
                |     sum(power_kw) AS sum_power_kw,                                                          +
                |     count(*) AS reading_count                                                               +
                |    FROM energy_readings r                                                                   +
                |   GROUP BY (time_bucket('1 day'::interval, event_time, 'Africa/Kigali'::text)), meter_id;
-[ RECORD 3 ]---+-----------------------------------------------------------------------------------------------
view_name       | cagg_hourly_region
view_definition |  SELECT time_bucket('01:00:00'::interval, r.event_time, 'Africa/Kigali'::text) AS bucket,    +
                |     m.region,                                                                               +
                |     sum(r.energy_kwh) AS sum_energy_kwh,                                                    +
                |     sum(r.power_kw) AS sum_power_kw,                                                        +
                |     count(*) AS reading_count                                                               +
                |    FROM (energy_readings r                                                                  +
                |      JOIN meters m ON ((r.meter_id = m.meter_id)))                                           +
                |   GROUP BY (time_bucket('01:00:00'::interval, r.event_time, 'Africa/Kigali'::text)), m.region;


\x off
Expanded display is off.
\echo '--- refresh policy settings per view (fresh_test_hourly must show NO job) ---'
--- refresh policy settings per view (fresh_test_hourly must show NO job) ---
SELECT c.view_name, j.job_id, j.schedule_interval,
       j.config->>'start_offset' AS start_offset,
       j.config->>'end_offset'   AS end_offset,
       j.scheduled, j.next_start
FROM timescaledb_information.continuous_aggregates c
LEFT JOIN timescaledb_information.jobs j
  ON j.proc_name = 'policy_refresh_continuous_aggregate'
```

Screenshot 6.1-10 (Screenshot 2026-10-14 115103.png)



Screenshot 6.1-11 (Screenshot 2026-10-14 115115.png)

6.2 Test freshness and delayed arrivals

Test on 14 Oct 2026 at 11:34:27 CAT: hour H (start of the most recent completed CAT hour) = 2026-10-14 10:00; older hour (H - 48 h) = 2026-10-12 10:00. Event times: 1 kWh at 2026-10-12 10:10, 0.5 kWh at 2026-10-12 10:25, 2 kWh at 2026-10-14 10:10. The test hypertable fresh_test and its hourly aggregate fresh_test_hourly have no refresh policy.

Stage	Older hour aggregate	Hour H aggregate
Initial refresh of the older hour only, real-time off	1 kWh, 1 row	no bucket
After both new inserts, before refresh, real-time off	1 kWh, 1 row	no bucket
Real-time on, still before refresh	1 kWh, 1 row	2 kWh, 1 row
After manual refresh of both hours	1.5 kWh, 2 rows	2 kWh, 1 row

The raw table showed 1.5 kWh in 2 rows (older hour) and 2 kWh in 1 row (hour H) after the inserts.

What became visible when. Materialized-only views show only what the last refresh stored: the late 0.5 kWh reading and the new hour were invisible until a refresh. Real-time aggregation added raw rows after the watermark, so hour H appeared (2 kWh) but the late reading inside the already-materialized older hour did not (still 1 kWh), because real-time results cover only data newer than the last materialized bucket. Only the manual refresh brought both hours to the raw totals. An absent bucket is not a zero reading.

Policy window. At the recorded test time the 6.1 policies would refresh [11 Oct 11:34, 14 Oct 10:34): this includes the older hour (12 Oct 10:00) but not hour H (14 Oct 10:00-11:00), whose bucket ends after the window end; hour H would be picked up by a policy run at or after 12:00 CAT.

```
PS C:\Users\ebyiringiro\smart-grid> psql -U postgres -d smart_meters -a -f sql\6_2_freshness.sql
Password for user postgres:

-- 6.2: freshness and delayed-arrival test on a separate hypertable
-- Run once, with no other workload. Times are relative to the actual test time.
\set ON_ERROR_STOP on
SET timezone = 'Africa/Kigali';
SET
\echo '=== Setup: table, hypertable, hourly continuous aggregate (no data, real-time OFF, no policy) ==='
=== Setup: table, hypertable, hourly continuous aggregate (no data, real-time OFF, no policy) ===
DROP MATERIALIZED VIEW IF EXISTS fresh_test_hourly;
psql:sql/6_2_freshness.sql:7: NOTICE:  materialized view "fresh_test_hourly" does not exist, skipping
DROP MATERIALIZED VIEW
DROP TABLE IF EXISTS fresh_test;
psql:sql/6_2_freshness.sql:8: NOTICE:  table "fresh_test" does not exist, skipping
DROP TABLE
CREATE TABLE fresh_test (
    event_time TIMESTAMPTZ      NOT NULL,
    energy_kwh DOUBLE PRECISION NOT NULL
);
CREATE TABLE
SELECT create_hypertable('fresh_test', by_range('event_time', INTERVAL '1 day'));
 create_hypertable
-------------------
 (9,t)
(1 row)


CREATE MATERIALIZED VIEW fresh_test_hourly
WITH (timescaledb.continuous, timescaledb.materialized_only = true) AS
SELECT time_bucket(INTERVAL '1 hour', event_time, 'Africa/Kigali') AS bucket,
       sum(energy_kwh) AS sum_energy_kwh,
       count(*)        AS reading_count
FROM fresh_test
GROUP BY 1
WITH NO DATA;
CREATE MATERIALIZED VIEW
\d+ fresh_test
                                          Table "public.fresh_test"
   Column   |           Type           | Collation | Nullable | Default | Storage | Compression | Stats target | Description
------------+--------------------------+-----------+----------+---------+---------+-------------+--------------+-------------
 event_time | timestamp with time zone |           | not null |         | plain   |             |              |
```

Screenshot 6.2-1 (Screenshot 2026-10-14 114925.png)

```
                                          Table "public.fresh_test"
   Column   |           Type           | Collation | Nullable | Default | Storage | Compression | Stats target | Description
------------+--------------------------+-----------+----------+---------+---------+-------------+--------------+-------------
 event_time | timestamp with time zone |           | not null |         | plain   |             |              |
 energy_kwh | double precision         |           | not null |         | plain   |             |              |
Indexes:
    "fresh_test_event_time_idx" btree (event_time DESC)
Not-null constraints:
    "fresh_test_event_time_not_null" NOT NULL "event_time"
    "fresh_test_energy_kwh_not_null" NOT NULL "energy_kwh"
Access method: heap


\d+ fresh_test_hourly
                                View "public.fresh_test_hourly"
     Column     |           Type           | Collation | Nullable | Default | Storage | Description
----------------+--------------------------+-----------+----------+---------+---------+-------------
 bucket         | timestamp with time zone |           |          |         | plain   |
 sum_energy_kwh | double precision         |           |          |         | plain   |
 reading_count  | bigint                   |           |          |         | plain   |
View definition:
 SELECT bucket,
    sum_energy_kwh,
    reading_count
   FROM _timescaledb_internal._materialized_hypertable_10;


SELECT view_name, materialized_only FROM timescaledb_information.continuous_aggregates
WHERE view_name = 'fresh_test_hourly';
     view_name     | materialized_only
-------------------+-------------------
 fresh_test_hourly | t
(1 row)


SELECT count(*) AS refresh_policies_on_test_view
FROM timescaledb_information.jobs j
JOIN timescaledb_information.continuous_aggregates c
  ON j.hypertable_name = c.materialization_hypertable_name
WHERE c.view_name = 'fresh_test_hourly';
 refresh_policies_on_test_view
-------------------------------
                             0
```

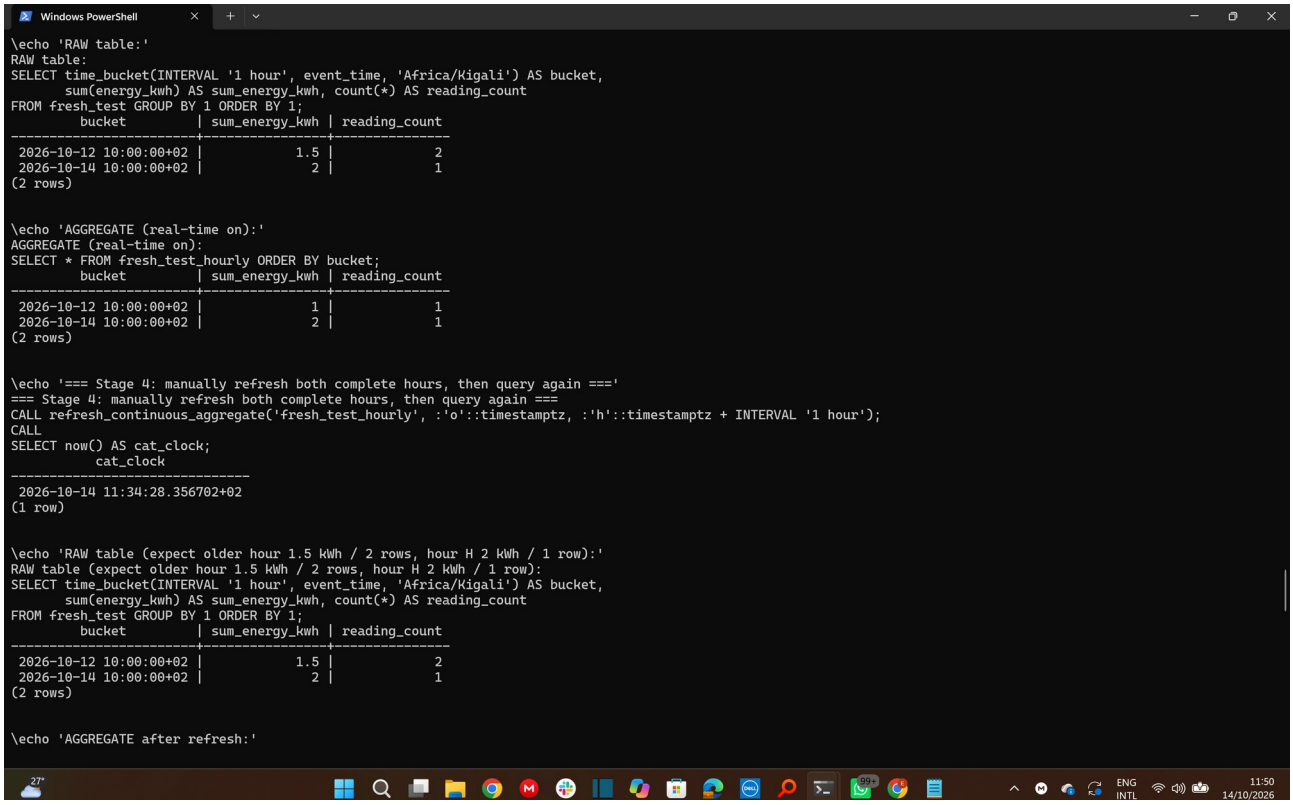
Screenshot 6.2-2 (Screenshot 2026-10-14 114938.png)

```
\echo '=== Record test time, H (start of most recent completed CAT hour) and older hour (H - 48 h) ==='
=== Record test time, H (start of most recent completed CAT hour) and older hour (H - 48 h) ===
SELECT now() AS test_time,
       date_trunc('hour', now()) - INTERVAL '1 hour' AS h,
       date_trunc('hour', now()) - INTERVAL '1 hour' - INTERVAL '48 hours' AS o \gset
SELECT :'o'::timestamptz + INTERVAL '10 minutes' AS e1,
       :'o'::timestamptz + INTERVAL '25 minutes' AS e2,
       :'h'::timestamptz + INTERVAL '10 minutes' AS e3 \gset
SELECT :'test_time'::timestamptz AS actual_test_time_cat,
       :'h'::timestamptz AS hour_h_start,
       :'o'::timestamptz AS older_hour_start,
       :'e1'::timestamptz AS insert1_1kwh_event_time,
       :'e2'::timestamptz AS insert2_0_5kwh_event_time,
       :'e3'::timestamptz AS insert3_2kwh_event_time;
     actual_test_time_cat      |      hour_h_start      |    older_hour_start    | insert1_1kwh_event_time | insert2_0_5kwh_event_time | insert3_2kwh_event_time
-------------------------------+------------------------+------------------------+-------------------------+---------------------------+-------------------------
 2026-10-14 11:34:27.861953+02 | 2026-10-14 10:00:00+02 | 2026-10-12 10:00:00+02 | 2026-10-12 10:10:00+02  | 2026-10-12 10:25:00+02    | 2026-10-14 10:10:00+02
(1 row)


\echo '=== Stage 1: insert 1 kWh into older hour, refresh ONLY that hour ==='
=== Stage 1: insert 1 kWh into older hour, refresh ONLY that hour ===
INSERT INTO fresh_test VALUES (:'e1', 1.0);
INSERT 0 1
CALL refresh_continuous_aggregate('fresh_test_hourly', :'o'::timestamptz, :'o'::timestamptz + INTERVAL '1 hour');
CALL
SELECT now() AS cat_clock;
          cat_clock
-------------------------------
 2026-10-14 11:34:27.947122+02
(1 row)


SELECT * FROM fresh_test_hourly ORDER BY bucket;
         bucket          | sum_energy_kwh | reading_count
-------------------------+----------------+---------------
 2026-10-12 10:00:00+02 |              1 |             1
(1 row)


\echo '=== Stage 2: insert 0.5 kWh (older hour) and 2 kWh (hour H); BEFORE refresh, real-time OFF ==='
=== Stage 2: insert 0.5 kWh (older hour) and 2 kWh (hour H); BEFORE refresh, real-time OFF ===
INSERT INTO fresh_test VALUES (:'e2', 0.5);
INSERT 0 1
INSERT INTO fresh_test VALUES (:'e3', 2.0);
```

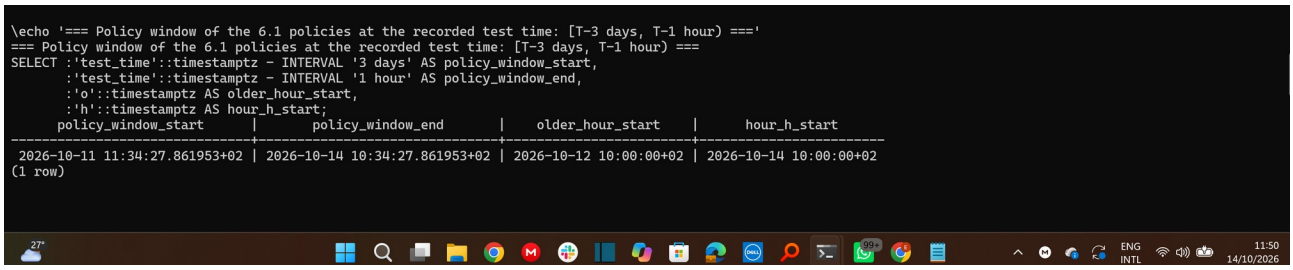
Screenshot 6.2-3 (Screenshot 2026-10-14 114950.png)

```
INSERT INTO fresh_test VALUES (:'e3', 2.0);
INSERT 0 1
SELECT now() AS cat_clock;
          cat_clock
-------------------------------
 2026-10-14 11:34:28.04164+02
(1 row)


\echo 'RAW table:'
RAW table:
SELECT time_bucket(INTERVAL '1 hour', event_time, 'Africa/Kigali') AS bucket,
       sum(energy_kwh) AS sum_energy_kwh, count(*) AS reading_count
FROM fresh_test GROUP BY 1 ORDER BY 1;
         bucket          | sum_energy_kwh | reading_count
-------------------------+----------------+---------------
 2026-10-12 10:00:00+02 |            1.5 |             2
 2026-10-14 10:00:00+02 |              2 |             1
(2 rows)


\echo 'AGGREGATE (materialized only):'
AGGREGATE (materialized only):
SELECT * FROM fresh_test_hourly ORDER BY bucket;
         bucket          | sum_energy_kwh | reading_count
-------------------------+----------------+---------------
 2026-10-12 10:00:00+02 |              1 |             1
(1 row)


\echo '=== Stage 3: enable real-time aggregation, still BEFORE refresh ==='
=== Stage 3: enable real-time aggregation, still BEFORE refresh ===
ALTER MATERIALIZED VIEW fresh_test_hourly SET (timescaledb.materialized_only = false);
ALTER MATERIALIZED VIEW
SELECT view_name, materialized_only FROM timescaledb_information.continuous_aggregates
WHERE view_name = 'fresh_test_hourly';
     view_name     | materialized_only
-------------------+-------------------
 fresh_test_hourly | f
(1 row)


\echo 'RAW table:'
RAW table:
SELECT time_bucket(INTERVAL '1 hour', event_time, 'Africa/Kigali') AS bucket,
       sum(energy_kwh) AS sum_energy_kwh, count(*) AS reading_count
```

Screenshot 6.2-4 (Screenshot 2026-10-14 115003.png)



Screenshot 6.2-5 (Screenshot 2026-10-14 115014.png)



Screenshot 6.2-6 (Screenshot 2026-10-14 115030.png)

7 Build and compare dashboards

7.1 Build the Raw dashboard

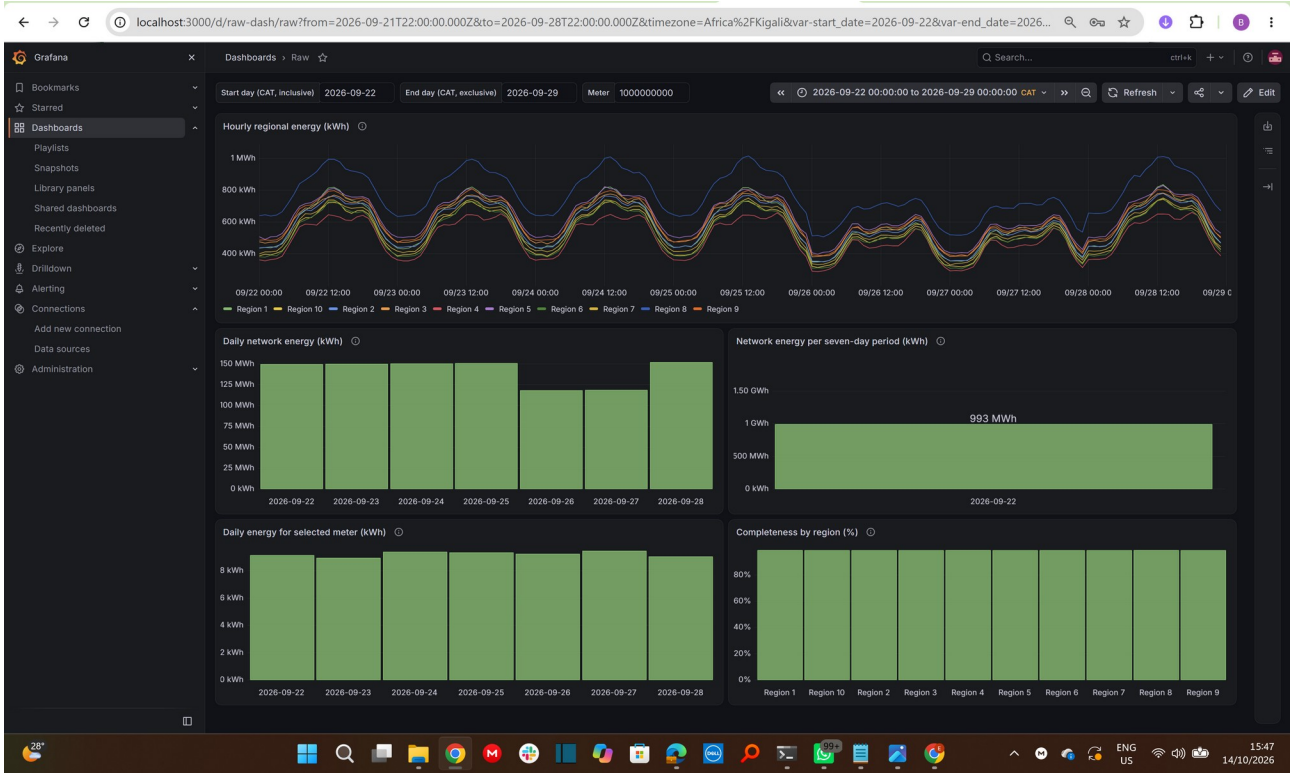
Tool: Grafana 13.2 with a PostgreSQL data source on smart_meters. The dashboard Raw reads energy_readings directly and joins meters where needed. Filters: Start day (CAT, inclusive), End day (CAT, exclusive) and Meter, converted to CAT midnights in SQL so only whole days are used; the time picker is set to the same range. Time zone: Africa/Kigali.

Panel	Content	Unit
1	Hourly regional energy	kWh
2	Daily network energy	kWh
3	Network energy per seven-day period (periods from 1, 8, 15 and 22 September, clipped to the range)	kWh
4	Daily energy for the selected meter	kWh
5	Completeness by region (observed unique / planned)	%

Panel queries are listed in Appendix A and kept in dashboards\Raw.json and section7_dashboards.py.

```
PS C:\Users\ebyiringiro\smart-grid> python section7_dashboards.py build
wrote dashboards/Raw.json
wrote dashboards/Aggregate.json
Import both in Grafana (Dashboards > New > Import). Set the smart_meters PostgreSQL data source as default first. Time picker: 22 Sep 00:00 - 29 Sep 0
0:00 CAT.
PS C:\Users\ebyiringiro\smart-grid> python section7_dashboards.py verify
7.3 verification started 2026-10-14 12:32:58 CAT
Range: 2026-09-22 00:00 CAT to 2026-09-29 00:00 CAT (exclusive); meter 1000000000

Panel                                      rows  keys=  counts=    max |diff|       tol  result
1. Hourly regional energy (kWh)            1680   True     True   0.000000000     0.001  PASS
2. Daily network energy (kWh)                 7   True     True   0.000000003     0.001  PASS
3. Network energy per seven-day period (kWh)      1   True     True   0.000000012     0.001  PASS
4. Daily energy for selected meter (kWh)      7   True     True   0.000000000     0.001  PASS
5. Completeness by region (%)                10   True     True   0.000000000     1e-06  PASS

Per-panel CSVs saved in results\ (7_3_panel<N>_raw.csv / _agg.csv). Done 2026-10-14 12:33:00 CAT
PS C:\Users\ebyiringiro\smart-grid>
```

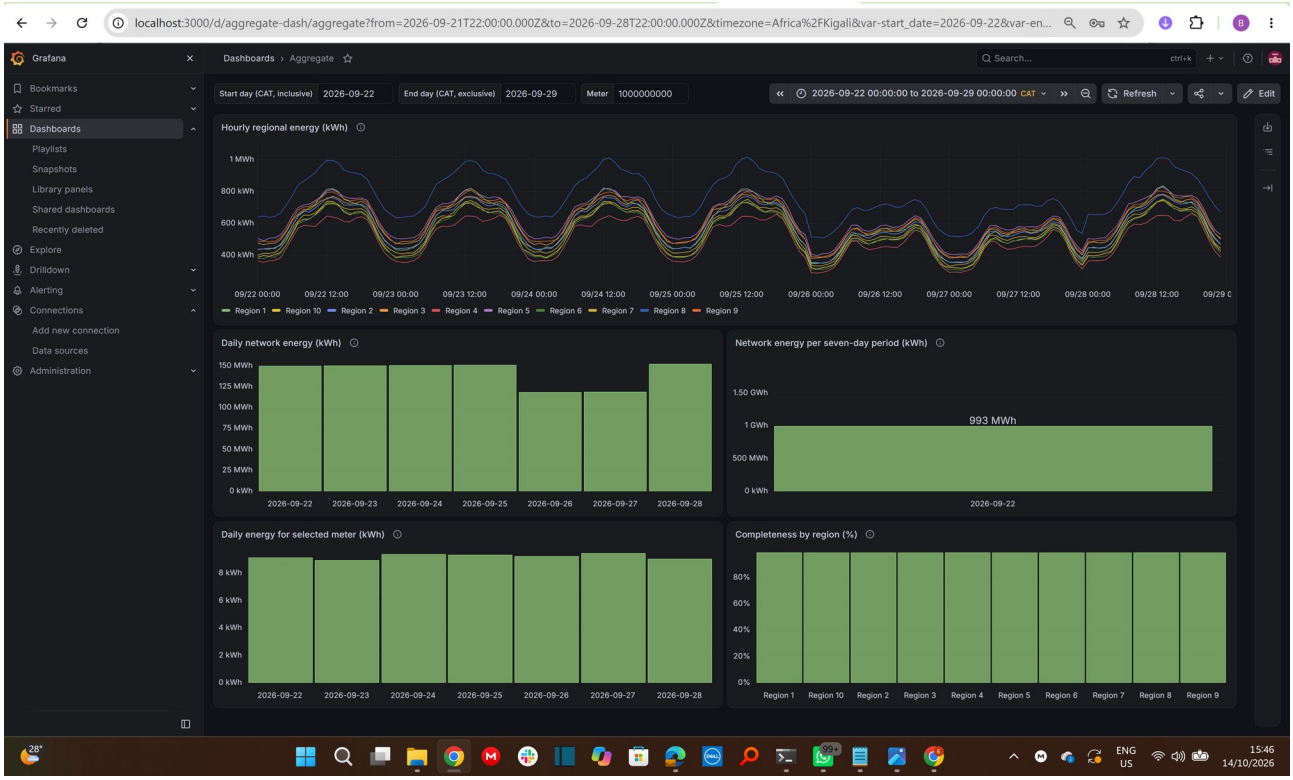
Screenshot 7.1-1 (Screenshot 2026-10-14 123548.png)



Screenshot 7.1-2 (Screenshot 2026-10-14 154719.png)

7.2 Build the Aggregate dashboard

The dashboard Aggregate has the same five panels, layout, filters and date range. Hourly regional energy comes from cagg_hourly_region, daily and seven-day network energy and completeness counts from cagg_15min_region, and meter history from cagg_daily_meter. Stored energy sums and reading counts are summed when combining buckets, and planned readings are calculated from registered meter counts in meters (used only for labels and counts).



Screenshot 7.2-1 (Screenshot 2026-10-14 154704.png)

7.3 Verify matching results

Both dashboards were set to 22 Sep 00:00 to 29 Sep 00:00 CAT (exclusive), all regions, meter 1000000000. The numbers behind every panel were queried (python section7_dashboards.py verify, 14 Oct 2026 12:32:58 CAT) and matched by bucket, region or meter.

Panel	Compared rows	Keys identical	Reading counts identical	Max abs difference	Tolerance	Result
1 Hourly regional energy	1,680	yes	yes	0.000000000 kWh	0.001	PASS
2 Daily network energy	7	yes	yes	0.000000003 kWh	0.001	PASS
3 Seven-day period	1	yes	yes	0.000000012 kWh	0.001	PASS
4 Selected meter daily	7	yes	yes	0.000000000 kWh	0.001	PASS
5 Completeness by region	10	yes	yes	0.000000000 pp	0.000001	PASS

The seven-day panel has one row because only the period starting 22 September lies inside the selected range. No unexplained differences remained. The per-panel data are in results\7_3_panel<N>_raw.csv and _agg.csv.

```
PS C:\Users\ebyiringiro\smart-grid> python section7_dashboards.py verify
7.3 verification started 2026-10-14 15:53:22 CAT
Range: 2026-09-22 00:00 CAT to 2026-09-29 00:00 CAT (exclusive); meter 1000000000

Panel                                    rows  keys=  counts=    max |diff|      tol  result
1. Hourly regional energy (kWh)          1680   True     True   0.000000000    0.001  PASS
2. Daily network energy (kWh)               7   True     True   0.000000005    0.001  PASS
3. Network energy per seven-day period (kWh)      1   True     True   0.000000012    0.001  PASS
4. Daily energy for selected meter (kWh)          7   True     True   0.000000000    0.001  PASS
5. Completeness by region (%)              10   True     True   0.000000000    1e-06  PASS

Per-panel CSVs saved in results\ (7_3_panel<N>_raw.csv / _agg.csv). Done 2026-10-14 15:53:25 CAT
PS C:\Users\ebyiringiro\smart-grid>
```

Screenshot 7.3-1 (Screenshot 2026-10-14 155336.png)

7.4 Measure performance and judge the tradeoff

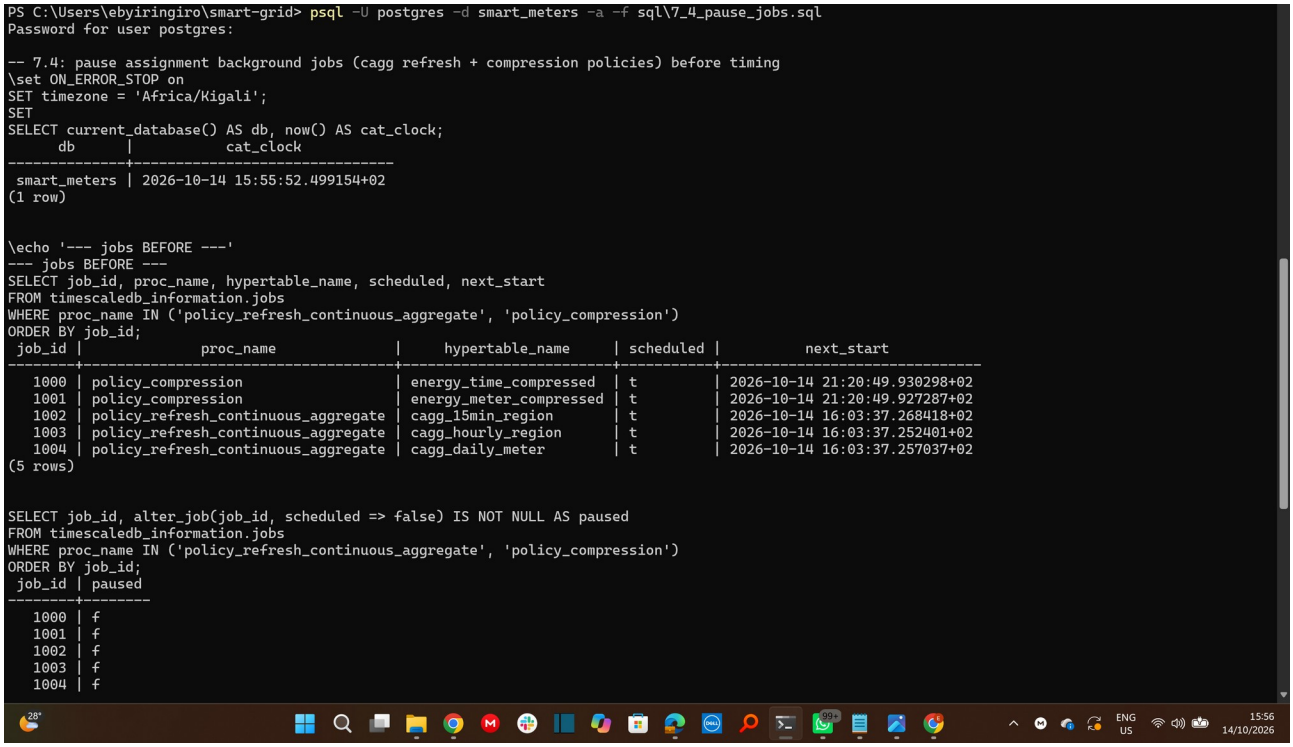
Method: one warm-up refresh followed by ten measured refreshes per dashboard, same browser (Google Chrome 155, driven by Playwright 1600x1500 viewport), same machine and connection, browser cache disabled. Grafana OSS has no query-result cache to disable. The assignment background jobs were paused (15:55 CAT) and resumed after the measurement (16:06 CAT); no other sessions were active. Per-panel duration = browser request start to the last byte of that panel's query response; whole-dashboard time = click on Refresh until all five panel queries finished and two animation frames had been drawn, which approximates the time until all panels finished rendering. Measured 14 Oct 2026, 16:05:00 to 16:05:59 CAT; per-run values are in results\7_4_runs.csv.

Item	Raw median (min-max) ms	Aggregate median (min-max) ms	Improvement
1 Hourly regional	1,003.1 (930.7-1,100.0)	26.8 (24.4-41.0)	97.3%
2 Daily network	695.4 (620.2-845.0)	21.5 (16.6-29.9)	96.9%
3 Seven-day period	951.8 (872.7-1,126.6)	22.2 (16.6-34.7)	97.7%
4 Selected meter	20.3 (16.2-26.4)	18.2 (14.3-31.1)	10.3% (ranges overlap)
5 Completeness	557.5 (412.2-709.5)	19.2 (16.8-21.3)	96.5%
Whole dashboard	1,065.0 (999.1-1,190.6)	108.2 (87.9-123.5)	89.8% (about 9.8 times)

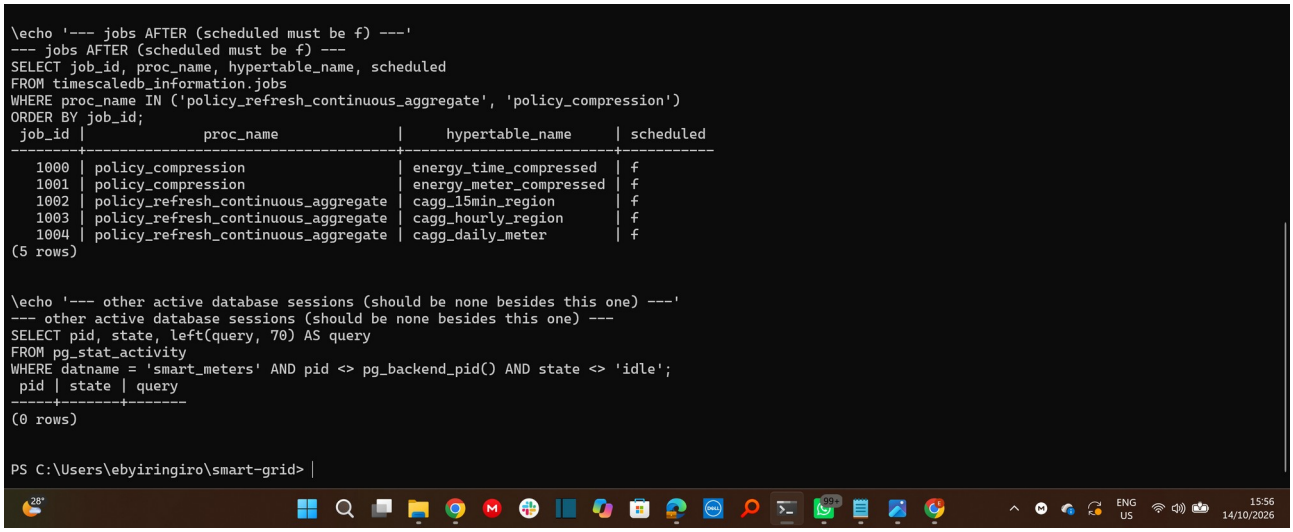
Observation. Panels 1, 2, 3 and 5 are 29 to 43 times faster from the aggregates; panel 4 shows no meaningful gain because the ranges overlap. The panels load in parallel, so the Raw total (1,065 ms) is close to the slowest panel (1,003 ms), not the sum of medians (3,228 ms). Aggregate mode still takes about 108 ms in total although no panel exceeds 27 ms, leaving about 80 ms of fixed browser and render overhead.

Cost. The three aggregates took 107.8 s to refresh once (6.1) and occupy 18,866,176 bytes (about 18.0 MB, about 1.3% of the 1,356.7 MB baseline table). The refresh policies were cheap when nothing needed refreshing (about 0.07 s per run in the first runs).

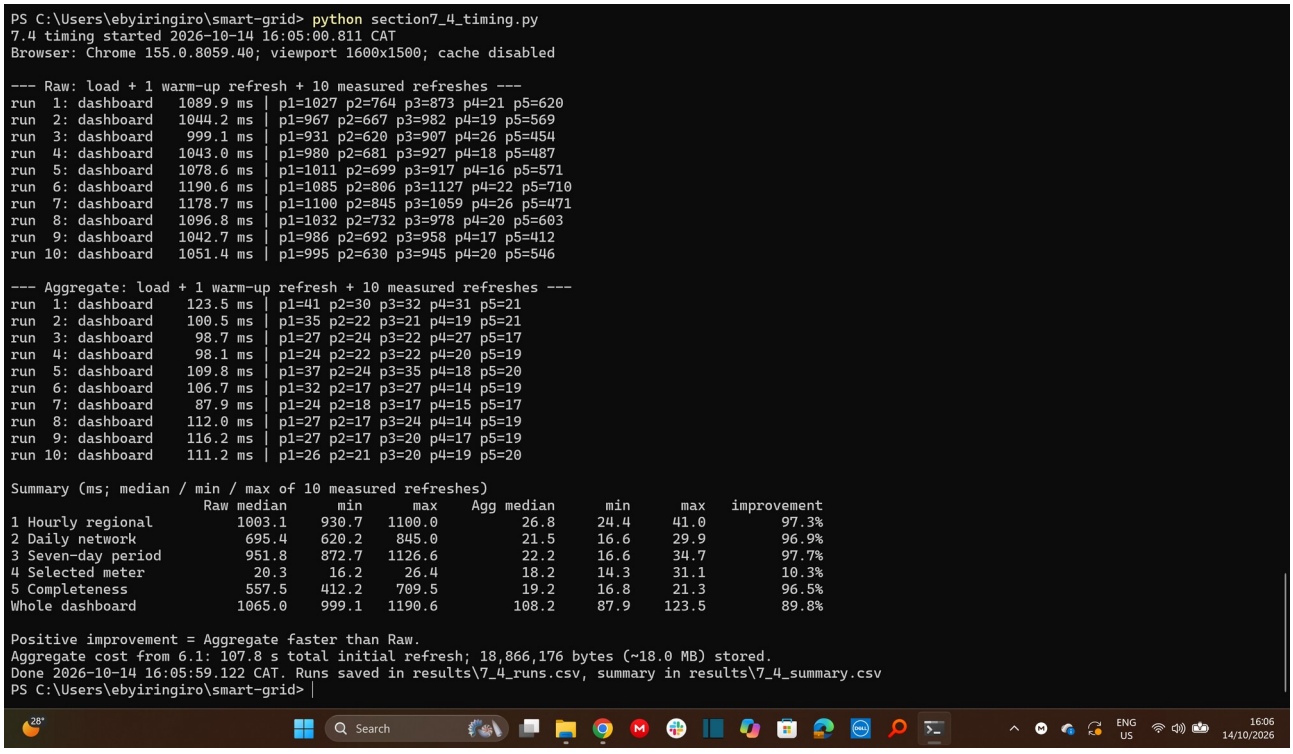
Judgment. Yes for the regional and network-wide panels: about 957 ms is saved per dashboard refresh for 18 MB and a one-time 108 s refresh (repaid after about 113 refreshes as an illustration, though server work and interactive latency are different units). The daily per-meter aggregate is hard to justify for this dashboard: it holds about 70% of the storage and 30% of the refresh time but saves about 2 ms on panel 4. Limits: Raw was timed before Aggregate, measurements are warm-cache with an 8 GB shared_buffers, and the dashboard covers 7 of 28 days; a longer range would probably widen the gap but was not tested.



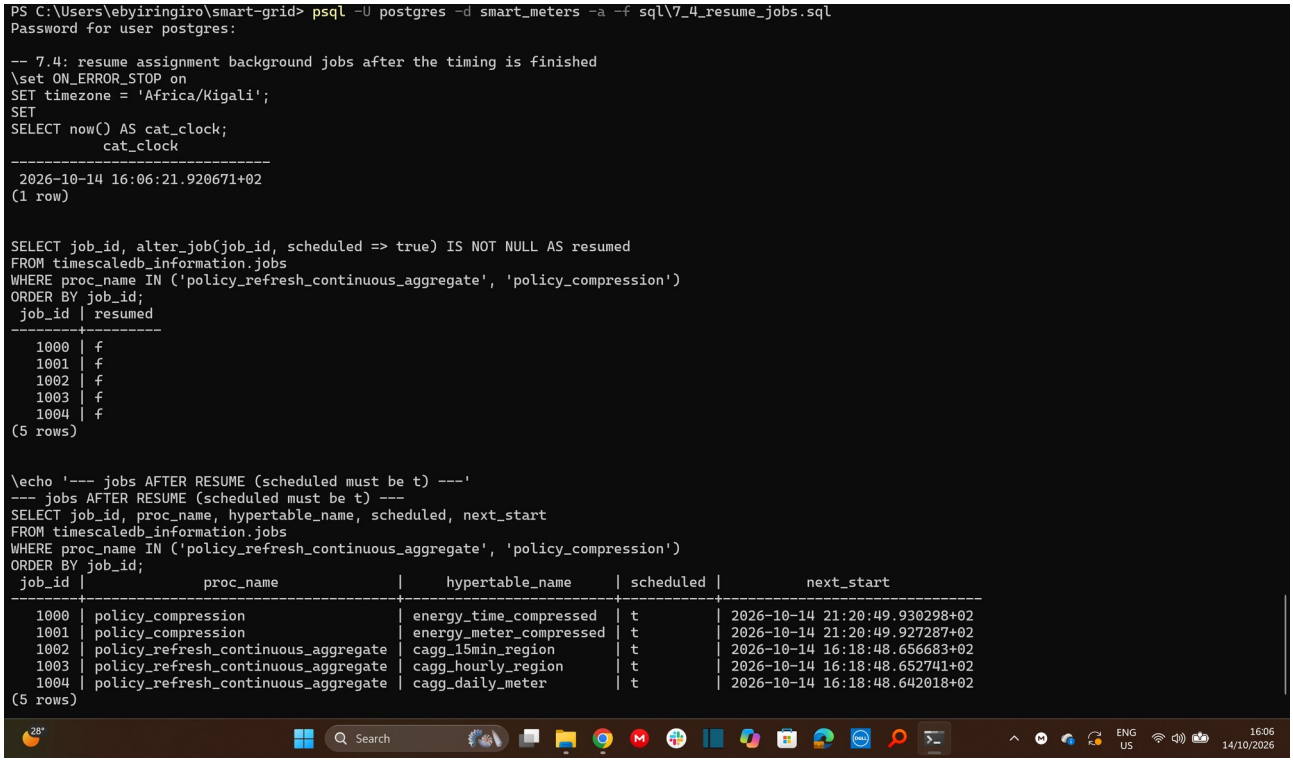
Screenshot 7.4-1 (Screenshot 2026-10-14 155649.png)



Screenshot 7.4-2 (Screenshot 2026-10-14 155701.png)



Screenshot 7.4-3 (Screenshot 2026-10-14 160606.png)



Screenshot 7.4-4 (Screenshot 2026-10-14 160630.png)

8 Forecast the next day

8.1 Prepare chronological data and features

Dataset: one row per region and CAT hour, summing energy_kwh from the cleaned readings (target, kWh). 6,720 of 6,720 region-hours are present (no absent hours); readings per region-hour range from 1,562 to 1,597 (mean 1,583.9; a full hour is 1,600) and total 10,643,735. Totals are observed sums; absent hours would stay absent and a missing predictor would be imputed with the training median (none occurred).

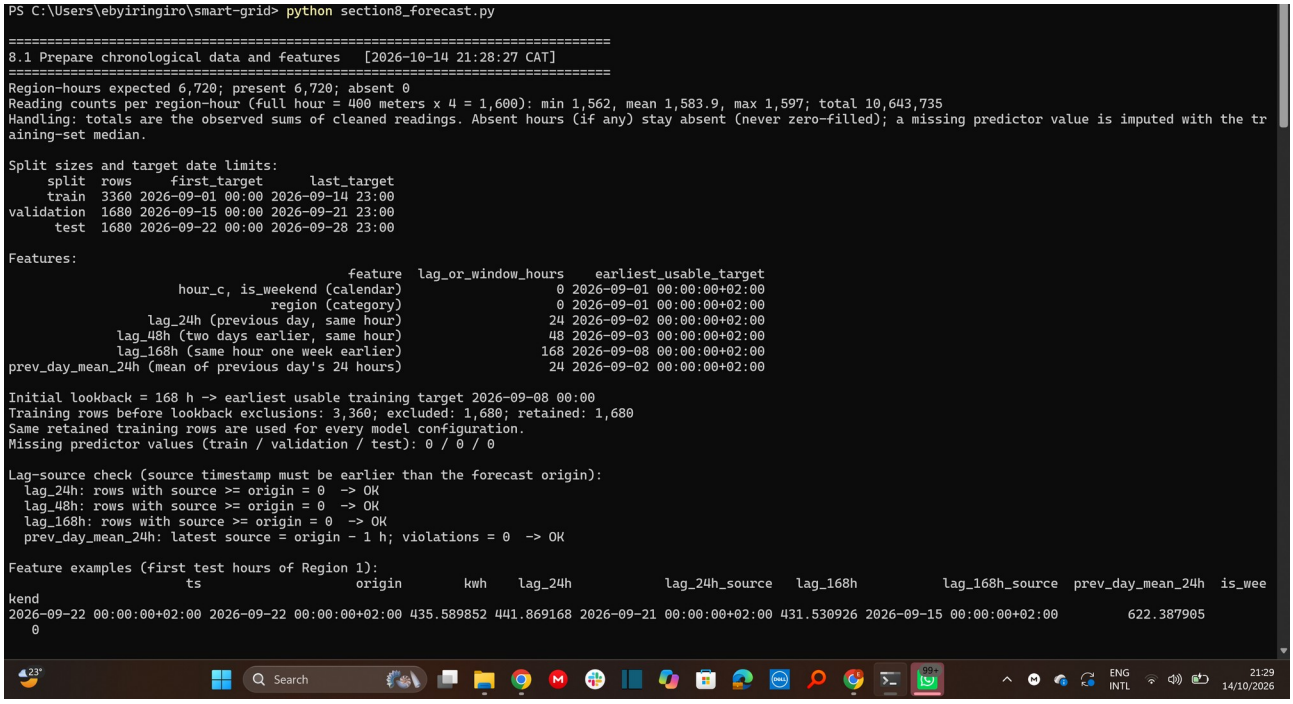
Split	Target dates	Rows	First target	Last target
train	1-14 Sep	3,360	2026-09-01 00:00	2026-09-14 23:00
validation	15-21 Sep	1,680	2026-09-15 00:00	2026-09-21 23:00
test	22-28 Sep	1,680	2026-09-22 00:00	2026-09-28 23:00

For each target hour the forecast origin is 00:00 at the start of its day. Preprocessing (median imputation, standardization of the numeric lags, one-hot encoding of region and hour) was fitted on training rows only.

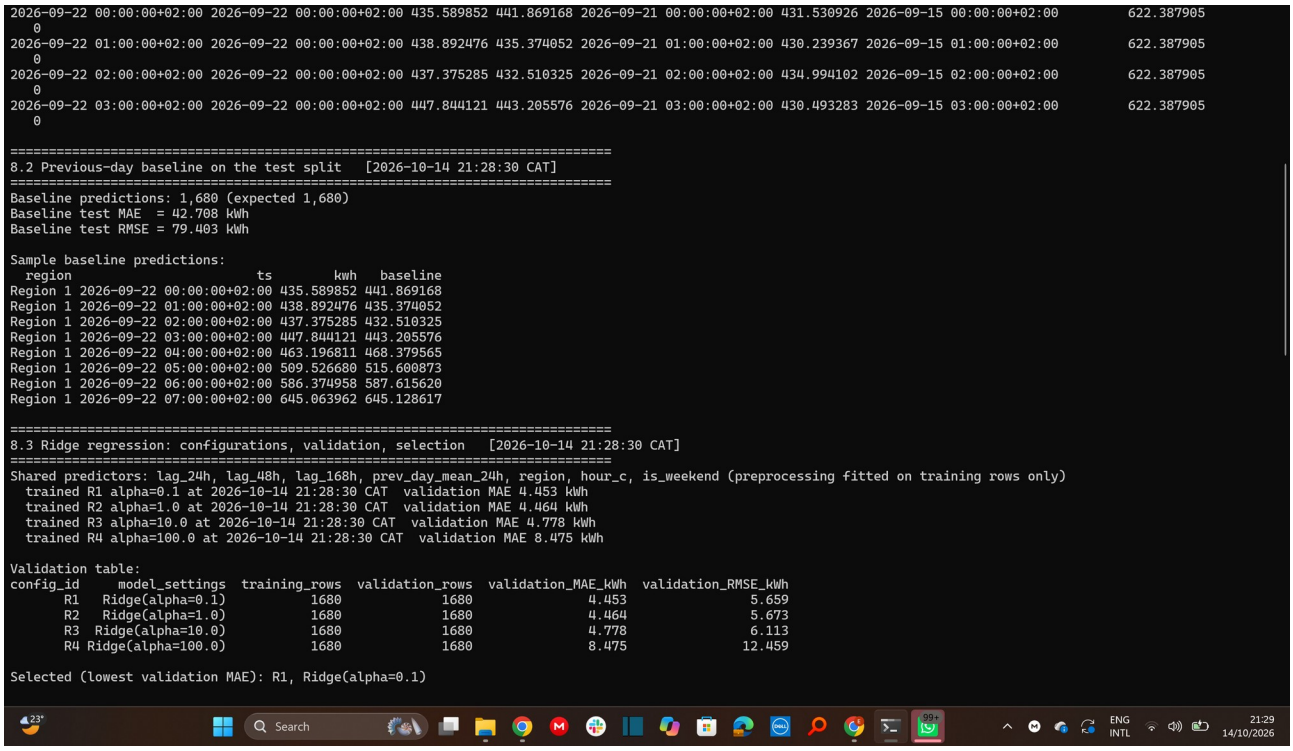
Feature	Lag / window (h)	Earliest usable target
hour of day, weekend flag (calendar)	0	2026-09-01 00:00
region	0	2026-09-01 00:00
lag_24h (previous day, same hour)	24	2026-09-02 00:00
lag_48h (two days earlier, same hour)	48	2026-09-03 00:00
lag_168h (same hour one week earlier)	168	2026-09-08 00:00
prev_day_mean_24h (mean of the previous day's 24 hours)	24	2026-09-02 00:00

Initial lookback = 168 h (the longest lag), so the earliest usable training target is 2026-09-08 00:00. Training rows before exclusions: 3,360; excluded: 1,680; retained: 1,680. The same retained rows are used for every configuration. Missing predictor values: 0 in train, validation and test.

Lag check: for lag_24h, lag_48h and lag_168h the number of rows whose source timestamp is not earlier than the forecast origin is 0; prev_day_mean_24h uses only the previous day (latest source = origin - 1 h), 0 violations.



Screenshot 8.1-1 (Screenshot 2026-10-14 212920.png)

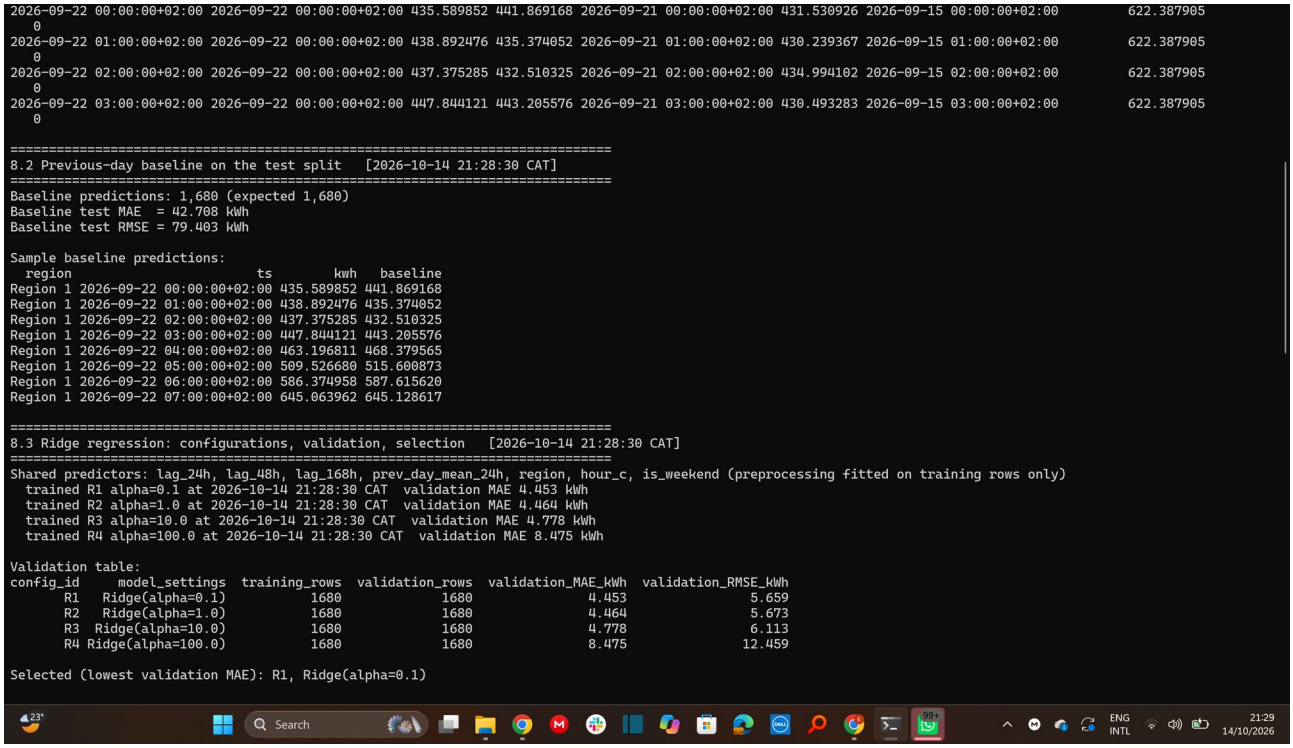


Screenshot 8.1-2 (Screenshot 2026-10-14 212937.png)

8.2 Evaluate the previous-day baseline

At 00:00 CAT of each test day the 24 hourly values were forecast with the previous day's corresponding hour, for all ten regions: 1,680 test predictions.

Method	Predictions	MAE (kWh)	RMSE (kWh)
Previous-day baseline	1,680	42.708	79.403



Screenshot 8.2-1 (Screenshot 2026-10-14 212937.png)

8.3 Train and evaluate one regression model

Model: Ridge regression. Shared predictors: lag_24h, lag_48h, lag_168h, prev_day_mean_24h, region, hour of day and weekend flag. Four configurations differ only in the regularization strength alpha.

Config	Model settings	Training rows	Validation rows	Validation MAE (kWh)	Validation RMSE (kWh)
R1	Ridge(alpha=0.1)	1,680	1,680	4.453	5.659
R2	Ridge(alpha=1.0)	1,680	1,680	4.464	5.673
R3	Ridge(alpha=10.0)	1,680	1,680	4.778	6.113
R4	Ridge(alpha=100.0)	1,680	1,680	8.475	12.459

R1 has the lowest validation MAE and was frozen (model and preprocessing fitted on training rows only, not refitted on validation or test data). The same 1,680 test predictions as the baseline were generated using only information available before each daily origin; earlier completed test days supplied later lag values but were not used for fitting.

```
2026-09-22 00:00:00+02:00 2026-09-22 00:00:00+02:00 435.589852 441.869168 2026-09-21 00:00:00+02:00 431.530926 2026-09-15 00:00:00+02:00         622.387905
   0
2026-09-22 01:00:00+02:00 2026-09-22 00:00:00+02:00 438.892476 435.374052 2026-09-21 01:00:00+02:00 430.239367 2026-09-15 01:00:00+02:00         622.387905
   0
2026-09-22 02:00:00+02:00 2026-09-22 00:00:00+02:00 437.375285 432.510325 2026-09-21 02:00:00+02:00 434.994102 2026-09-15 02:00:00+02:00         622.387905
   0
2026-09-22 03:00:00+02:00 2026-09-22 00:00:00+02:00 447.844121 443.205576 2026-09-21 03:00:00+02:00 430.493283 2026-09-15 03:00:00+02:00         622.387905
   0

================================================================================
8.2 Previous-day baseline on the test split   [2026-10-14 21:28:30 CAT]
================================================================================
Baseline predictions: 1,680 (expected 1,680)
Baseline test MAE  = 42.708 kWh
Baseline test RMSE = 79.403 kWh

Sample baseline predictions:
  region                    ts        kwh   baseline
Region 1 2026-09-22 00:00:00+02:00 435.589852 441.869168
Region 1 2026-09-22 01:00:00+02:00 438.892476 435.374052
Region 1 2026-09-22 02:00:00+02:00 437.375285 432.510325
Region 1 2026-09-22 03:00:00+02:00 447.844121 443.205576
Region 1 2026-09-22 04:00:00+02:00 463.196811 468.379565
Region 1 2026-09-22 05:00:00+02:00 509.526680 515.600873
Region 1 2026-09-22 06:00:00+02:00 586.374958 587.615620
Region 1 2026-09-22 07:00:00+02:00 645.063962 645.128617

================================================================================
8.3 Ridge regression: configurations, validation, selection   [2026-10-14 21:28:30 CAT]
================================================================================
Shared predictors: lag_24h, lag_48h, lag_168h, prev_day_mean_24h, region, hour_c, is_weekend (preprocessing fitted on training rows only)
  trained R1 alpha=0.1 at 2026-10-14 21:28:30 CAT  validation MAE 4.453 kWh
  trained R2 alpha=1.0 at 2026-10-14 21:28:30 CAT  validation MAE 4.464 kWh
  trained R3 alpha=10.0 at 2026-10-14 21:28:30 CAT  validation MAE 4.778 kWh
  trained R4 alpha=100.0 at 2026-10-14 21:28:30 CAT  validation MAE 8.475 kWh

Validation table:
config_id     model_settings  training_rows  validation_rows  validation_MAE_kWh  validation_RMSE_kWh
       R1   Ridge(alpha=0.1)           1680             1680               4.453                5.659
       R2   Ridge(alpha=1.0)           1680             1680               4.464                5.673
       R3  Ridge(alpha=10.0)           1680             1680               4.778                6.113
       R4 Ridge(alpha=100.0)           1680             1680               8.475               12.459

Selected (lowest validation MAE): R1, Ridge(alpha=0.1)
```

Screenshot 8.3-1 (Screenshot 2026-10-14 212937.png)

```
Model and preprocessing are frozen; they are not refitted on validation or test data.

Model test predictions: 1,680 (expected 1,680)
Sample test predictions:
  region                    ts        kwh   baseline      model
Region 1 2026-09-22 00:00:00+02:00 435.589852 441.869168 439.884751
Region 1 2026-09-22 01:00:00+02:00 438.892476 435.374052 439.958670
Region 1 2026-09-22 02:00:00+02:00 437.375285 432.510325 442.935017
Region 1 2026-09-22 03:00:00+02:00 447.844121 443.205576 440.041678
Region 1 2026-09-22 04:00:00+02:00 463.196811 468.379565 467.970970
Region 1 2026-09-22 05:00:00+02:00 509.526680 515.600873 512.280368
Region 1 2026-09-22 06:00:00+02:00 586.374958 587.615620 585.280249
Region 1 2026-09-22 07:00:00+02:00 645.063962 645.128617 653.566180

================================================================================
8.4 Judge forecast usefulness (error = predicted - actual, kWh)   [2026-10-14 21:28:30 CAT]
================================================================================
Overall over 1,680 test observations:
  method  MAE_kWh  RMSE_kWh
baseline   42.708    79.403
   model    4.554     5.786
MAE change vs baseline: +89.3% | RMSE change: +92.7% (positive = model better)

Per region:
   region   n  base_MAE  base_RMSE  model_MAE  model_RMSE  MAE_improvement_%
 Region 1 168    44.873     83.696      4.720       5.910             89.482
 Region 2 168    41.629     76.811      4.538       5.675             89.099
 Region 3 168    42.754     77.607      4.291       5.593             89.964
 Region 4 168    34.856     64.233      4.070       5.149             88.325
 Region 5 168    44.400     81.084      5.218       6.569             88.247
 Region 6 168    39.426     74.766      3.679       4.691             90.668
 Region 7 168    39.688     73.945      4.398       5.705             88.919
 Region 8 168    55.967    102.057      5.706       7.080             89.806
 Region 9 168    43.772     80.178      4.664       5.800             89.346
Region 10 168    39.717     74.282      4.257       5.324             89.282

MAE by test day (explains where the baseline fails):
         day_s  base_MAE  model_MAE
2026-09-22 Tue     4.991      4.548
2026-09-23 Wed     4.652      4.650
2026-09-24 Thu     5.311      5.181
2026-09-25 Fri     5.008      4.962
2026-09-26 Sat   135.695      3.645
2026-09-27 Sun     3.840      4.003
```

Screenshot 8.3-2 (Screenshot 2026-10-14 212954.png)

8.4 Judge forecast usefulness

Method	MAE (kWh)	RMSE (kWh)
Previous-day baseline	42.708	79.403
Ridge model (R1)	4.554	5.786
Model change vs baseline	-89.3%	-92.7%

Error = predicted - actual (kWh); MAE is the mean absolute error and RMSE the square root of the mean squared error, both in kWh, over all 1,680 test observations.

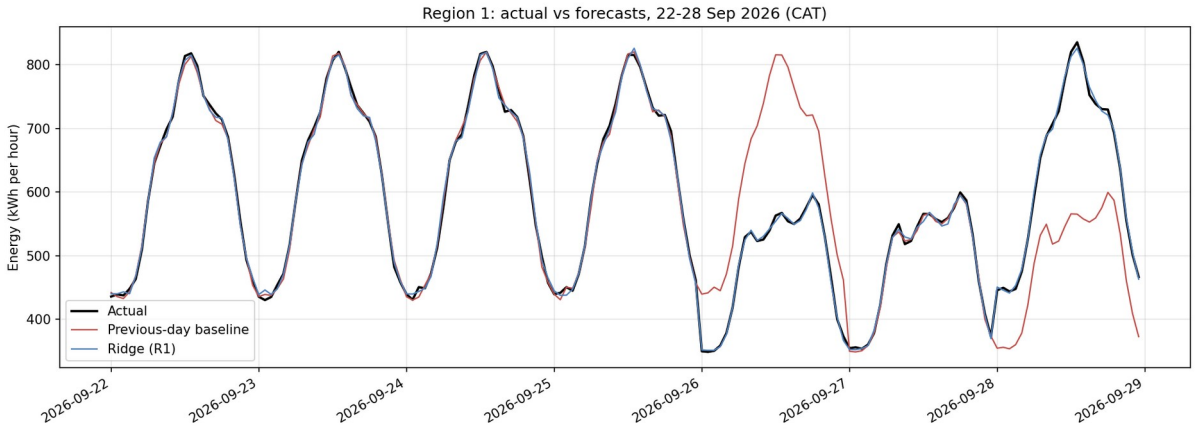
Region	n	Baseline MAE	Baseline RMSE	Model MAE	Model RMSE	MAE improvement %
Region 1	168	44.873	83.696	4.720	5.910	89.482
Region 2	168	41.629	76.811	4.538	5.675	89.099
Region 3	168	42.754	77.607	4.291	5.593	89.964
Region 4	168	34.856	64.233	4.070	5.149	88.325

Region 5	168	44.400	81.084	5.218	6.569	88.247
Region 6	168	39.426	74.766	3.679	4.691	90.668
Region 7	168	39.688	73.945	4.398	5.705	88.919
Region 8	168	55.967	102.057	5.706	7.080	89.806
Region 9	168	43.772	80.178	4.664	5.800	89.346
Region 10	168	39.717	74.282	4.257	5.324	89.282

Test day	Baseline MAE (kWh)	Model MAE (kWh)
2026-09-22 Tue	4.991	4.548
2026-09-23 Wed	4.652	4.650
2026-09-24 Thu	5.311	5.181
2026-09-25 Fri	5.008	4.962
2026-09-26 Sat	135.695	3.645
2026-09-27 Sun	3.840	4.003
2026-09-28 Mon	139.461	4.889

Highest actual test-hour consumption per region (ties to the earliest hour)

Region	Peak hour	Actual kWh	Baseline pred.	Baseline abs err	Model pred.	Model abs err
Region 1	2026-09-28 13:00	835.61	565.27	270.34	826.29	9.32
Region 2	2026-09-28 13:00	780.13	546.91	233.21	773.78	6.34
Region 3	2026-09-28 13:00	783.00	554.69	228.32	790.33	7.33
Region 4	2026-09-28 18:00	659.90	561.41	98.50	655.56	4.34
Region 5	2026-09-28 13:00	828.23	577.36	250.87	826.02	2.21
Region 6	2026-09-28 13:00	749.11	504.41	244.70	730.97	18.14
Region 7	2026-09-25 13:00	751.05	736.64	14.41	743.29	7.76
Region 8	2026-09-25 13:00	1,016.55	1,010.53	6.02	1,011.78	4.77
Region 9	2026-09-23 13:00	808.96	796.89	12.07	803.95	5.01
Region 10	2026-09-28 13:00	743.03	502.33	240.71	734.61	8.42



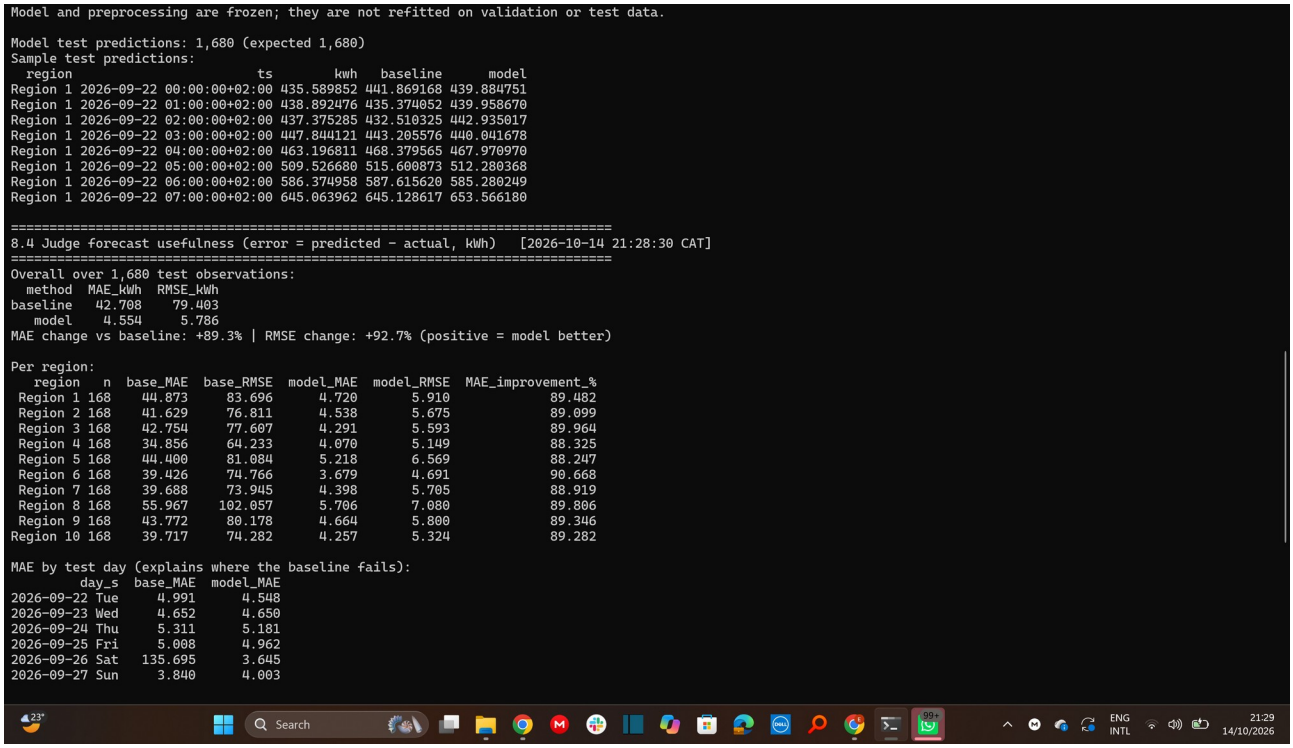
Region 1: actual, baseline and model consumption, 22-28 Sep 2026

Does the model improve on the baseline? Yes. Test MAE is 4.554 kWh against 42.708 kWh (89.3% lower) and RMSE 5.786 against 79.403 kWh (92.7% lower); every region improves (MAE 88.2-90.7% lower) and the validation MAE (4.453 kWh) is close to the test MAE, so there is no sign of overfitting. The gain is concentrated on Saturday 26 and Monday 28 September (baseline MAE 135.7 and 139.5 kWh against 3.6 and 4.9 kWh), where the baseline copies a weekend day onto a weekday (explanation, not separately tested). On the other five days the methods are within about 0.5 kWh and on Sunday the baseline is slightly better (3.84 against 4.00 kWh). A same-weekday-last-week baseline was not tested and might narrow the gap.

Is it useful for planning? A 4.55 kWh MAE is about 0.8% of the mean test-hour regional consumption (about 591 kWh), accurate enough for day-ahead planning. At each region's peak hour the model's absolute error was 2.2 to 18.1 kWh (at most 2.4% of the peak) against up to 270.3 kWh for the baseline. Seven of the ten peaks fall on Monday 28 September. The model under-predicted nine of the ten peaks, so keep a small margin above the forecast (cause not tested).

Missing readings and delayed arrivals. About 1% of readings never arrive, so the totals are observed consumption, a lower bound on true use. In a live forecast at 00:00 the previous day's late-evening readings may still be in transit, because delays reach three hours (in 5.3, 531 rows or 0.10% of 21 September arrived after midnight), so the lag features would be slightly understated and live errors could exceed these offline results, which treat history as finalized.

Limits. One test week, 1,680 training rows after the lookback, simulated data and no holidays; confirm on more weeks before relying on the model.



Screenshot 8.4-1 (Screenshot 2026-10-14 212954.png)

9 Submit the report and code

9.1 Organize the report

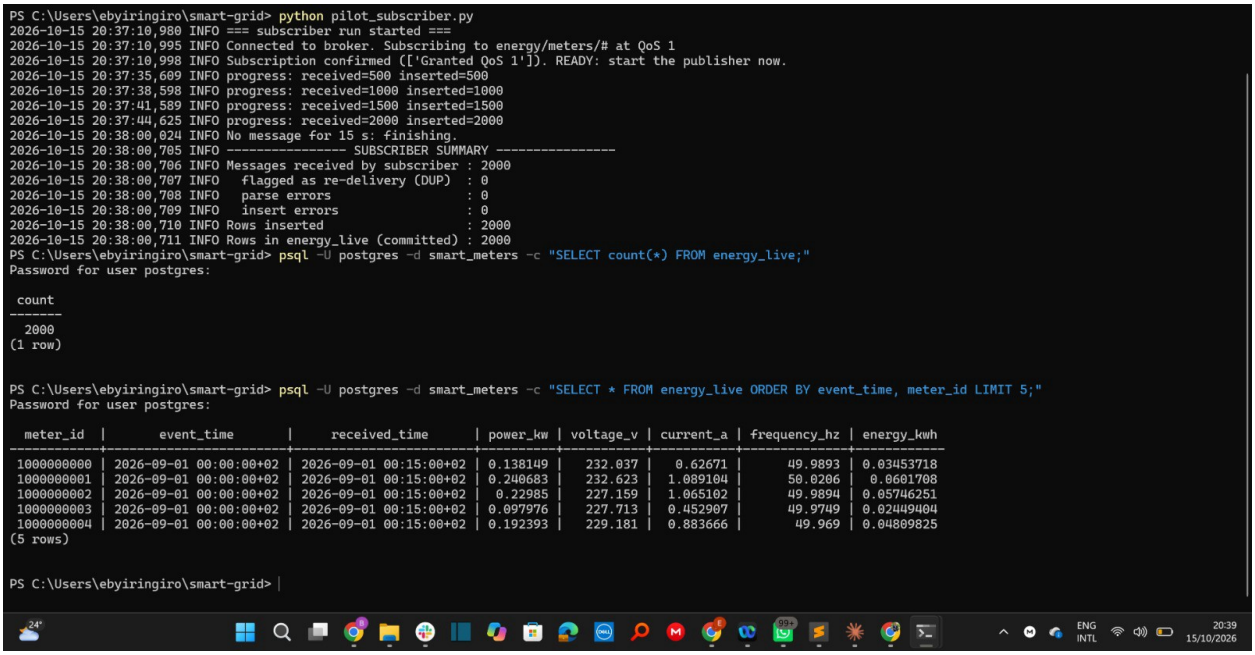
This report follows the subsection order of the assignment. Required screenshots are embedded beside their subsections with the command or action, result, taskbar and clock visible; where a later subsection asks only for interpretation, earlier evidence is reused.

9.2 Make the work reproducible

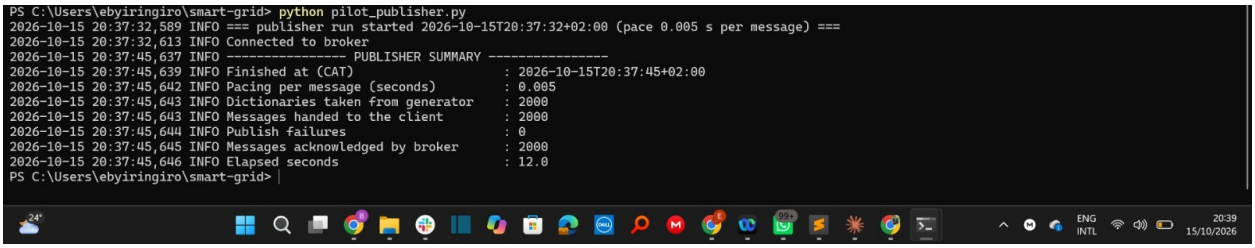
Item	Value
GitHub repository	[[FILL: repository URL]]
ZIP file	[[FILL: ZIP file name]] (same files as the repository)
Generator	smart_meter_simulator.py (generator version 3.0) is submitted unchanged. SHA-256: 6b16d6bb3191e528844d0ec75656096d8bfb2e92c99f83e2b75d586bf21cff8. Run unmodified, it emits 10,696,848 readings, of which 10,643,735 are unique, matching expected_counts in dataset_info().
README	README.md gives the execution order, commands, placeholders for secrets and the regeneration steps
Full dataset	kept local (data\readings.csv, about 10.7 million rows); regenerate with capture_to_csv.py and load_csv.py

Repository contents: unchanged generator; capture and ingestion programs (load_meters.py, pilot_publisher.py, pilot_subscriber.py, analyse_losses.py, capture_to_csv.py, load_csv.py); SQL files (sql\); benchmark and compression programs (benchmark.py, compress_load.py, improvement.py, section4_facts.py); analysis and model code (section5_analysis.py, section5_4_anomaly.py, section8_forecast.py); dashboard code and configuration (section7_dashboards.py, section7_4_timing.py, dashboards\Raw.json, dashboards\Aggregate.json); recorded benchmark timings and the validation comparison (results\); environment details (README.md, requirements.txt).

Pilot instructions from the README were run again on 15 Oct 2026 (subscriber started 20:37:10 CAT and READY before the publisher, publisher run 20:37:32-20:37:45 CAT) and the four counts from 1.3 verified: 2,000 dictionaries taken, 2,000 messages acknowledged by the broker, 2,000 messages received by the subscriber (0 DUP, 0 parse errors, 0 insert errors) and 2,000 rows in energy_live, with sample rows shown below.



Screenshot 9.2-1 (subscriber summary, row count and sample rows, 15 Oct 2026 20:39 CAT)



Screenshot 9.2-2 (publisher summary, 15 Oct 2026 20:37 CAT)

9.3 Document contributions

Name	Student ID	Individual contribution
ERIC BYIRINGIRO	101389	Sections 7-9: building and comparing the Raw and Aggregate dashboards (matching results and performance), the next-day forecast (features, baseline, regression model, usefulness), and organizing the report, code and reproducibility.
SANDRINE MFURANZIMA	101411	Sections 4-6: compression strategies (configurations, storage and compression cost, query comparison, storage recommendation), investigation of the utility data (demand patterns, regions and customer types, reporting quality, abnormal consumption, management recommendation), and continuous aggregates including the freshness test.
NDANYUZWE DUNCAN	101413	Sections 1-3: building the ingestion pipeline (environment, database structures, pilot stream), capturing and loading the historical dataset, data-quality investigation and cleaning, and the chunk-interval comparison.

All members are prepared to explain and present the group's work if called upon.

Appendix A: dashboard panel queries

Raw dashboard

Panel 1: Hourly regional energy (kWh)

```
SELECT time_bucket(INTERVAL '1 hour', r.event_time, 'Africa/Kigali') AS "time",
       m.region AS metric,
       sum(r.energy_kwh) AS "Energy (kWh)"
FROM energy_readings r
JOIN meters m ON m.meter_id = r.meter_id
WHERE r.event_time >= '$start_date'::date::timestamp AT TIME ZONE 'Africa/Kigali'
  AND r.event_time < '$end_date'::date::timestamp AT TIME ZONE 'Africa/Kigali'
GROUP BY 1, 2
ORDER BY 1, 2
```

Panel 2: Daily network energy (kWh)

```
SELECT (r.event_time AT TIME ZONE 'Africa/Kigali')::date::text AS day,
       sum(r.energy_kwh) AS "Energy (kWh)"
FROM energy_readings r
WHERE r.event_time >= '$start_date'::date::timestamp AT TIME ZONE 'Africa/Kigali'
  AND r.event_time < '$end_date'::date::timestamp AT TIME ZONE 'Africa/Kigali'
GROUP BY 1
ORDER BY 1
```

Panel 3: Network energy per seven-day period (kWh)

```
SELECT to_char(DATE '2026-09-01' + 7 * (((r.event_time AT TIME ZONE 'Africa/Kigali')::date - DATE '2026-
09-01') / 7), 'YYYY-MM-DD') AS period_start,
       sum(r.energy_kwh) AS "Energy (kWh)"
FROM energy_readings r
WHERE r.event_time >= '$start_date'::date::timestamp AT TIME ZONE 'Africa/Kigali'
  AND r.event_time < '$end_date'::date::timestamp AT TIME ZONE 'Africa/Kigali'
GROUP BY 1
ORDER BY 1
```

Panel 4: Daily energy for selected meter (kWh)

```
SELECT (r.event_time AT TIME ZONE 'Africa/Kigali')::date::text AS day,
       sum(r.energy_kwh) AS "Energy (kWh)"
FROM energy_readings r
WHERE r.meter_id = $meter_id
  AND r.event_time >= '$start_date'::date::timestamp AT TIME ZONE 'Africa/Kigali'
  AND r.event_time < '$end_date'::date::timestamp AT TIME ZONE 'Africa/Kigali'
GROUP BY 1
ORDER BY 1
```

Panel 5: Completeness by region (%)

```
WITH reg AS (SELECT region, count(*) AS meters FROM meters GROUP BY region),
obs AS (
  SELECT m.region, count(*) AS observed
  FROM energy_readings r
  JOIN meters m ON m.meter_id = r.meter_id
  WHERE r.event_time >= '$start_date'::date::timestamp AT TIME ZONE 'Africa/Kigali'
  AND r.event_time < '$end_date'::date::timestamp AT TIME ZONE 'Africa/Kigali'
  GROUP BY m.region)
SELECT reg.region,
       100.0 * obs.observed / (reg.meters * (('$end_date'::date - '$start_date'::date) * 96)) AS
"Completeness (%)"
FROM reg
JOIN obs ON obs.region = reg.region
ORDER BY reg.region
```

Aggregate dashboard

Panel 1: Hourly regional energy (kWh)

```
SELECT c.bucket AS "time",
       c.region AS metric,
       sum(c.sum_energy_kwh) AS "Energy (kWh)"
```

```
FROM cagg_hourly_region c
WHERE c.bucket >= '$start_date'::date::timestamp AT TIME ZONE 'Africa/Kigali'
  AND c.bucket < '$end_date'::date::timestamp AT TIME ZONE 'Africa/Kigali'
GROUP BY 1, 2
ORDER BY 1, 2
```

Panel 2: Daily network energy (kWh)

```
SELECT (c.bucket AT TIME ZONE 'Africa/Kigali')::date::text AS day,
       sum(c.sum_energy_kwh) AS "Energy (kWh)"
FROM cagg_15min_region c
WHERE c.bucket >= '$start_date'::date::timestamp AT TIME ZONE 'Africa/Kigali'
  AND c.bucket < '$end_date'::date::timestamp AT TIME ZONE 'Africa/Kigali'
GROUP BY 1
ORDER BY 1
```

Panel 3: Network energy per seven-day period (kWh)

```
SELECT to_char(DATE '2026-09-01' + 7 * (((c.bucket AT TIME ZONE 'Africa/Kigali')::date - DATE '2026-09-
01') / 7), 'YYYY-MM-DD') AS period_start,
       sum(c.sum_energy_kwh) AS "Energy (kWh)"
FROM cagg_15min_region c
WHERE c.bucket >= '$start_date'::date::timestamp AT TIME ZONE 'Africa/Kigali'
  AND c.bucket < '$end_date'::date::timestamp AT TIME ZONE 'Africa/Kigali'
GROUP BY 1
ORDER BY 1
```

Panel 4: Daily energy for selected meter (kWh)

```
SELECT (c.bucket AT TIME ZONE 'Africa/Kigali')::date::text AS day,
       sum(c.sum_energy_kwh) AS "Energy (kWh)"
FROM cagg_daily_meter c
WHERE c.meter_id = $meter_id
  AND c.bucket >= '$start_date'::date::timestamp AT TIME ZONE 'Africa/Kigali'
  AND c.bucket < '$end_date'::date::timestamp AT TIME ZONE 'Africa/Kigali'
GROUP BY 1
ORDER BY 1
```

Panel 5: Completeness by region (%)

```
WITH reg AS (SELECT region, count(*) AS meters FROM meters GROUP BY region),
obs AS (
  SELECT c.region, sum(c.reading_count) AS observed
  FROM cagg_15min_region c
  WHERE c.bucket >= '$start_date'::date::timestamp AT TIME ZONE 'Africa/Kigali'
  AND c.bucket < '$end_date'::date::timestamp AT TIME ZONE 'Africa/Kigali'
  GROUP BY c.region)
SELECT reg.region,
       100.0 * obs.observed / (reg.meters * (('$end_date'::date - '$start_date'::date) * 96)) AS
"Completeness (%)"
FROM reg
JOIN obs ON obs.region = reg.region
ORDER BY reg.region
```